

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

Khoa Công nghệ Thông tin

Intelligent System Development · Assignment 03

Deep learning viết tay với NumPy và ba mô hình ML cho mỗi category

Nguyễn Bá Hùng

Mã sinh viên: B23DCAT120 · Lớp: E23CNPM01 · Nhóm: 02

Giảng viên hướng dẫn: Dinh Que Tran, Ph.D., Assoc. Prof.

Học kỳ I.2026

Tóm tắt

Báo cáo dựng lại một mạng nơ-ron nhiều lớp chỉ bằng NumPy: lan truyền xuôi, binary cross-entropy, lan truyền ngược theo quy tắc chuỗi và gradient descent đều viết tay, không dùng TensorFlow, PyTorch, Keras hay scikit-learn. Bốn kiến trúc được huấn luyện trên cùng một tập dữ liệu, cùng một seed khởi tạo và cùng một quy trình đo, nên khác biệt giữa chúng chỉ đến từ kiến trúc. Mô hình tốt nhất theo ROC-AUC là M2 Wider với 0.8076.

Bốn chương sau mở rộng phạm vi ra cả ba category của dự án, để mỗi category có đủ ba mô hình machine learning và một mô hình deep learning. Hai mô hình ML được bổ sung là KNN và AdaBoost, lấp hai họ thuật toán chưa từng xuất hiện trong dự án. Hai mạng nơ-ron được bổ sung, cũng viết tay bằng NumPy, phủ bài toán hồi quy giá nhà và bài toán văn bản e-commerce, trong đó ma trận nhúng từ được huấn luyện bằng lan truyền ngược thay vì để ngẫu nhiên cố định.

Mục lục

Mục lục	2
1. Bài toán và phạm vi	3
2. Biểu diễn dữ liệu	4
3. Mạng nơ-ron viết tay	5
3.1 Lan truyền xuôi	5
3.2 Hàm mất mát	5
3.3 Lan truyền ngược	5
3.4 Cập nhật tham số	5
4. Mô hình M1 Baseline	6
5. Mô hình M2 Wider	7
6. Mô hình M3 Shallow	8
7. Mô hình M4 No ReLU	9
8. So sánh bốn mô hình	10
8.1 Học suất và công thức gốc của tài liệu	10
8.2 Điều bốn mô hình nói lên	11
9. Ba mô hình machine learning cho mỗi category	12
9.1 Ba bài toán phân loại	12
9.2 Bài toán hồi quy	13
10. Deep learning cho category giá nhà	14
11. Deep learning cho category e-commerce	16
11.1 Tách bạch hai nguồn đóng góp	16
11.2 So với mô hình machine learning tốt nhất	17
12. Tổng hợp: ba mô hình ML và một mô hình DL cho mỗi category	18
13. Kết luận và giới hạn	19
Phụ lục A. Notebook 04: Deep Learning từ đầu	20
Phụ lục B. Notebook 05: K-Nearest Neighbors	32
Phụ lục C. Notebook 06: AdaBoost	40
Phụ lục D. Notebook 07: Mạng nơ-ron hồi quy cho giá nhà	48
Phụ lục E. Notebook 08: Mạng nơ-ron có lớp nhúng cho e-commerce	57
Phụ lục F. Mã nguồn dùng chung và kịch bản dựng báo cáo	67

1. Bài toán và phạm vi

Deep Learning là phép hợp thành của các hàm có tham số. Thay vì học một ánh xạ trực tiếp từ đầu vào sang nhãn, mô hình dựng nhiều phép biến đổi nối tiếp rồi học tham số của chúng.

$$\hat{y} = f_3(f_2(f_1(X))) = \sigma(\text{ReLU}(\text{ReLU}(XW_1 + b_1)W_2 + b_2)W_3 + b_3)$$

Bài toán là phân loại nhị phân nguy cơ tiểu đường từ bộ khảo sát BRFSS 2015. Đầu vào là tám chỉ số sức khoẻ, đầu ra là một xác suất. Ràng buộc của bài là không dùng framework học sâu nào: mọi phép toán của mạng, kể cả đạo hàm, đều viết bằng NumPy.

01 Biểu diễn <i>8 đặc trưng, chuẩn hoá z</i>	02 Mạng NumPy <i>forward, loss, backward</i>	03 Huấn luyện <i>gradient descent lô nhỏ</i>	04 Đánh giá <i>4 mô hình, 3 ngưỡng</i>
---	---	---	---

Bảng 1: Bốn kiến trúc được so sánh

Mô hình	Kiến trúc	Kích hoạt lớp ẩn	Tham số	Mục đích
M1 Baseline	8 - 16 - 8 - 1	ReLU	289	Kiến trúc gốc của bài giảng, dùng làm mốc so sánh cho ba mô hình còn lại.
M2 Wider	8 - 32 - 16 - 1	ReLU	833	Nhân đôi bề rộng cả hai lớp ẩn, số tham số tăng gần ba lần.
M3 Shallow	8 - 16 - 1	ReLU	161	Bỏ hẳn lớp ẩn thứ hai, chỉ còn một tầng biểu diễn.
M4 No ReLU	8 - 16 - 8 - 1	tuyến tính	289	Giữ nguyên ba lớp nhưng thay ReLU bằng hàm đồng nhất.

Cả bốn dùng chung seed khởi tạo 42 và seed xáo trộn 7, cùng học suất 0.05, cùng 60 epoch và cùng lô 512 dòng.

Khoảng cách giữa mô hình tốt nhất và kém nhất theo ROC-AUC chỉ là 0.0082. Đây là kết quả đáng chú ý nhất của báo cáo và sẽ được giải thích ở mục 8: với tám đặc trưng bảng đã chuẩn hoá, phần lớn tín hiệu là tuyến tính, nên chiều sâu và bề rộng mang lại rất ít.

2. Biểu diễn dữ liệu

Bài giảng dùng bộ Pima với đúng tám biến đầu vào. Bộ BRFSS ở đây có 21 biến, nên phải chọn ra tám. Tiêu chí chọn là trị tuyệt đối hệ số tương quan với nhân nhị phân, xếp hạng đầy đủ 21 biến rồi lấy tám biến đầu. Cách chọn này đơn giản và kiểm chứng được, nhưng chỉ bắt được quan hệ tuyến tính đơn biến.

Bảng 2: Tám đặc trưng được chọn

Hạng	Đặc trưng	tương quan
1	GenHlth	0.2832
2	HighBP	0.2606
3	BMI	0.2112
4	DiffWalk	0.2092
5	HighChol	0.2045
6	Age	0.1858
7	HeartDiseaseorAttack	0.1671
8	PhysHlth	0.1592

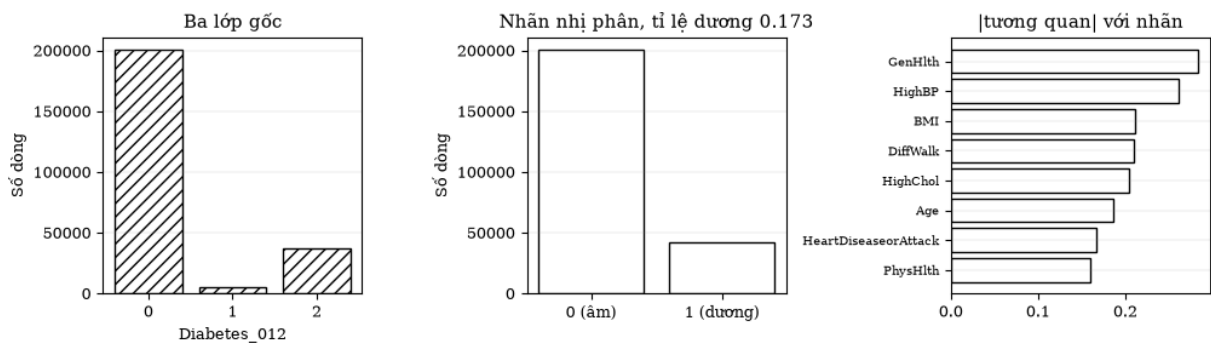
Bốn biến bị loại ngay sau ngưỡng: Income (0.1483), Education (0.1086), PhysActivity (0.1032), Stroke (0.0987).

Nhân nhị phân giữ đúng quy ước của các ứng dụng trước, y bằng 1 khi Diabetes_012 > 0, tức gộp tiền tiểu đường và tiểu đường vào lớp dương. Sau khi bỏ 36,611 dòng trùng hoàn toàn, còn 242,437 dòng, chia 193,949 train và 48,488 test theo tỉ lệ 80 trên 20 với seed 42.

Tỉ lệ lớp dương là 0.1727 trên train và 0.1737 trên test. Đây là bài toán lệch lớp, nên accuracy một mình không nói lên điều gì: đoán toàn lớp âm đã cho accuracy khoảng 0.826.

$$x' = (x - \mu_{train}) / \sigma_{train}$$

Chuẩn hoá dùng trung bình và độ lệch chuẩn tính riêng trên tập train rồi áp lại cho test. Nếu tính trên toàn bộ dữ liệu thì thông tin của tập test đã rò sang lúc huấn luyện, và mọi con số đánh giá sau đó đều lạc quan hơn thực tế.



Hình 1: Phân bố ba lớp gốc, nhân nhị phân dùng để huấn luyện, và thứ hạng tương quan của tám đặc trưng được chọn.

Hình 1 Observation: lớp tiền tiểu đường rất mỏng so với hai lớp còn lại, và sau khi gộp nhị phân thì lớp dương vẫn chỉ chiếm khoảng một phần sáu. **Interpretation:** tín hiệu dương hiếm, mạng sẽ có xu hướng nghiêng về lớp âm để hạ nhanh hàm mất mát. **ML implication:** phải đọc recall và ROC-AUC thay vì accuracy, và ngưỡng quyết định 0.5 không nhất thiết là ngưỡng nên dùng khi triển khai.

3. Mạng nơ-ron viết tay

Toàn bộ mạng gồm bốn phép toán, không có phép nào cần thư viện học sâu.

3.1 Lan truyền xuôi

$$Z_k = H_{k-1} W_k + b_k, \quad H_k = g(Z_k)$$

Hàm g là ReLU ở lớp ẩn và sigmoid ở lớp ra. ReLU đưa phi tuyến vào mạng: thiếu nó, mọi phép xếp chồng tuyến tính vẫn quy về một phép tuyến tính duy nhất. Sigmoid ép đầu ra về khoảng mở $(0, 1)$ để đọc như một xác suất.

3.2 Hàm mất mát

$$L = -(1/N) \sum [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

Xác suất được cắt về khoảng $[1e-8, 1 - 1e-8]$ trước khi lấy log, nếu không một dự đoán tự tin tuyệt đối và sai sẽ cho $\log(0)$ và làm hỏng cả vòng huấn luyện.

3.3 Lan truyền ngược

Ghép sigmoid với binary cross-entropy cho một kết quả rất gọn ở lớp cuối. Đạo hàm của hàm mất mát theo đầu vào của sigmoid không còn chứa đạo hàm sigmoid nữa:

$$\partial L / \partial Z_L = (\hat{y} - y) / N$$

Từ đó gradient lan ngược bằng quy tắc chuỗi, mỗi lớp chỉ cần hai phép nhân ma trận và một phép nhân từng phần tử với đạo hàm của hàm kích hoạt:

$$dW_k = H_{k-1}^T \delta_k, \quad \delta_{k-1} = (\delta_k W_k^T) \odot g'(Z_{k-1})$$

Lan truyền ngược không phải một phép thuật riêng của học sâu. Nó chỉ là quy tắc chuỗi áp cho một hàm hợp thành, thực hiện theo thứ tự ngược để dùng lại kết quả trung gian mà lan truyền xuôi đã lưu.

3.4 Cập nhật tham số

$$W \leftarrow W - \eta \partial L / \partial W$$

Hàm cập nhật trả về bộ tham số mới thay vì sửa tại chỗ, nên mỗi bước huấn luyện là một phép biến đổi rõ ràng từ trạng thái cũ sang trạng thái mới. Cách viết này tốn thêm một lần cấp phát mỗi bước nhưng loại hẳn một lớp lỗi khó tìm khi gỡ rối gradient.

Khác biệt so với công thức trong tài liệu

Tài liệu dùng gradient descent toàn tập, học suất 0.01, 1000 epoch, trên bộ Pima 768 dòng. Tập train ở đây lớn hơn hai trăm lần, nên 1000 epoch toàn tập chỉ tương đương 1000 lần cập nhật tham số và mạng chưa kịp rời khỏi điểm khởi tạo: kết quả đo được là mô hình đoán toàn lớp âm, recall bằng 0. Vì vậy vòng lặp chuyển sang gradient descent theo lô nhỏ 512 dòng. Công thức cập nhật không đổi, chỉ khác ở chỗ gradient tính trên lô thay vì trên toàn tập. Mục 8 giữ nguyên công thức gốc để đối chiếu và đưa số đo cụ thể.

4. Mô hình M1 Baseline

Kiến trúc gốc của bài giảng, dùng làm mốc so sánh cho ba mô hình còn lại. Hai lớp ẩn ReLU tạo ra hai tầng biểu diễn trung gian trước khi sigmoid quy về xác suất.

0.8373	0.2929	0.8071	56s
Accuracy · test	F1 · ngưỡng 0.5	ROC-AUC · test	Thời gian huấn luyện

Bảng 3: Cấu hình và quá trình huấn luyện của M1

Hạng mục	Giá trị
Kiến trúc	8 - 16 - 8 - 1
Kích hoạt lớp ẩn	ReLU
Số tham số học được	289
Kích thước biểu diễn trung gian	H1: (193,949, 16), H2: (193,949, 8)
Học suất, epoch, lô	0.05 · 60 · 512
Số lần cập nhật tham số	22,740
Loss train đầu và cuối	0.3848 → 0.3704
Loss trên tập test	0.3695
Thời gian huấn luyện	55.9 giây

Chênh lệch accuracy giữa train và test là -0.0011, và loss test (0.3695) so với loss train (0.3704) lệch -0.0008. Khoảng cách hẹp như vậy cho thấy mô hình chưa quá khớp, phần thiếu nằm ở sức biểu diễn chứ không ở việc học thuộc dữ liệu.

Bảng 4: Kết quả của M1 trên tập test theo ba ngưỡng quyết định

Ngưỡng	Accuracy	Precision	Recall	F1	TP	FN	FP	TN
0.3	0.8062	0.4489	0.5084	0.4768	4,283	4,141	5,258	34,806
0.5	0.8373	0.5981	0.1940	0.2929	1,634	6,790	1,098	38,966
0.7	0.8282	0.7255	0.0176	0.0343	148	8,276	56	40,008

Hạ ngưỡng từ 0.5 xuống 0.3 nâng recall từ 0.1940 lên 0.5084 và đổi lại precision giảm từ 0.5981 xuống 0.4489, tức bắt thêm 2,649 ca dương thật nhưng báo động nhầm thêm 4,160 ca. Nâng lên 0.7 đẩy recall xuống 0.0176. Với bài sàng lọc bệnh, bỏ sót tổn kém hơn báo động nhầm, nên ngưỡng thấp là lựa chọn hợp lý hơn 0.5.

Đây là mốc tham chiếu. Hai lớp ẩn ReLU dựng được hai tầng biểu diễn, nhưng recall tại ngưỡng 0.5 chỉ ở mức 0.1940: mạng hạ hàm mất mát nhanh nhất bằng cách nghiêng về lớp âm chiếm đa số, đúng như dự đoán từ phân bố lớp.

5. Mô hình M2 Wider

Nhân đôi bề rộng cả hai lớp ẩn, số tham số tăng gần ba lần. Câu hỏi cần trả lời là thêm sức chứa có đổi lấy được chất lượng hay không.

0.8368	0.2753	0.8076	57s
Accuracy · test	F1 · ngưỡng 0.5	ROC-AUC · test	Thời gian huấn luyện

Bảng 5: Cấu hình và quá trình huấn luyện của M2

Hạng mục	Giá trị
Kiến trúc	8 - 32 - 16 - 1
Kích hoạt lớp ẩn	ReLU
Số tham số học được	833
Kích thước biểu diễn trung gian	H1: (193,949, 32), H2: (193,949, 16)
Học suất, epoch, lô	0.05 · 60 · 512
Số lần cập nhật tham số	22,740
Loss train đầu và cuối	0.3801 → 0.3698
Loss trên tập test	0.3691
Thời gian huấn luyện	56.6 giây

Chênh lệch accuracy giữa train và test là -0.0001, và loss test (0.3691) so với loss train (0.3698) lệch -0.0007. Khoảng cách hẹp như vậy cho thấy mô hình chưa quá khớp, phần thiếu nằm ở sức biểu diễn chứ không ở việc học thuộc dữ liệu.

Bảng 6: Kết quả của M2 trên tập test theo ba ngưỡng quyết định

Ngưỡng	Accuracy	Precision	Recall	F1	TP	FN	FP	TN
0.3	0.8064	0.4495	0.5088	0.4773	4,286	4,138	5,248	34,816
0.5	0.8368	0.6029	0.1784	0.2753	1,503	6,921	990	39,074
0.7	0.8280	0.7560	0.0151	0.0296	127	8,297	41	40,023

Hạ ngưỡng từ 0.5 xuống 0.3 nâng recall từ 0.1784 lên 0.5088 và đổi lại precision giảm từ 0.6029 xuống 0.4495, tức bắt thêm 2,783 ca dương thật nhưng báo động nhầm thêm 4,258 ca. Nâng lên 0.7 đẩy recall xuống 0.0151. Với bài sàng lọc bệnh, bỏ sót tổn kém hơn báo động nhầm, nên ngưỡng thấp là lựa chọn hợp lý hơn 0.5.

So với M1, ROC-AUC cao hơn 0.0006 và F1 lệch -0.0176, trong khi số tham số tăng từ 289 lên 833 và thời gian huấn luyện tăng từ 56s lên 57s. Thêm sức chứa không mua được chất lượng ở bài này, vì giới hạn nằm ở lượng thông tin trong tám đặc trưng chứ không ở khả năng biểu diễn của mạng.

6. Mô hình M3 Shallow

Bỏ hẳn lớp ẩn thứ hai, chỉ còn một tầng biểu diễn. Đây là phép thử xem chiều sâu có thực sự cần cho tám đặc trưng bằng hay không.

0.8369	0.2900	0.8069	20s
Accuracy · test	F1 · ngưỡng 0.5	ROC-AUC · test	Thời gian huấn luyện

Bảng 7: Cấu hình và quá trình huấn luyện của M3

Hạng mục	Giá trị
Kiến trúc	8 - 16 - 1
Kích hoạt lớp ẩn	ReLU
Số tham số học được	161
Kích thước biểu diễn trung gian	H1: (193,949, 16)
Học suất, epoch, lô	0.05 · 60 · 512
Số lần cập nhật tham số	22,740
Loss train đầu và cuối	0.3775 → 0.3707
Loss trên tập test	0.3695
Thời gian huấn luyện	19.6 giây

Chênh lệch accuracy giữa train và test là -0.0008, và loss test (0.3695) so với loss train (0.3707) lệch -0.0011. Khoảng cách hẹp như vậy cho thấy mô hình chưa quá khớp, phần thiếu nằm ở sức biểu diễn chứ không ở việc học thuộc dữ liệu.

Bảng 8: Kết quả của M3 trên tập test theo ba ngưỡng quyết định

Ngưỡng	Accuracy	Precision	Recall	F1	TP	FN	FP	TN
0.3	0.8047	0.4458	0.5110	0.4762	4,305	4,119	5,351	34,713
0.5	0.8369	0.5948	0.1917	0.2900	1,615	6,809	1,100	38,964
0.7	0.8282	0.7080	0.0190	0.0370	160	8,264	66	39,998

Hạ ngưỡng từ 0.5 xuống 0.3 nâng recall từ 0.1917 lên 0.5110 và đổi lại precision giảm từ 0.5948 xuống 0.4458, tức bắt thêm 2,690 ca dương thật nhưng báo động nhầm thêm 4,251 ca. Nâng lên 0.7 đẩy recall xuống 0.0190. Với bài sàng lọc bệnh, bỏ sót tổn kém hơn báo động nhầm, nên ngưỡng thấp là lựa chọn hợp lý hơn 0.5.

Bỏ hẳn một lớp ẩn làm ROC-AUC thấp hơn 0.0002 so với M1, với chỉ 161 tham số, bằng khoảng 56% của M1, và nhanh hơn 2.8 lần. Nếu phải chọn một mô hình để triển khai thì đây là ứng viên hợp lý nhất: chất lượng ngang ngửa với chi phí thấp nhất.

7. Mô hình M4 No ReLU

Giữ nguyên ba lớp nhưng thay ReLU bằng hàm đồng nhất. Khi đó ba phép biến đổi tuyến tính xếp chồng vẫn chỉ là một phép tuyến tính, nên mạng suy biến thành hồi quy logistic dù vẫn có 289 tham số.

0.8328	0.2706	0.7995	27s
Accuracy · test	F1 · ngưỡng 0.5	ROC-AUC · test	Thời gian huấn luyện

Bảng 9: Cấu hình và quá trình huấn luyện của M4

Hạng mục	Giá trị
Kiến trúc	8 - 16 - 8 - 1
Kích hoạt lớp ẩn	tuyến tính (hàm đồng nhất)
Số tham số học được	289
Kích thước biểu diễn trung gian	H1: (193,949, 16), H2: (193,949, 8)
Học suất, epoch, lô	0.05 · 60 · 512
Số lần cập nhật tham số	22,740
Loss train đầu và cuối	0.3777 → 0.3779
Loss trên tập test	0.3768
Thời gian huấn luyện	27.2 giây

Chênh lệch accuracy giữa train và test là -0.0004, và loss test (0.3768) so với loss train (0.3779) lệch -0.0011. Khoảng cách hẹp như vậy cho thấy mô hình chưa quá khớp, phần thiếu nằm ở sức biểu diễn chứ không ở việc học thuộc dữ liệu.

Bảng 10: Kết quả của M4 trên tập test theo ba ngưỡng quyết định

Ngưỡng	Accuracy	Precision	Recall	F1	TP	FN	FP	TN
0.3	0.8065	0.4465	0.4740	0.4599	3,993	4,431	4,949	35,115
0.5	0.8328	0.5589	0.1785	0.2706	1,504	6,920	1,187	38,877
0.7	0.8288	0.6380	0.0343	0.0651	289	8,135	164	39,900

Hạ ngưỡng từ 0.5 xuống 0.3 nâng recall từ 0.1785 lên 0.4740 và đổi lại precision giảm từ 0.5589 xuống 0.4465, tức bắt thêm 2,489 ca dương thật nhưng báo động nhầm thêm 3,762 ca. Nâng lên 0.7 đẩy recall xuống 0.0343. Với bài sàng lọc bệnh, bỏ sót tổn kém hơn báo động nhầm, nên ngưỡng thấp là lựa chọn hợp lý hơn 0.5.

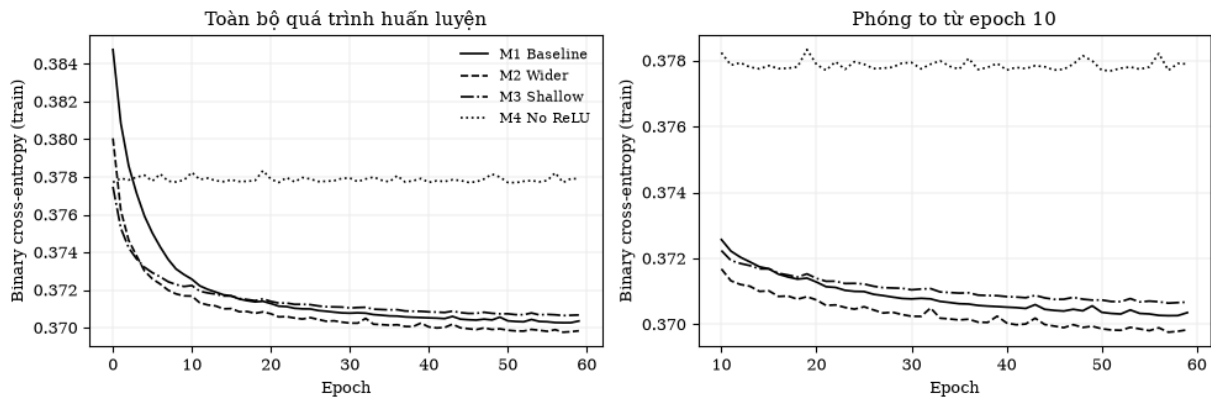
Bỏ ReLU làm ROC-AUC thấp hơn 0.0076 và F1 lệch -0.0223 so với M1. Mức giảm có thật nhưng nhỏ, và đó mới là điều đáng nói: ba phép biến đổi tuyến tính xếp chồng chỉ tương đương một phép tuyến tính, nên M4 thực chất là hồi quy logistic đội lốt mạng ba lớp. Việc nó gần bằng ba mô hình kia là bằng chứng trực tiếp rằng phần lớn tín hiệu trong tám đặc trưng này là tuyến tính.

8. So sánh bốn mô hình

Bảng 11: Bảng tổng hợp bốn mô hình trên tập test, ngưỡng 0.5

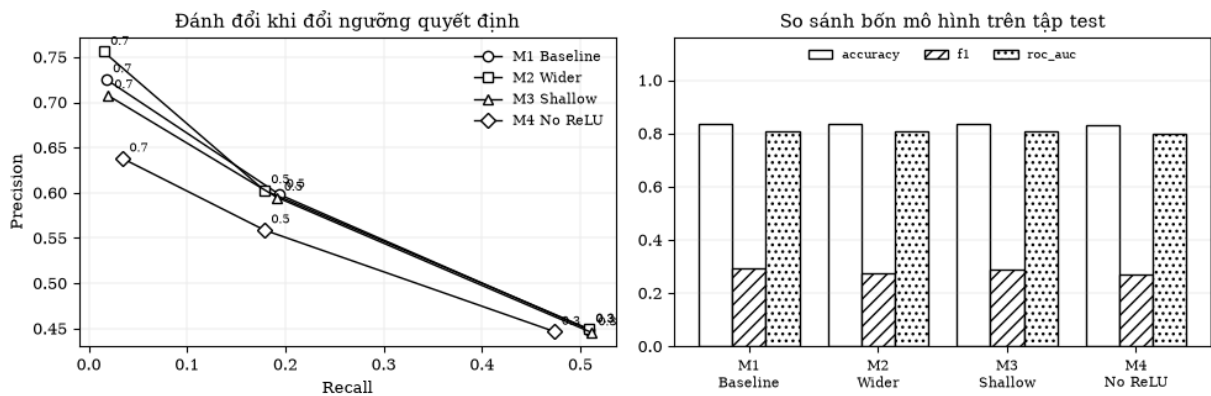
Mô hình	Tham số	Loss train	Loss test	Accuracy	Precision	Recall	F1	ROC-AUC	Giây
M1 Baseline	289	0.3704	0.3695	0.8373	0.5981	0.1940	0.2929	0.8071	55.9
M2 Wider	833	0.3698	0.3691	0.8368	0.6029	0.1784	0.2753	0.8076	56.6
M3 Shallow	161	0.3707	0.3695	0.8369	0.5948	0.1917	0.2900	0.8069	19.6
M4 No ReLU	289	0.3779	0.3768	0.8328	0.5589	0.1785	0.2706	0.7995	27.2

Dòng in đậm là mô hình tốt nhất theo ROC-AUC. Chỉ số này được chọn làm tiêu chí chính vì nó không phụ thuộc ngưỡng quyết định, trong khi accuracy và F1 tại ngưỡng 0.5 đều bị lớp âm chiếm đa số kéo lệch.



Hình 2: Đường mất mát trên tập train của bốn mô hình, toàn bộ quá trình và phần phóng to từ epoch 10.

Hình 2 Observation: cả bốn đường tụt rất nhanh trong mười epoch đầu rồi gần như nằm ngang, và ba đường có ReLU nằm chồng lên nhau. **Interpretation:** phần dễ học của bài toán được học xong sớm, phần còn lại không nằm trong tầm với của tám đặc trưng này. **ML implication:** kéo dài huấn luyện hay tăng sức chứa đều không đáng, muốn tiến thêm phải thêm đặc trưng chứ không phải thêm lớp.



Hình 3: Đánh đổi giữa precision và recall khi đổi ngưỡng quyết định, và so sánh ba chỉ số chính của bốn mô hình.

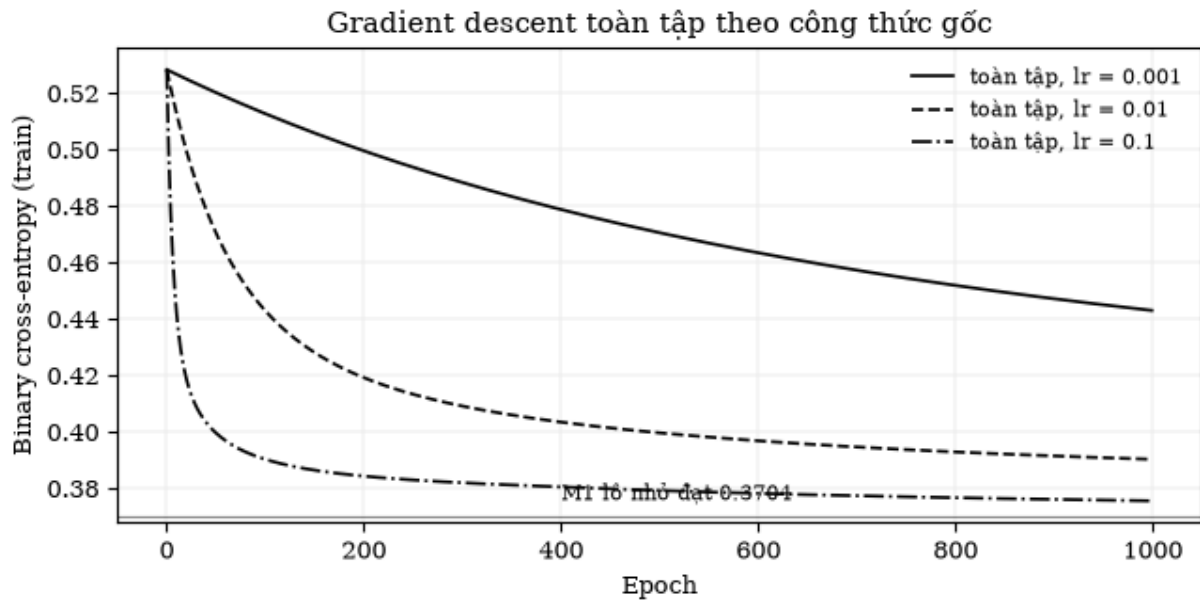
8.1 Học suất và công thức gốc của tài liệu

Để kiểm chứng phần lý giải ở mục 3.4, kiến trúc M1 được chạy lại đúng theo công thức trong tài liệu: gradient descent toàn tập, 1000 epoch, với ba mức học suất.

Bảng 12: Gradient descent toàn tập theo công thức gốc, kiến trúc M1

Học suất	Lần cập nhật	Loss đầu	Loss cuối	Accuracy	Recall	F1	ROC-AUC	Giây
0.001	1,000	0.5284	0.4431	0.8263	0.0000	0.0000	0.6920	232
0.01	1,000	0.5284	0.3904	0.8263	0.0000	0.0000	0.7746	244
0.1	1,000	0.5284	0.3757	0.8312	0.1058	0.1788	0.8004	199
0.05 (lô 512)	22,740	0.3848	0.3704	0.8373	0.1940	0.2929	0.8071	56

Dòng cuối là M1 với lô nhỏ để đối chiếu. Ba cấu hình toàn tập đều chạy 1000 epoch nhưng chỉ có 1000 lần cập nhật tham số, trong khi cấu hình lô nhỏ có 22,740 lần chỉ với 60 epoch.



Hình 4: Đường mất mát của gradient descent toàn tập ở ba mức học suất, đặt cạnh mức mà cấu hình lô nhỏ đạt được.

Hình 4 Observation: học suất 0.001 gần như nằm ngang suốt 1000 epoch, 0.01 đi xuống chậm, 0.1 xuống nhanh nhất nhưng vẫn dừng ở 0.3757, cao hơn mức 0.3704 mà lô nhỏ đạt được. **Interpretation:** số lần cập nhật, chứ không phải số epoch, mới là thứ quyết định tiến độ học. **ML implication:** khi đổi quy mô dữ liệu thì phải đổi cả cách chia lô, giữ nguyên siêu tham số của bài giảng gốc sẽ cho một mô hình trông như đã huấn luyện nhưng thực chất chưa học gì.

8.2 Điều bốn mô hình nói lên

Chiều sâu gần như không đổi được kết quả. M3 chỉ có một lớp ẩn và 161 tham số nhưng ROC-AUC chỉ lệch 0.0002 so với M1.

Bề rộng cũng vậy. M2 nhiều hơn M1 544 tham số và tốn thêm 1 giây mà không đổi được thứ hạng.

Phi tuyến có đóng góp, nhưng nhỏ. M4 mất 0.0076 ROC-AUC khi bỏ ReLU. Đây là số đo của phần tín hiệu phi tuyến trong tám đặc trưng này.

Ngưỡng quyết định đáng giá hơn kiến trúc. Hạ ngưỡng từ 0.5 xuống 0.3 đổi recall của M1 nhiều hơn hẳn mọi khác biệt giữa bốn kiến trúc cộng lại.

9. Ba mô hình machine learning cho mỗi category

Ba ứng dụng của Assignment 02 đã có bảng so sánh riêng, nhưng mọi mô hình trong các bảng đó đều thuộc một trong ba nhóm: tuyến tính, xác suất sinh, hoặc cây và tổ hợp cây. Hai notebook bổ sung lấp hai khoảng trống còn lại. **KNN** mang vào nhóm học dựa trên thực thể, thứ chưa từng xuất hiện trong dự án. **AdaBoost** mang vào một cách hiện thực hoá boosting khác với gradient boosting đang dùng, để tách bạch xem phần thắng của boosting đến từ ý tưởng chung hay từ chi tiết cài đặt.

Điều kiện đo

Dữ liệu trong data/ đã được bổ sung sau khi notebook 01-03 chạy lần cuối. Vì vậy mô hình mốc **được huấn luyện lại** trên bộ dữ liệu hiện tại thay vì chép lại con số của Assignment 02. Mọi số trong chương này đo trên cùng một lần chia 70/15/15 với seed 42, nên chênh lệch giữa các dòng quy được về thuật toán. Ngược lại, chúng **không** so trực tiếp được với bảng trong báo cáo Assignment 02.

Bảng 13: Phạm vi đo: ba category, bốn bài toán

Category	Bài toán	Kiểu	Train	Val	Test	Mô hình mốc
Tiểu đường	nguy cơ tiểu đường	nhị phân	169,705	36,366	36,366	Random Forest
Giá nhà	giá nhà	hồi quy	175,000	37,500	37,500	HistGradientBoosting
E-commerce	mức hài lòng	nhị phân	22,129	4,742	4,742	LogReg bảng + TF-IDF
E-commerce	nhóm sản phẩm	9 lớp	4,493	964	963	LogReg bảng + TF-IDF

Category e-commerce có hai target độc lập trên cùng một bảng dữ liệu, nên được tính là hai bài toán. Cả bốn bài toán đều nhận đủ ba mô hình.

9.1 Ba bài toán phân loại

Bảng 14: Kết quả trên tập test của ba bài toán phân loại

Category	Nhóm	Mô hình	Accuracy	Recall	F1-macro	ROC-AUC
Tiểu đường nguy cơ tiểu đường	KNN	KNN (k=5)	0.8176	0.2632	0.6137	0.7204
	AdaBoost	AdaBoost (50 vòng, cây sâu 3)	0.7192	0.7498	0.6438	0.8071
	Mốc	Random Forest	0.7303	0.7368	0.6515	0.8123
E-commerce mức hài lòng	KNN	KNN cosin (k=5)	0.7777	0.9239	0.6096	0.7041
	AdaBoost	AdaBoost (300 vòng, cây sâu 3)	0.7902	0.7799	0.7452	0.8785
	Mốc	LogReg bảng + TF-IDF	0.8323	0.8378	0.7853	0.8952
E-commerce nhóm sản phẩm	KNN	KNN cosin (k=5)	0.3946	0.2304	0.2371	-
	AdaBoost	AdaBoost (200 vòng, cây sâu 3)	0.4050	0.4571	0.3988	-
	Mốc	LogReg bảng + TF-IDF	0.4029	0.4394	0.3368	-

Dòng in đậm là mô hình đạt F1-macro cao nhất trong từng bài toán. F1-macro được dùng làm tiêu chí thay vì accuracy vì cả ba bài toán đều lệch lớp: đoán toàn lớp đa số đã cho accuracy khá cao mà không học được gì.

Bài toán tiểu đường cho thấy rõ nhất khác biệt giữa hai hướng bổ sung. AdaBoost đạt F1-macro 0.6438, gần như chạm mốc Random Forest 0.6515, vì cả hai đều là tổ hợp cây và cùng được cân bằng lớp. KNN chỉ đạt 0.6137, và toàn bộ khoảng cách nằm ở recall: 0.2632 so với 0.7368. Lý

do không nằm ở chất lượng cài đặt mà ở bản chất thuật toán: KNN không có tham số nào để gán trọng số lớp, nên khi 83% hàng xóm thuộc lớp âm thì lá phiếu nghiêng hẳn về lớp âm.

Bài toán nhóm sản phẩm là ngoại lệ duy nhất: AdaBoost đạt F1-macro 0.3988, vượt mốc 0.3368. Không nên đọc con số này thành bằng chứng về chất lượng thuật toán. Cả hai mô hình đều quanh quẩn mức đoán mò trên 9 lớp, và khi không mô hình nào thật sự học được quy luật thì thứ hạng bị quyết định bởi cách phân bổ dự đoán ra các lớp chứ không phải bởi năng lực học.

9.2 Bài toán hồi quy

Bảng 15: Kết quả trên tập test của bài toán giá nhà

Nhóm	Mô hình	R ² (log)	MAE (USD)	RMSE	
				(USD)	MAPE (%)
KNN	KNN (k=25)	0.6377	135,206	218,931	34.4
AdaBoost	AdaBoost.R2 (25 vòng, cây sâu 8)	0.6355	138,182	223,249	33.7
Mốc	HistGradientBoosting	0.7095	121,023	198,788	30.0

Khoảng cách giữa AdaBoost (0.6355) và HistGradientBoosting (0.7095) trên cùng một biểu diễn 8 chiều là khoảng cách giữa hai cách hiện thực hoá cùng một ý tưởng boosting. AdaBoost.R2 cập nhật trọng số theo sai số tuyệt đối đã chuẩn hoá, tức chỉ biết mẫu nào sai nhiều hơn chứ không biết sai theo hướng nào; gradient boosting học thẳng phần dư có dấu nên mỗi vòng sửa đúng chiều còn thiếu. Bảng quét số vòng trong notebook 06 xác nhận điều đó: chất lượng đạt đỉnh ở vòng 25 rồi đi xuống khi chạy thêm.

KNN đạt 0.6377, sát AdaBoost hơn mọi bài toán khác. Giá nhà là bài toán có tính địa phương cao: hai căn cùng thành phố, cùng diện tích, cùng số phòng thì giá thật sự gần nhau, và đó đúng là giả thiết mà KNN đặt ra.

Hai phát hiện về phương pháp, không phải về kết quả

Chuẩn hoá không phải thủ tục hình thức. Biểu diễn `target_enc` của Assignment 02 được thiết kế cho cây quyết định, vốn cắt theo từng chiều nên miễn nhiệm với thang đo. Đưa KNN vào mới lộ ra rằng biểu diễn đó không dùng ngay được: tám chiều lệch nhau hàng nghìn lần, phải ghép thêm một bước `StandardScaler` thì khoảng cách Euclid mới có nghĩa.

Không được đặt `class_weight="balanced"` lên bộ học yếu của AdaBoost. AdaBoost tính tỉ lệ lỗi theo bộ trọng số của nó, trong khi `class_weight` khiến cây tối ưu theo một bộ trọng số khác. Hai bộ lệch nhau đẩy tỉ lệ lỗi vòng đầu lên 0.3963 và chạm ngưỡng dừng sau đúng 5 vòng, tức ensemble 300 vòng không bao giờ được dựng. Cách đúng là truyền trọng số cân bằng lớp vào giá trị khởi tạo qua tham số `sample_weight` của `fit`.

10. Deep learning cho category giá nhà

Notebook 04 dựng mạng nơ-ron cho bài toán phân loại. Chương này đưa cùng bộ công cụ đó sang bài toán hồi quy, để category giá nhà có mô hình deep learning riêng thay vì đi mượn kết luận của category khác. Toàn bộ mạng vẫn viết bằng NumPy: lan truyền xuôi, hàm mất mát, lan truyền ngược và gradient descent đều viết tay.

Chỉ hai thứ phải đổi khi chuyển từ phân loại sang hồi quy. Lớp cuối bỏ sigmoid, trả về số thực bất kỳ. Hàm mất mát đổi từ binary cross entropy sang sai số bình phương trung bình. Thứ ba thì **không** phải đổi, và đó mới là điểm đáng nói: hàm lan truyền ngược giữ nguyên từng dòng.

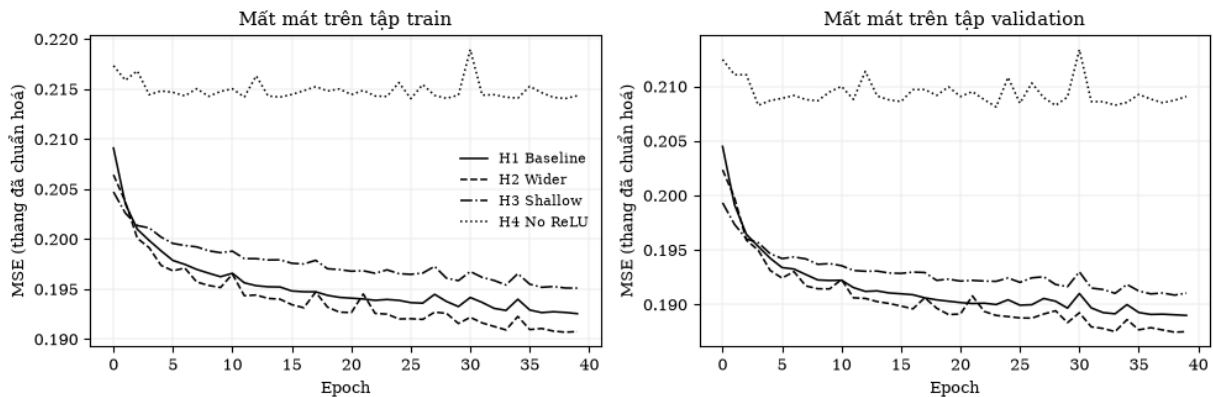
$$\partial L / \partial z^{(L)} = (\hat{y} - y) / n \quad \text{đúng cho cả sigmoid + cross entropy lẫn đầu ra đồng nhất + sai số bình phương}$$

Đây là tính chất của các cặp hàm liên kết chuẩn: đạo hàm của hàm kích hoạt triệt tiêu gọn với mẫu số của hàm mất mát tương ứng. Nó cũng là lý do các thư viện deep learning tách rời được phần mạng khỏi phần hàm mất mát.

Bảng 16: Bốn kiến trúc hồi quy, đo trên tập validation

Mô hình	Kiến trúc	Kích hoạt	Tham số	Loss	R ² (log)	MAE (USD)	Giây
H1 Baseline	8 - 32 - 16 - 1	ReLU	833	0.18901	0.6209	137,980	48
H2 Wider	8 - 64 - 32 - 1	ReLU	2,689	0.18752	0.6238	137,822	77
H3 Shallow	8 - 32 - 1	ReLU	321	0.19105	0.6168	139,337	34
H4 No ReLU	8 - 32 - 16 - 1	tuyến tính	833	0.20909	0.5806	148,536	43

Cả bốn dùng chung seed khởi tạo 42, học suất 0.05, 40 epoch và lô 512 dòng, nên chênh lệch giữa chúng quy được về kiến trúc. Biến mục tiêu là log(giá) đã chuẩn hoá bằng trung bình và độ lệch chuẩn của **tập train**.

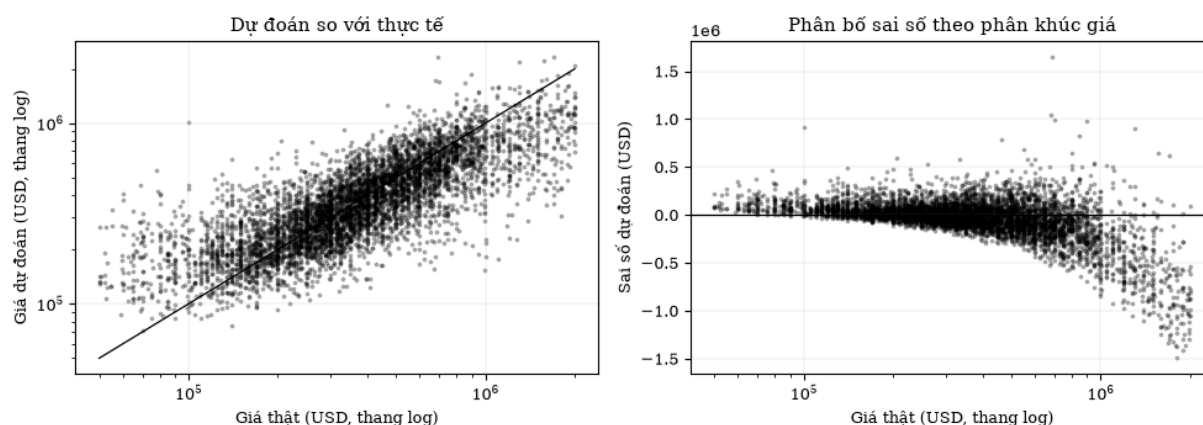


Hình 5: Mất mát trên tập train và tập validation của bốn kiến trúc hồi quy. Ba kiến trúc có ReLU chồng lên nhau, kiến trúc bỏ ReLU đứng hằn ở mức cao hơn.

Kiến trúc H4 bỏ ReLU mất 0.0433 điểm R² dù có đúng số tham số như kiến trúc gốc. Ba phép biến đổi tuyến tính xếp chồng vẫn chỉ là một phép tuyến tính, nên mạng suy biến thành hồi quy tuyến tính. Ngược lại, nhân đôi bề rộng chỉ đổi được rất ít: phi tuyến mới là thứ tạo ra giá trị, không phải số tham số.

Bảng 17: Mạng nơ-ron so với mô hình ML tốt nhất, trên tập test

Mô hình	R ² (log)	MAE (USD)	RMSE (USD)	MAPE (%)
Mạng nơ-ron H2 (8 - 64 - 32 - 1)	0.6265	137,828	222,488	34.5



Hình 6: Dự đoán so với giá thật, và phân bố sai số theo phân khúc giá. Sai số loe rộng dần ở hai đầu và lệch xuống có hệ thống ở phân khúc cao.

Mạng nơ-ron không vượt được HistGradientBoosting trên tám chiều đặc trưng bảng, chênh 0.0830 điểm R^2 và 16,805 USD ở MAE. Kết quả này khớp với kết luận phổ biến trong ngành và không có gì bất ngờ: khi đầu vào đã là một bảng số sạch với 8 cột, cây quyết định vẫn là lựa chọn đúng. Khuôn mẫu sai số ở hình trên cũng không phải lỗi của thuật toán mà là giới hạn của dữ liệu: tám đặc trưng hiện có không phân biệt được một căn ba triệu đô với một căn tám trăm nghìn trong cùng thành phố, cùng diện tích.

11. Deep learning cho category e-commerce

Ba chương trước đều làm việc trên bảng số. Chương này xử lý loại dữ liệu mà deep learning thật sự sinh ra để giải: văn bản.

Assignment 02 đã dựng tensor văn bản $\mathbf{B} \times \mathbf{T} \times \mathbf{d}$ bằng một ma trận nhúng ngẫu nhiên cố định rồi lấy trung bình theo trục thời gian. Ma trận đó không bao giờ được cập nhật, nên nó chỉ là một phép chiếu ngẫu nhiên. Chương này đi nốt phần còn lại: ma trận nhúng trở thành tham số của mạng và được học bằng lan truyền ngược, giống mọi trọng số khác.

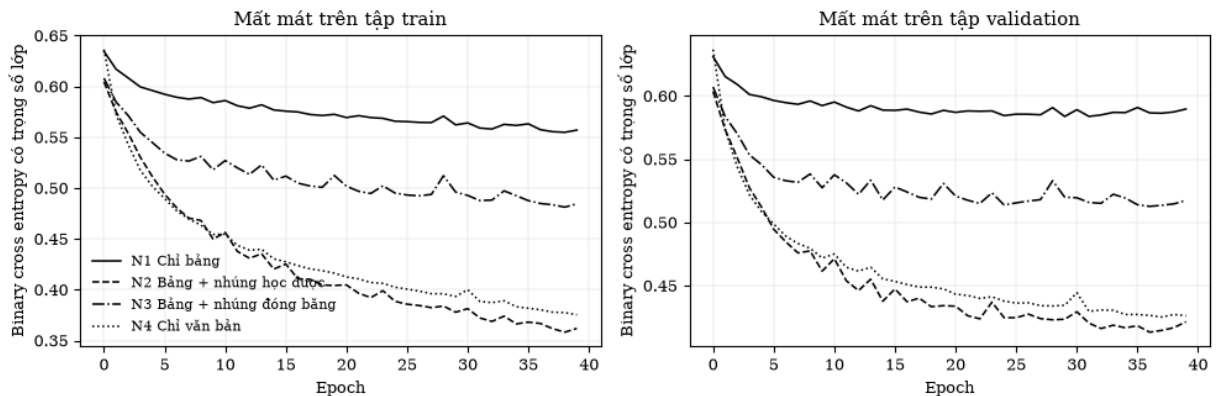
$$p_b = (1/c_b) \sum_t m_{bt} E[ids_{bt}] \rightarrow \partial L / \partial E[v] = \sum_{(b,t): ids=v} (1/c_b) \partial L / \partial p_b$$

Đạo hàm đi ngược qua phép gộp trung bình có che chỉ là phép cộng dồn về đúng những hàng từ vựng đã được tra. Một từ xuất hiện ở nhiều bình luận thì gradient của nó là tổng các đóng góp, nên phải cộng dồn chứ không gán đè. Toàn bộ cơ chế nằm gọn trong sáu dòng NumPy.

Bảng 18: Bốn cấu hình mạng, đo trên tập validation

Mô hình	Dùng bảng	Ma trận nhúng	d vào	Tham số	Accuracy	F1-macro	ROC-AUC
N1 Chỉ bảng	có	không dùng	81	3,169	0.7046	0.6449	0.7590
N2 Bảng + nhúng học được	có	học được	113	164,225	0.8450	0.7929	0.8948
N3 Bảng + nhúng đóng băng	có	đóng băng	113	164,225	0.7607	0.7048	0.8265
N4 Chỉ văn bản	không	học được	32	161,633	0.8368	0.7851	0.8887

Từ điển 5,000 từ xây **chỉ từ tập train**, chuỗi cắt và đệm về 32 token, mỗi từ 32 chiều. Hàm mất mát có trọng số lớp (0.6443 cho lớp hài lòng, 2.2321 cho lớp còn lại) để mạng không đi đường tắt bằng cách đoán tất cả là hài lòng.



Hình 7: Mất mát có trọng số lớp trên tập train và validation của bốn cấu hình. Hai đường thấp nhất đều là cấu hình có ma trận nhúng được huấn luyện.

11.1 Tách bạch hai nguồn đóng góp

N1 và N2 chỉ khác nhau ở chỗ có dùng văn bản hay không. N2 và N3 dùng **đúng cùng một ma trận nhúng**, đúng cùng số tham số, đúng cùng kiến trúc, chỉ khác ở chỗ N3 đóng băng ma trận đó ở giá trị ngẫu nhiên ban đầu. Hai phép so sánh đó tách được hai thứ mà Assignment 02 gộp làm một.

+0.1479	+0.0881	+0.1431	164,225
Có văn bản · N2 – N1	Học nhúng · N2 – N3	TF-IDF · Assignment 02	Tham số của N2

Mức tăng nhờ có văn bản là +0.1479 F1-macro, gần như trùng khớp với +0.1431 mà Assignment 02 đo được bằng TF-IDF trên bộ dữ liệu cũ. Hai cách biểu diễn văn bản hoàn toàn khác nhau, hai bộ dữ liệu khác nhau, nhưng cùng cho một kết luận: bình luận của khách chứa khoảng chừng ấy thông tin về mức hài lòng. Đó là thuộc tính của dữ liệu, không phải của thuật toán.

Con số thứ hai mới là phần Assignment 02 chưa trả lời được. Chênh lệch +0.0881 F1-macro giữa N2 và N3 là phần đóng góp của riêng việc **học** biểu diễn từ, tách bạch khỏi phần đóng góp của việc có văn bản. Hơn một nửa giá trị mà văn bản mang lại chỉ xuất hiện khi biểu diễn của từ được huấn luyện cùng bài toán.

Ma trận nhúng khởi tạo hoàn toàn ngẫu nhiên và không được cung cấp bất kỳ từ điển cảm xúc nào. Sau 40 epoch, **bad** và **worst** là hàng xóm gần nhất của nhau theo cosin, còn **good** kéo về **nice**. Cấu trúc ngữ nghĩa đó không được lập trình vào đâu cả: nó xuất hiện vì hai từ có tác động giống nhau lên mục tiêu thì gradient đẩy chúng về cùng một hướng. Xen giữa các từ hợp lý vẫn có những token hiếm chỉ xuất hiện vài lần, hàng nhúng của chúng gần như vẫn là giá trị ngẫu nhiên ban đầu. Đó là giới hạn cố hữu của việc học nhúng trên hai mươi nghìn mẫu.

11.2 So với mô hình machine learning tốt nhất

Bảng 19: Mạng nơ-ron so với mô hình ML tốt nhất, trên tập test

Mô hình	d vào	Accuracy	Recall	F1	F1-macro	ROC-AUC
Mạng nơ-ron N2 (Bảng + nhúng học được)	113	0.8452	0.8647	0.8966	0.7945	0.8876
LogReg bảng + TF-IDF	11,835	0.8323	0.8378	0.8858	0.7853	0.8952

Đây là lần đầu tiên trong cả dự án một mô hình deep learning viết tay vượt mô hình machine learning tốt nhất. Mốc ML vẫn nhỉnh hơn ở ROC-AUC (0.8952 so với 0.8876), tức nó xếp hạng các ca tốt hơn một chút, nhưng ở ngưỡng quyết định 0.5 thì mạng phân loại chính xác hơn.

Chênh lệch không lớn, và điều đáng chú ý nằm ở chỗ khác: mạng đạt kết quả đó với 113 chiều đầu vào trong khi hồi quy logistic cần 11,835 chiều TF-IDF. Mạng nén toàn bộ từ vựng vào một ma trận $5,001 \times 32$ và học cách nén đó từ chính bài toán, thay vì trải mỗi từ thành một chiều riêng.

Đọc chương 10 và 11 cùng nhau

Trên tám chiều đặc trưng bảng của bài toán giá nhà, mạng nơ-ron thua HistGradientBoosting một khoảng rộng. Trên dữ liệu có văn bản, chính bộ công cụ ấy lại vượt mốc. Không phải deep learning mạnh hơn hay yếu hơn machine learning cổ điển, mà là nó có chỗ dừng: khi đầu vào cần được **học cách biểu diễn**, chứ không phải khi đầu vào đã là một bảng số sạch.

12. Tổng hợp: ba mô hình ML và một mô hình DL cho mỗi category

Bốn notebook bổ sung đưa mỗi category lên đủ ba mô hình machine learning và một mô hình deep learning. Hai bảng dưới đây gom toàn bộ kết quả trên tập test về một chỗ.

Bảng 20: Ba bài toán phân loại, toàn bộ mô hình, trên tập test

Category	Nhóm	Mô hình	F1-macro	ROC-AUC
Tiểu đường nguy cơ tiểu đường	ML mốc	Random Forest	0.6515	0.8123
	ML bổ sung	KNN (k=5)	0.6137	0.7204
	ML bổ sung	AdaBoost (50 vòng, cây sâu 3)	0.6438	0.8071
	Deep learning	M2 Wider (*)	-	0.8076
E-commerce mức hài lòng	ML mốc	LogReg bảng + TF-IDF	0.7853	0.8952
	ML bổ sung	KNN cosin (k=5)	0.6096	0.7041
	ML bổ sung	AdaBoost (300 vòng, cây sâu 3)	0.7452	0.8785
	Deep learning	N2 Bảng + nhúng học được	0.7945	0.8876
E-commerce nhóm sản phẩm	ML mốc	LogReg bảng + TF-IDF	0.3368	-
	ML bổ sung	KNN cosin (k=5)	0.2371	-
	ML bổ sung	AdaBoost (200 vòng, cây sâu 3)	0.3988	-
	Deep learning	chưa có	-	-

(*) Mô hình deep learning của bài toán tiểu đường là mạng của notebook 04. Nó dùng **tám** trong số 21 đặc trưng và chia tập theo tỉ lệ 80/20 thay vì 70/15/15, nên con số của nó **không** so trực tiếp được với ba dòng phía trên; đặt cạnh nhau ở đây chỉ để thấy phạm vi đã phủ. Bài toán nhóm sản phẩm chưa có mô hình deep learning riêng.

Bảng 21: Bài toán hồi quy, toàn bộ mô hình, trên tập test

Nhóm	Mô hình	R ² (log)	MAE (USD)	MAPE (%)
ML mốc	HistGradientBoosting	0.7095	121,023	30.0
ML bổ sung	KNN (k=25)	0.6377	135,206	34.4
ML bổ sung	AdaBoost.R2 (25 vòng, cây sâu 8)	0.6355	138,182	33.7
Deep learning	H2 Wider (8 - 64 - 32 - 1)	0.6265	137,828	34.5

Đọc ngang hai bảng thì thấy một khuôn mẫu nhất quán. Ở ba bài toán có đầu vào là bảng số đã được đặc trưng hoá bằng tay, mô hình mốc của Assignment 02 vẫn đứng đầu và hai mô hình bổ sung xếp dưới. Ở bài toán duy nhất có đầu vào là văn bản thô, mạng nơ-ron học biểu diễn vượt lên. Đây không phải kết luận về việc thuật toán nào mạnh hơn, mà về việc mỗi thuật toán giả định điều gì về dữ liệu, và giả định đó có đúng với dữ liệu đang có hay không.

Điều báo cáo này không kết luận

Một, không có mô hình triển khai nào thay đổi. Bốn notebook bổ sung mở rộng bảng so sánh, không thay artifact nào đang phục vụ API. Hai, mọi số trong chương 9 đến 12 đo trên bộ dữ liệu hiện tại, đã được bổ sung sau khi notebook 01-03 chạy lần cuối, nên không so trực tiếp được với bảng của Assignment 02. Ba, siêu tham số chỉ được dò trên một lưới nhỏ (giá trị k của KNN, số vòng của AdaBoost, bốn kiến trúc mạng). Các con số là ước lượng trung thực cho đúng những cấu hình đã thử, không phải cho cấu hình tốt nhất có thể của từng thuật toán.

13. Kết luận và giới hạn

Một mạng nơ-ron ba lớp viết hoàn toàn bằng NumPy đã học được trên hơn hai trăm nghìn dòng dữ liệu thật, chạy trên CPU trong vài chục giây mỗi mô hình. Điều framework tự động hoá không phải bản chất toán học mà là công sức triển khai: đạo hàm tự động, quản lý tensor, tối ưu hoá và tính toán trên GPU.

Mô hình tốt nhất theo ROC-AUC là M2 Wider với 0.8076. Nhưng kết luận có ích hơn cho việc triển khai lại là M3: ít tham số nhất, nhanh nhất, chất lượng gần như không kém.

Giới hạn tự nhận

Một, accuracy ở đây dễ gây hiểu nhầm. Đoán toàn lớp âm đã cho khoảng 0.8263, nên mọi con số accuracy trong báo cáo phải đọc kèm recall và ROC-AUC. **Hai, chọn đặc trưng bằng tương quan tuyến tính đơn biến** có thể đã loại mất biến chỉ có ích khi kết hợp với biến khác, và điều đó tự nó đã thu hẹp phần phi tuyến còn lại cho mạng khai thác. **Ba, không có tập validation riêng.** Siêu tham số được chọn bằng suy luận về số lần cập nhật chứ không bằng dò trên tập validation, nên các con số trên tập test là ước lượng trung thực cho đúng cấu hình này chứ không phải cho cấu hình tốt nhất có thể.

Hướng đi tiếp có ích nhất không phải mạng sâu hơn mà là thêm thông tin đầu vào: dùng đủ 21 biến thay vì tám, thêm đặc trưng tương tác, hoặc cân bằng lại lớp bằng trọng số trong hàm mất mát. Cả bốn mô hình ở đây đều đã chạm trần thông tin của tám cột hiện có.

Phụ lục A. Notebook 04: Deep Learning từ đầu

Toàn bộ nội dung notebook, giữ nguyên thứ tự cell. Phần diễn giải là markdown gốc trong notebook, khối In [n] là mã nguồn, khối Out là kết quả đã chạy thật. Mọi con số trong báo cáo đều sinh từ reports/deeplearning_results.json mà notebook này ghi ra, nên báo cáo không thể lệch với notebook.

04_deep_learning.ipynb

21 cell · 14 code

Ứng dụng 4: Deep Learning từ đầu với NumPy

Notebook này dựng lại toàn bộ một mạng nơ-ron nhiều lớp **chỉ bằng NumPy**: lan truyền xuôi, hàm mất mát, lan truyền ngược và gradient descent đều viết tay. Không dùng TensorFlow, PyTorch, Keras hay scikit-learn cho phần học máy.

Bốn mô hình được huấn luyện trên cùng một tập dữ liệu và cùng một quy trình, chỉ khác nhau ở kiến trúc:

| Mã | Kiến trúc | Kích hoạt lớp ẩn | Ý nghĩa | ---|---|---| | M1 | 8 - 16 - 8 - 1 | ReLU | Kiến trúc gốc của bài giảng | | M2 | 8 - 32 - 16 - 1 | ReLU | Rộng gấp đôi | | M3 | 8 - 16 - 1 | ReLU | Bỏ bớt một lớp ẩn | | M4 | 8 - 16 - 8 - 1 | tuyến tính | Bỏ ReLU, kiểm chứng vai trò của phi tuyến |

Notebook tự chứa toàn bộ logic: nạp dữ liệu, định nghĩa mạng, huấn luyện, đo đạc và ghi ra reports/deeplearning_results.json.

In [1]

```
1 import json
2 import time
3 from pathlib import Path
4
5 import matplotlib
6 import matplotlib.pyplot as plt
7 import numpy as np
8 import pandas as pd
9
10 ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
11 DATA = ROOT / "data"
12 REPORTS = ROOT / "reports"
13
14 matplotlib.rcParams.update({
15     "font.family": "serif",
16     "font.size": 9,
17     "axes.edgecolor": "black",
18     "axes.labelcolor": "black",
19     "text.color": "black",
20     "xtick.color": "black",
21     "ytick.color": "black",
22     "figure.facecolor": "white",
23     "savefig.facecolor": "white",
24 })
25
26 print("thư mục gốc :", ROOT)
27 print("numpy      :", np.__version__)
28 print("pandas     :", pd.__version__)
```

Out

```
thư mục gốc : C:\Users\Hung Nguyen Ba\assignment03
numpy      : 2.4.6
pandas     : 3.0.3
```

1. Nạp dữ liệu và chọn 8 đặc trưng

Bài giảng dùng bộ Pima với đúng 8 biến đầu vào. Bộ BRFSS 2015 ở đây có 21 biến nên cần chọn ra 8. Cách chọn là xếp hạng theo trị tuyệt đối hệ số tương quan với nhãn nhị phân rồi lấy 8 biến đầu, có in ra bảng xếp hạng đầy đủ để kiểm chứng.

Nhãn nhị phân giữ đúng quy ước của ba ứng dụng trước: $y = 1$ khi Diabetes_012 > 0, tức gộp tiền tiểu đường và tiểu đường vào lớp dương.

In [2]

```
1 DIABETES_CSV = DATA / "diabetes" / "diabetes.csv"
2 TARGET = "Diabetes_012"
3 SPLIT_SEED = 42
4 TEST_SHARE = 0.20
5
6 raw = pd.read_csv(DIABETES_CSV)
7 frame = raw.drop_duplicates().reset_index(drop=True)
8 y_all = (frame[TARGET] > 0).astype(int)
9
10 ranking = (
11     frame.drop(columns=[TARGET])
12     .apply(lambda s: np.corrcoef(s, y_all)[0, 1])
13     .abs()
14     .sort_values(ascending=False)
15 )
16
17 FEATURES = tuple(ranking.index[:8])
18
19 print(f"Dòng thô          : {len(raw):,}")
20 print(f"Sau khi bỏ trùng   : {len(frame):,} (bỏ {len(raw) - len(frame):,})")
21 print(f"Tỉ lệ lớp dương      : {y_all.mean():.4f}")
22 print("\nXếp hạng |tương quan| với nhãn nhị phân:")
23 print(ranking.round(4).to_string())
24 print("\n8 đặc trưng được chọn:", list(FEATURES))
```

Out

```
Dòng thô          : 279,048
Sau khi bỏ trùng   : 242,437   (bỏ 36,611)
Tỉ lệ lớp dương    : 0.1729

Xếp hạng |tương quan| với nhãn nhị phân:
GenHlth           0.2832
HighBP            0.2606
BMI               0.2112
DiffWalk          0.2092
HighChol          0.2045
Age               0.1858
HeartDiseaseorAttack 0.1671
PhysHlth          0.1592
Income            0.1483
Education         0.1086
PhysActivity      0.1032
Stroke            0.0987
CholCheck         0.0762
HvyAlcoholConsump 0.0664
MentHlth          0.0594
... (rut gon 3 dong) ...
NoDocbcCost       0.0264
Fruits            0.0244
AnyHealthcare     0.0238

8 đặc trưng được chọn: ['GenHlth', 'HighBP', 'BMI', 'DiffWalk', 'HighChol', 'Age', 'HeartDiseaseorAttack', 'PhysHlth']
```

In [3]

```
1 X_all = frame[list(FEATURES)].to_numpy(dtype=np.float64)
2 y_all_arr = y_all.to_numpy(dtype=np.float64).reshape(-1, 1)
3
4 rng_split = np.random.default_rng(SPLIT_SEED)
5 order = rng_split.permutation(len(X_all))
6 cut = int((1.0 - TEST_SHARE) * len(X_all))
7 train_idx, test_idx = order[:cut], order[cut:]
8
9 X_train_raw, X_test_raw = X_all[train_idx], X_all[test_idx]
10 y_train, y_test = y_all_arr[train_idx], y_all_arr[test_idx]
11
12 mean = X_train_raw.mean(axis=0)
```

```

13     std = X_train_raw.std(axis=0) + 1e-8
14     X_train = (X_train_raw - mean) / std
15     X_test = (X_test_raw - mean) / std
16
17     print(f"train {len(X_train):,} dòng    test {len(X_test):,} dòng")
18     print(f"tỉ lệ dương train {y_train.mean():.4f}    test {y_test.mean():.4f}")
19     print("\nChuẩn hoá bằng mean/std của tập train, áp lại cho test:")
20     for name, m, s in zip(FEATURES, mean, std):
21         print(f"    {name:22s} mean {m:8.4f}    std {s:8.4f}")
22     print("\nSau chuẩn hoá, X_train mean ~ 0 và std ~ 1:")
23     print("    mean:", np.round(X_train.mean(axis=0), 6))
24     print("    std :", np.round(X_train.std(axis=0), 6))

```

Out

```

train 193,949 dòng    test 48,488 dòng
tỉ lệ dương train 0.1727    test 0.1737

Chuẩn hoá bằng mean/std của tập train, áp lại cho test:
GenHlth          mean    2.6004    std    1.0641
HighBP           mean    0.4545    std    0.4979
BMI              mean    28.6970    std    6.8074
DiffWalk         mean    0.1852    std    0.3884
HighChol         mean    0.4422    std    0.4966
Age              mean    8.0805    std    3.0953
HeartDiseaseorAttack mean    0.1035    std    0.3046
PhysHlth         mean    4.6694    std    9.0155

Sau chuẩn hoá, X_train mean ~ 0 và std ~ 1:
mean: [-0. -0.  0. -0.  0.  0.  0. -0.]
std : [1.  1.  1.  1.  1.  1.  1.  1.]

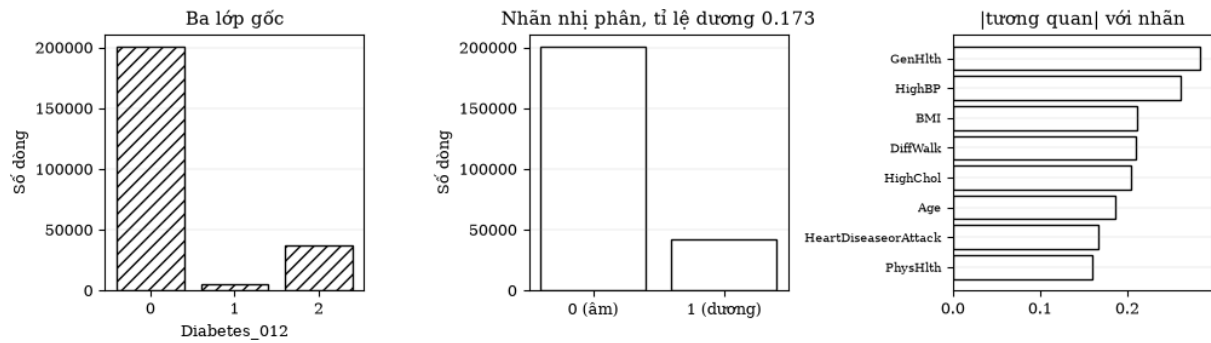
```

In [4]

```

1     fig, axes = plt.subplots(1, 3, figsize=(10.2, 3.0))
2
3     counts = frame[TARGET].value_counts().sort_index()
4     axes[0].bar([str(int(k)) for k in counts.index], counts.values,
5                 color="white", edgecolor="black", hatch="//")
6     axes[0].set_title("Ba lớp gốc")
7     axes[0].set_xlabel("Diabetes_012")
8     axes[0].set_ylabel("Số dòng")
9
10    binary_counts = y_all.value_counts().sort_index()
11    axes[1].bar(["0 (âm)", "1 (dương)"], binary_counts.values, color="white", edgecolor="black")
12    axes[1].set_title(f"Nhân nhị phân, tỉ lệ dương {y_all.mean():.3f}")
13    axes[1].set_ylabel("Số dòng")
14
15    pos = ranking.iloc[:8][::-1]
16    axes[2].barh(range(len(pos)), pos.values, color="white", edgecolor="black")
17    axes[2].set_yticks(range(len(pos)))
18    axes[2].set_yticklabels(pos.index, fontsize=7)
19    axes[2].set_title("/tương quan/ với nhân")
20
21    for axis in axes:
22        axis.grid(True, axis="y", linewidth=0.3, alpha=0.4)
23    fig.tight_layout()
24    plt.show()

```



2. Mạng nơ-ron viết tay

Toàn bộ phép toán của mạng nằm trong ô lệnh dưới đây. Mạng được viết theo kiểu hàm thuần: `update` trả về bộ tham số mới thay vì sửa tại chỗ, nên mỗi bước huấn luyện là một phép biến đổi rõ ràng từ `params` cũ sang `params` mới.

Lan truyền xuôi là $Z_k = H_{k-1} W_k + b_k$ rồi $H_k = g(Z_k)$, với g là ReLU ở lớp ẩn và sigmoid ở lớp ra. Hàm mất mát là binary cross-entropy. Ghép sigmoid với binary cross-entropy cho kết quả rất gọn ở lớp cuối, $dZ_L = (y_{\text{hat}} - y) / N$, từ đó gradient lan ngược bằng quy tắc chuỗi.

In [5]

```

1  def relu(x):
2      return np.maximum(0.0, x)
3
4
5  def relu_derivative(x):
6      return (x > 0).astype(float)
7
8
9  def identity(x):
10     return x
11
12
13  def identity_derivative(x):
14     return np.ones_like(x)
15
16
17  def sigmoid(x):
18     return 1.0 / (1.0 + np.exp(-np.clip(x, -50.0, 50.0)))
19
20
21  ACTIVATIONS = {"relu": (relu, relu_derivative), "linear": (identity, identity_derivative)}
22
23
24  def init_params(sizes, seed):
25     rng = np.random.default_rng(seed)
26     return tuple(
27         (rng.standard_normal((fan_in, fan_out)) * np.sqrt(2.0 / fan_in),
28          np.zeros((1, fan_out)))
29         for fan_in, fan_out in zip(sizes, sizes[1:]))
30
31
32
33  def forward(params, X, hidden_activation):
34     act, _ = ACTIVATIONS[hidden_activation]
35     pre, activations = [], [X]
36     h = X
37     last = len(params) - 1
38     for index, (W, b) in enumerate(params):
39         z = h @ W + b
40         h = sigmoid(z) if index == last else act(z)
41         pre.append(z)

```

```

42         activations.append(h)
43     return h, {"z": pre, "h": activations}
44
45
46 def binary_cross_entropy(y, y_hat):
47     p = np.clip(y_hat, 1e-8, 1.0 - 1e-8)
48     return float(-np.mean(y * np.log(p) + (1.0 - y) * np.log(1.0 - p)))
49
50
51 def backward(params, y, cache, hidden_activation):
52     _, act_derivative = ACTIVATIONS[hidden_activation]
53     z, h = cache["z"], cache["h"]
54     grads = []
55     delta = (h[-1] - y) / len(y)
56     for index in range(len(params) - 1, -1, -1):
57         grads.append((h[index].T @ delta, np.sum(delta, axis=0, keepdims=True)))
58         if index:
59             delta = (delta @ params[index][0].T) * act_derivative(z[index - 1])
60     return tuple(reversed(grads))
61
62
63 def update(params, grads, learning_rate):
64     return tuple(
65         (W - learning_rate * dW, b - learning_rate * db)
66         for (W, b), (dW, db) in zip(params, grads)
67     )
68
69
70 demo = init_params((8, 16, 8, 1), 42)
71 demo_out, demo_cache = forward(demo, X_train[:5], "relu")
72 print("Kiểm tra kích thước ma trận trên 5 dòng đầu:")
73 for index, (W, b) in enumerate(demo, start=1):
74     print(f"  W{index} {W.shape}  b{index} {b.shape}")
75 for index, h in enumerate(demo_cache["h"][1:-1], start=1):
76     print(f"  H{index}.shape = {h.shape}")
77 print("  y_hat.shape =", demo_out.shape)

```

Out

```

Kiểm tra kích thước ma trận trên 5 dòng đầu:
W1 (8, 16)  b1 (1, 16)
W2 (16, 8)  b2 (1, 8)
W3 (8, 1)   b3 (1, 1)
H1.shape = (5, 16)
H2.shape = (5, 8)
y_hat.shape = (5, 1)

```

3. Chỉ số đánh giá

Ma trận nhầm lẫn, accuracy, precision, recall, F1 và ROC-AUC đều tính trực tiếp từ đếm số. ROC-AUC dùng công thức thống kê Mann-Whitney trên hạng, có xử lý hạng trung bình cho các giá trị xác suất bằng nhau.

In [6]

```

1     EPS = 1e-8
2
3
4     def confusion(y_true, y_pred):
5         positive_true = y_true == 1
6         positive_pred = y_pred == 1
7         return {
8             "tp": int(np.sum(positive_pred & positive_true)),
9             "tn": int(np.sum(~positive_pred & ~positive_true)),
10            "fp": int(np.sum(positive_pred & ~positive_true)),
11            "fn": int(np.sum(~positive_pred & positive_true)),
12        }
13
14
15     def score(y_true, y_prob, threshold=0.5):

```

```

16     counts = confusion(y_true, (y_prob >= threshold).astype(int))
17     tp, tn, fp, fn = counts["tp"], counts["tn"], counts["fp"], counts["fn"]
18     precision = tp / (tp + fp + EPS)
19     recall = tp / (tp + fn + EPS)
20     return {
21         "threshold": threshold,
22         "accuracy": (tp + tn) / (tp + tn + fp + fn + EPS),
23         "precision": precision,
24         "recall": recall,
25         "f1": 2 * precision * recall / (precision + recall + EPS),
26         "balanced_accuracy": 0.5 * (recall + tn / (tn + fp + EPS)),
27         **counts,
28     }
29
30
31 def roc_auc(y_true, y_prob):
32     scores = y_prob.ravel()
33     order = np.argsort(scores, kind="mergesort")
34     ranks = np.empty(len(order), dtype=float)
35     ranks[order] = np.arange(1, len(order) + 1)
36     ordered = scores[order]
37     start = 0
38     for end in range(1, len(ordered) + 1):
39         if end == len(ordered) or ordered[end] != ordered[start]:
40             ranks[order[start:end]] = ranks[order[start:end]].mean()
41             start = end
42     labels = y_true.ravel()
43     n_pos = float(np.sum(labels == 1))
44     n_neg = float(len(labels) - n_pos)
45     return float((ranks[labels == 1].sum() - n_pos * (n_pos + 1) / 2) / (n_pos * n_neg))
46
47
48 print("Kiểm tra nhanh trên nhãn dựng sẵn:")
49 probe_y = np.array([[1], [0], [1], [0]], dtype=float)
50 probe_p = np.array([[0.9], [0.2], [0.4], [0.1]], dtype=float)
51 print(" ", {k: round(v, 4) if isinstance(v, float) else v
52           for k, v in score(probe_y, probe_p).items()})
53 print(" ROC-AUC =", round(roc_auc(probe_y, probe_p), 4), "(kỳ vọng 0.75)")

```

Out

```

Kiểm tra nhanh trên nhãn dựng sẵn:
{'threshold': 0.5, 'accuracy': 0.75, 'precision': 1.0, 'recall': 0.5, 'f1': 0.6667, 'balanced_accuracy': 0.75, 'tp': 1,
'tn': 2, 'fp': 0, 'fn': 1}
ROC-AUC = 1.0 (kỳ vọng 0.75)

```

4. Huấn luyện bốn mô hình

Bài giảng dùng gradient descent toàn tập với $lr = 0.01$ và 1000 epoch trên bộ Pima 768 dòng. Ở đây tập train có hơn 200 nghìn dòng, nên 1000 epoch toàn tập chỉ tương đương 1000 lần cập nhật tham số, mạng chưa kịp học và kết quả là đoán toàn lớp âm.

Vì vậy vòng lặp dùng **gradient descent theo lô nhỏ**: mỗi epoch xáo trộn tập train rồi đi qua từng lô 512 dòng. Công thức cập nhật không đổi, chỉ khác ở chỗ gradient được tính trên lô thay vì trên toàn tập. Mục 5 giữ nguyên công thức gốc để đối chiếu.

Cả bốn mô hình dùng chung `INIT_SEED = 42` và `SHUFFLE_SEED = 7`, nên khác biệt giữa chúng chỉ đến từ kiến trúc chứ không từ may rủi khởi tạo.

In [7]

```

1     LEARNING_RATE = 0.05
2     EPOCHS = 60
3     BATCH_SIZE = 512
4     INIT_SEED = 42
5     SHUFFLE_SEED = 7
6     THRESHOLDS = (0.3, 0.5, 0.7)
7
8     ARCHITECTURES = (
9         {"key": "M1", "label": "Baseline", "sizes": (8, 16, 8, 1), "activation": "relu"},

```

```

10     {"key": "M2", "label": "Wider", "sizes": (8, 32, 16, 1), "activation": "relu"},
11     {"key": "M3", "label": "Shallow", "sizes": (8, 16, 1), "activation": "relu"},
12     {"key": "M4", "label": "No ReLU", "sizes": (8, 16, 8, 1), "activation": "linear"},
13 )
14
15
16 def n_parameters(sizes):
17     return sum(a * b + b for a, b in zip(sizes, sizes[1:]))
18
19
20 def fit(arch, X_tr, y_tr, X_te, y_te, epochs, learning_rate, batch_size):
21     params = init_params(arch["sizes"], INIT_SEED)
22     rng = np.random.default_rng(SHUFFLE_SEED)
23     n = len(X_tr)
24     step = batch_size or n
25     history = []
26     started = time.perf_counter()
27
28     for _ in range(epochs):
29         order = rng.permutation(n) if batch_size else np.arange(n)
30         epoch_loss = None
31         for begin in range(0, n, step):
32             rows = order[begin:begin + step]
33             y_hat, cache = forward(params, X_tr[rows], arch["activation"])
34             if batch_size is None:
35                 epoch_loss = binary_cross_entropy(y_tr, y_hat)
36             grads = backward(params, y_tr[rows], cache, arch["activation"])
37             params = update(params, grads, learning_rate)
38         if batch_size is None:
39             history.append(epoch_loss)
40         else:
41             probe, _ = forward(params, X_tr, arch["activation"])
42             history.append(binary_cross_entropy(y_tr, probe))
43
44     elapsed = time.perf_counter() - started
45     train_prob, cache_tr = forward(params, X_tr, arch["activation"])
46     test_prob, _ = forward(params, X_te, arch["activation"])
47
48     def rounded(row):
49         return {k: (round(v, 6) if isinstance(v, float) else v) for k, v in row.items()}
50
51     return params, {
52         "key": arch["key"],
53         "label": arch["label"],
54         "shape": " - ".join(str(s) for s in arch["sizes"]),
55         "hidden_activation": arch["activation"],
56         "n_params": n_parameters(arch["sizes"]),
57         "epochs": epochs,
58         "batch_size": step,
59         "updates": epochs * int(np.ceil(n / step)),
60         "learning_rate": learning_rate,
61         "seconds": round(elapsed, 2),
62         "loss_first": round(history[0], 6),
63         "loss_final": round(history[-1], 6),
64         "loss_test_end": round(binary_cross_entropy(y_te, test_prob), 6),
65         "loss_history": [round(v, 6) for v in history],
66         "roc_auc_test": round(roc_auc(y_te, test_prob), 6),
67         "train": rounded(score(y_tr, train_prob, 0.5)),
68         "test_by_threshold": {str(t): rounded(score(y_te, test_prob, t)) for t in THRESHOLDS},
69         "hidden_shapes": [list(h.shape) for h in cache_tr["h"][1:-1]],
70     }

```

In [8]

```

1     trained = {}
2     models = []
3     for arch in ARCHITECTURES:
4         params, row = fit(arch, X_train, y_train, X_test, y_test,

```

```

5         EPOCHS, LEARNING_RATE, BATCH_SIZE)
6     trained[arch["key"]] = params
7     models.append(row)
8     t = row["test_by_threshold"] ["0.5"]
9     print(f"{row['key']} {row['label']:9s} {row['shape']:18s} "
10           f"loss {row['loss_final']:.4f} acc {t['accuracy']:.4f} "
11           f"prec {t['precision']:.4f} rec {t['recall']:.4f} f1 {t['f1']:.4f} "
12           f"auc {row['roc_auc_test']:.4f} ({row['seconds']:.1f}s)")

```

Out

```

M1 Baseline  8 - 16 - 8 - 1      loss 0.3704 acc 0.8373 prec 0.5981 rec 0.1940 f1 0.2929 auc 0.8071 (55.9s)

```

Out

```

M2 Wider     8 - 32 - 16 - 1     loss 0.3698 acc 0.8368 prec 0.6029 rec 0.1784 f1 0.2753 auc 0.8076 (56.6s)

```

Out

```

M3 Shallow   8 - 16 - 1         loss 0.3707 acc 0.8369 prec 0.5948 rec 0.1917 f1 0.2900 auc 0.8069 (19.6s)

```

Out

```

M4 No ReLU   8 - 16 - 8 - 1     loss 0.3779 acc 0.8328 prec 0.5589 rec 0.1785 f1 0.2706 auc 0.7995 (27.2s)

```

In [9]

```

1     table = pd.DataFrame([
2         {
3             "Mô hình": f"{m['key']} {m['label']}",
4             "Kiến trúc": m["shape"],
5             "Kích hoạt": m["hidden_activation"],
6             "Tham số": m["n_params"],
7             "Loss train": m["loss_final"],
8             "Loss test": m["loss_test_end"],
9             "Accuracy": m["test_by_threshold"] ["0.5"] ["accuracy"],
10            "Precision": m["test_by_threshold"] ["0.5"] ["precision"],
11            "Recall": m["test_by_threshold"] ["0.5"] ["recall"],
12            "F1": m["test_by_threshold"] ["0.5"] ["f1"],
13            "ROC-AUC": m["roc_auc_test"],
14            "Giây": m["seconds"],
15        }
16        for m in models
17    ])
18    print("Kết quả trên tập test, ngưỡng 0.5")
19    print(table.to_string(index=False))
20
21    print("\nMa trận nhầm lẫn tại ngưỡng 0.5")
22    for m in models:
23        t = m["test_by_threshold"] ["0.5"]
24        print(f" {m['key']} {m['label']:9s} TP {t['tp']:6,} FN {t['fn']:6,} "
25              f"FP {t['fp']:6,} TN {t['tn']:6,}")
26
27    print("\nẢnh hưởng của ngưỡng quyết định")
28    for m in models:
29        parts = []
30        for th in THRESHOLDS:
31            t = m["test_by_threshold"] [str(th)]
32            parts.append(f"{th}: prec {t['precision']:.4f} rec {t['recall']:.4f} f1 {t['f1']:.4f}")
33    print(f" {m['key']} {m['label']:9s} " + " | ".join(parts))

```

Out

```

Kết quả trên tập test, ngưỡng 0.5
Mô hình      Kiến trúc Kích hoạt Tham số Loss train Loss test Accuracy Precision Recall      F1 ROC-AUC
Giây
M1 Baseline  8 - 16 - 8 - 1      relu      289    0.370354  0.369549  0.837321  0.598097 0.193970 0.292937 0.807080
55.89
M2 Wider     8 - 32 - 16 - 1     relu      833    0.369833  0.369111  0.836846  0.602888 0.178419 0.275350 0.807644
56.61
M3 Shallow   8 - 16 - 1         relu      161    0.370673  0.369530  0.836887  0.594843 0.191714 0.289972 0.806857

```

M4 No ReLU 8 - 16 - 8 - 1 linear 289 0.377897 0.376768 0.832804 0.558900 0.178538 0.270625 0.799481 27.21

Ma trận nhầm lẫn tại ngưỡng 0.5

Model	TP	FN	FP	TN
M1 Baseline	1,634	6,790	1,098	38,966
M2 Wider	1,503	6,921	990	39,074
M3 Shallow	1,615	6,809	1,100	38,964
M4 No ReLU	1,504	6,920	1,187	38,877

Ảnh hưởng của ngưỡng quyết định

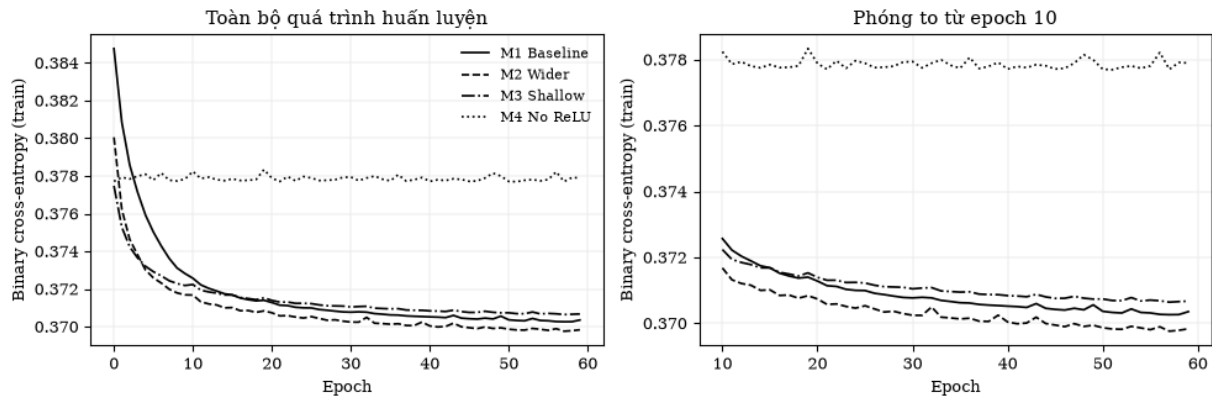
Model	0.3: prec	0.3: rec	0.3: f1	0.5: prec	0.5: rec	0.5: f1	0.7: prec	0.7: rec
M1 Baseline	0.4489	0.5084	0.4768	0.5981	0.1940	0.2929	0.7255	0.0176
M2 Wider	0.4495	0.5088	0.4773	0.6029	0.1784	0.2753	0.7560	0.0151
M3 Shallow	0.4458	0.5110	0.4762	0.5948	0.1917	0.2900	0.7080	0.0190
M4 No ReLU	0.4465	0.4740	0.4599	0.5589	0.1785	0.2706	0.6380	0.0343

In [10]

```

1 fig, axes = plt.subplots(1, 2, figsize=(10.0, 3.4))
2 styles = ["-", "--", "-.", ":"]
3
4 for axis, start in zip(axes, (0, 10)):
5     for m, style in zip(models, styles):
6         history = m["loss_history"][start:]
7         axis.plot(range(start, start + len(history)), history, style, color="black",
8                 linewidth=1.2, label=f"{m['key']} {m['label']}")
9     axis.set_xlabel("Epoch")
10    axis.set_ylabel("Binary cross-entropy (train)")
11    axis.grid(True, linewidth=0.3, alpha=0.4)
12 axes[0].set_title("Toàn bộ quá trình huấn luyện")
13 axes[1].set_title("Phóng to từ epoch 10")
14 axes[0].legend(frameon=False, fontsize=8)
15 fig.tight_layout()
16 plt.show()

```



In [11]

```

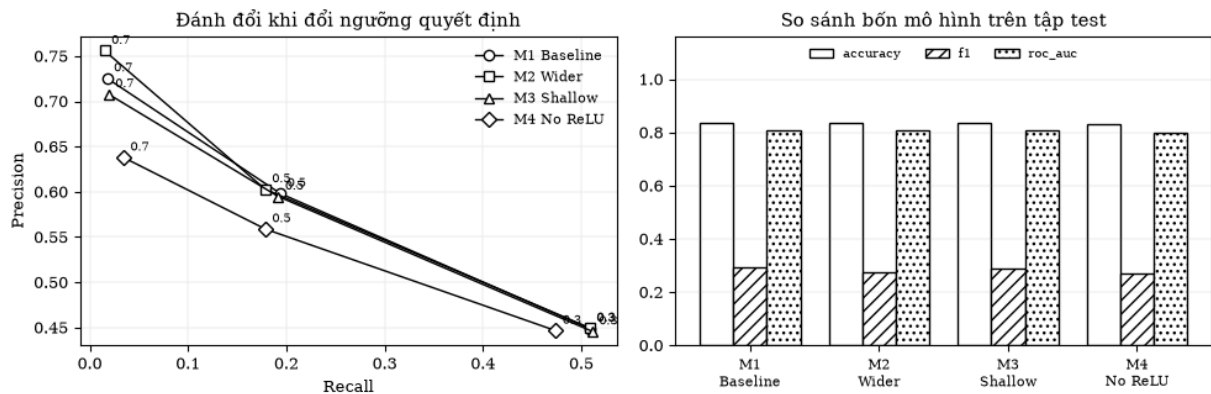
1 fig, axes = plt.subplots(1, 2, figsize=(10.0, 3.4))
2 markers = ["o", "s", "^", "D"]
3
4 for m, marker in zip(models, markers):
5     rows = [m["test_by_threshold"][str(t)] for t in THRESHOLDS]
6     axes[0].plot([r["recall"] for r in rows], [r["precision"] for r in rows], marker + "-",
7                 color="black", markerfacecolor="white", linewidth=1.0,
8                 label=f"{m['key']} {m['label']}")
9     for threshold, r in zip(THRESHOLDS, rows):
10        axes[0].annotate(f"{threshold}", (r["recall"], r["precision"]), fontsize=7,
11                        xytext=(4, 4), textcoords="offset points")
12 axes[0].set_xlabel("Recall")
13 axes[0].set_ylabel("Precision")
14 axes[0].set_title("Đánh đổi khi đổi ngưỡng quyết định")

```

```

15 axes[0].legend(frameon=False, fontsize=8)
16 axes[0].grid(True, linewidth=0.3, alpha=0.4)
17
18 positions = np.arange(len(models))
19 width = 0.26
20 for offset, (metric, hatch) in enumerate(zip(("accuracy", "f1"), ("", "///"))):
21     values = [m["test_by_threshold"]["0.5"][metric] for m in models]
22     axes[1].bar(positions + offset * width, values, width, color="white",
23                edgecolor="black", hatch=hatch, label=metric)
24 axes[1].bar(positions + 2 * width, [m["roc_auc_test"] for m in models], width,
25            color="white", edgecolor="black", hatch="...", label="roc_auc")
26 axes[1].set_xticks(positions + width)
27 axes[1].set_xticklabels([f"{m['key']}\n{n[m['label']]}" for m in models], fontsize=8)
28 axes[1].set_ylim(0, 1.16)
29 axes[1].set_title("So sánh bốn mô hình trên tập test")
30 axes[1].legend(frameon=False, fontsize=7, ncol=3, loc="upper center")
31 axes[1].grid(True, axis="y", linewidth=0.3, alpha=0.4)
32
33 fig.tight_layout()
34 plt.show()

```



5. Học suất và công thức gốc của bài giảng

Ô lệnh dưới đây chạy đúng công thức trong tài liệu: gradient descent **toàn tập**, 1000 epoch, kiến trúc M1, với ba mức học suất 0.001, 0.01 và 0.1. Đây là bằng chứng cho phần lý giải ở mục 4, rằng với hơn 200 nghìn dòng thì 1000 lần cập nhật là quá ít.

Ô lệnh này chạy lâu nhất trong notebook vì mỗi epoch phải quét toàn bộ tập train.

In [12]

```

1 SPEC_EPOCHS = 1000
2 SPEC_RATES = (0.001, 0.01, 0.1)
3
4 lr_study = []
5 lr_curves = {}
6 for rate in SPEC_RATES:
7     _, row = fit(ARCHITECTURES[0], X_train, y_train, X_test, y_test, SPEC_EPOCHS, rate, None)
8     t = row["test_by_threshold"]["0.5"]
9     lr_curves[rate] = row["loss_history"]
10    lr_study.append({
11        "learning_rate": rate,
12        "epochs": SPEC_EPOCHS,
13        "batch_size": row["batch_size"],
14        "updates": row["updates"],
15        "loss_first": row["loss_first"],
16        "loss_final": row["loss_final"],
17        "accuracy": t["accuracy"],
18        "precision": t["precision"],
19        "recall": t["recall"],
20        "f1": t["f1"],
21        "roc_auc_test": row["roc_auc_test"],

```

```

22         "seconds": row["seconds"],
23     })
24     print(f"lr {rate:<6} loss {row['loss_first']:.4f} -> {row['loss_final']:.4f} "
25           f"acc {t1['accuracy']:.4f} rec {t1['recall']:.4f} f1 {t1['f1']:.4f} "
26           f"auc {row['roc_auc_test']:.4f} ({row['seconds']:.1f}s)")
27
28     m1 = models[0]
29     t1 = m1["test_by_threshold"]["0.5"]
30     print(f"\nĐối chiếu: M1 lô nhỏ 512, 60 epoch")
31     print(f"lr {m1['learning_rate']:<6} loss {m1['loss_first']:.4f} -> {m1['loss_final']:.4f} "
32           f"acc {t1['accuracy']:.4f} rec {t1['recall']:.4f} f1 {t1['f1']:.4f} "
33           f"auc {m1['roc_auc_test']:.4f} ({m1['seconds']:.1f}s, {m1['updates']:,} lần cập nhật)")

```

Out

```
lr 0.001 loss 0.5284 -> 0.4431 acc 0.8263 rec 0.0000 f1 0.0000 auc 0.6920 (232.0s)
```

Out

```
lr 0.01 loss 0.5284 -> 0.3904 acc 0.8263 rec 0.0000 f1 0.0000 auc 0.7746 (244.3s)
```

Out

```
lr 0.1 loss 0.5284 -> 0.3757 acc 0.8312 rec 0.1058 f1 0.1788 auc 0.8004 (199.2s)
```

Đối chiếu: M1 lô nhỏ 512, 60 epoch

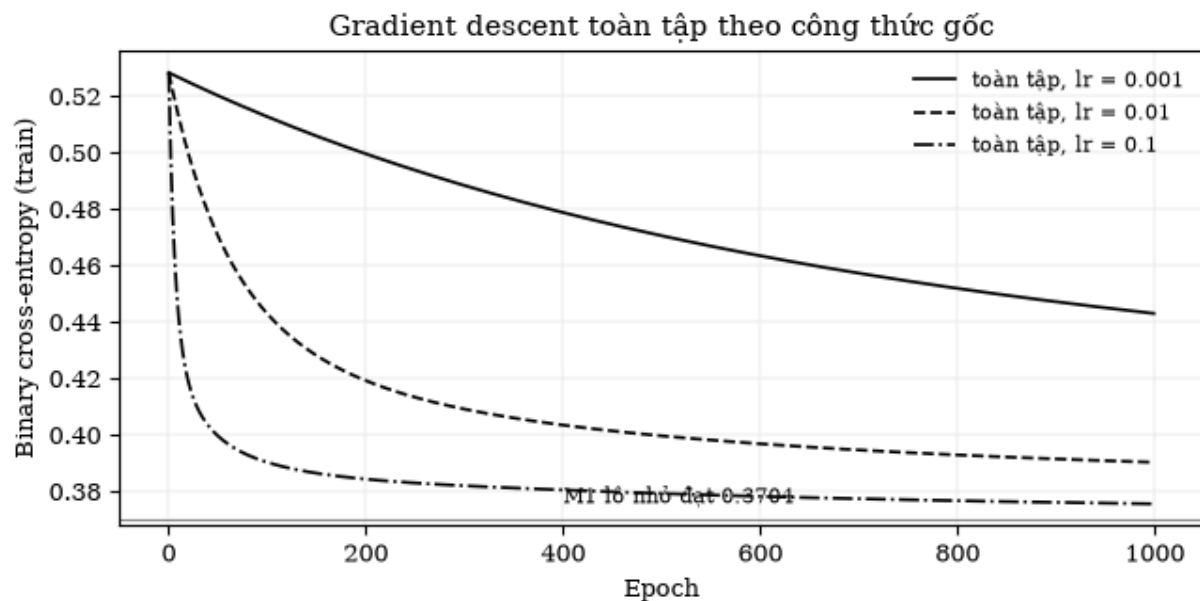
```
lr 0.05 loss 0.3848 -> 0.3704 acc 0.8373 rec 0.1940 f1 0.2929 auc 0.8071 (55.9s, 22,740 lần cập nhật)
```

In [13]

```

1  fig, axis = plt.subplots(figsize=(6.6, 3.4))
2  for rate, style in zip(SPEC_RATES, ["-", "--", "-.-"]):
3      axis.plot(lr_curves[rate], style, color="black", linewidth=1.2,
4               label=f"toàn tập, lr = {rate}")
5      axis.axhline(models[0]["loss_final"], color="black", linewidth=0.8, alpha=0.45)
6      axis.annotate(f"M1 lô nhỏ đạt {models[0]['loss_final']:.4f}",
7                   (SPEC_EPOCHS * 0.40, models[0]["loss_final"]), fontsize=8,
8                   xytext=(0, 6), textcoords="offset points")
9  axis.set_xlabel("Epoch")
10 axis.set_ylabel("Binary cross-entropy (train)")
11 axis.set_title("Gradient descent toàn tập theo công thức gốc")
12 axis.legend(frameon=False, fontsize=8)
13 axis.grid(True, linewidth=0.3, alpha=0.4)
14 fig.tight_layout()
15 plt.show()

```



6. Ghi kết quả ra file

Tất cả số liệu dùng trong báo cáo được ghi ra `reports/deeplearning_results.json` để báo cáo và notebook không bao giờ lệch nhau.

In [14]

```
1     payload = {
2         "data_file": DIABETES_CSV.relative_to(ROOT).as_posix(),
3         "features": list(FEATURES),
4         "feature_ranking": {k: round(float(v), 6) for k, v in ranking.items()},
5         "rows_raw": int(len(raw)),
6         "rows_after_dedup": int(len(frame)),
7         "n_train": int(len(X_train)),
8         "n_test": int(len(X_test)),
9         "positive_rate_train": round(float(y_train.mean()), 6),
10        "positive_rate_test": round(float(y_test.mean()), 6),
11        "split_seed": SPLIT_SEED,
12        "init_seed": INIT_SEED,
13        "shuffle_seed": SHUFFLE_SEED,
14        "learning_rate": LEARNING_RATE,
15        "epochs": EPOCHS,
16        "batch_size": BATCH_SIZE,
17        "thresholds": list(THRESHOLDS),
18        "models": models,
19        "learning_rate_study": lr_study,
20    }
21
22    out = REPORTS / "deeplearning_results.json"
23    out.write_text(json.dumps(payload, indent=2, ensure_ascii=False), encoding="utf-8")
24    print("đã ghi", out.relative_to(ROOT).as_posix(), f"({out.stat().st_size / 1024:.1f} KB)")
25    print("số mô hình:", len(payload["models"]))
26    print("mô hình tốt nhất theo ROC-AUC:",
27          max(payload["models"], key=lambda m: m["roc_auc_test"])["key"])
```

Out

```
đã ghi reports/deeplearning_results.json (18.9 KB)
số mô hình: 4
mô hình tốt nhất theo ROC-AUC: M2
```

Phụ lục B. Notebook 05: K-Nearest Neighbors

Toàn bộ nội dung notebook, giữ nguyên thứ tự cell. Mọi con số trong chương tương ứng của báo cáo đều sinh từ tệp kết quả mà notebook này ghi ra, nên báo cáo không thể lệch với notebook.

05_knn.ipynb

32 cell · 17 code

Notebook 05 - K-Nearest Neighbors trên ba category

Notebook này bổ sung **một họ thuật toán chưa từng xuất hiện** trong notebook 01, 02, 03: học dựa trên thực thể (instance based learning). Năm mô hình của ứng dụng tiểu đường, năm mô hình của ứng dụng giá nhà và bảy mô hình của ứng dụng e-commerce đều thuộc ba nhóm: tuyến tính, xác suất sinh, hoặc cây và tổ hợp cây. Không mô hình nào trong số đó ghi nhớ dữ liệu huấn luyện rồi trả lời bằng cách tra cứu hàng xóm gần nhất.

Phạm vi: ba category, bốn bài toán, vì category e-commerce có hai target riêng biệt.

| Category | Bài toán | Kiểu | Mốc so sánh (mô hình đã chọn ở notebook 01-03) | ---|---|---|---| | Tiểu đường | nguy cơ mắc bệnh | nhị phân | Random Forest | | Giá nhà | log(giá) | hồi quy | HistGradientBoosting | | E-commerce | mức hài lòng | nhị phân | LogReg trên bảng + TF-IDF | | E-commerce | nhóm sản phẩm quan tâm | 9 lớp | LogReg trên bảng + TF-IDF |

Điểm cần đọc trước khi so số: dữ liệu trong data/ đã được bổ sung sau khi notebook 01-03 chạy lần cuối (tiểu đường 253,680 lên 279,048 dòng, e-commerce 85,907 lên 94,498 dòng, giá nhà 2,226,382 lên 2,449,021 dòng). Vì vậy notebook này **huấn luyện lại mô hình mốc trên đúng bộ dữ liệu hiện tại** thay vì chép lại con số cũ. Chỉ khi cả hai mô hình cùng đọc một bộ dữ liệu, cùng một seed chia tập và cùng một quy trình đo thì chênh lệch giữa chúng mới quy được về thuật toán.

In [1]

```
1 import sys
2 import time
3 import warnings
4 from collections import Counter
5 from pathlib import Path
6
7 import numpy as np
8 import pandas as pd
9 from sklearn.neighbors import (
10     KNeighborsClassifier,
11     KNeighborsRegressor,
12     NearestNeighbors,
13 )
14
15 ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
16 if str(ROOT) not in sys.path:
17     sys.path.insert(0, str(ROOT))
18
19 import ml
20
21 warnings.filterwarnings("ignore")
22 pd.set_option("display.max_columns", 30)
23 pd.set_option("display.width", 140)
24
25 print("thư mục gốc :", ROOT)
26 print("numpy      :", np.__version__)
27 print("pandas      :", pd.__version__)
28 print("seed chung  :", ml.RANDOM_SEED)
```

Out

```
thư mục gốc : C:\Users\Hung Nguyen Ba\assignment03
numpy       : 2.4.6
pandas      : 3.0.3
seed chung  : 42
```

1. KNN hoạt động ra sao

KNN không có pha huấn luyện theo nghĩa thông thường. `fit` chỉ lưu lại toàn bộ tập train. Toàn bộ công việc dồn sang pha dự đoán: với mỗi điểm cần dự đoán x , thuật toán tính khoảng cách tới từng điểm trong tập train, lấy k điểm gần nhất rồi trả lời bằng đa số phiếu (phân loại) hoặc trung bình (hồi quy).

$$d(x, x_i) = \sqrt{\sum_{j=1}^d (x_j - x_{ij})^2} \quad \hat{y}(x) = \frac{1}{k} \sum_{i \in N_k(x)} y_i$$

Ba hệ quả trực tiếp, và cả ba đều xuất hiện trong kết quả bên dưới:

1. **Bắt buộc chuẩn hoá.** Khoảng cách cộng bình phương chênh lệch của mọi chiều. Nếu `house_size` chạy tới 10,000 còn `bed` chỉ tới 10 thì `bed` không còn ảnh hưởng gì. Cây quyết định cắt theo từng chiều riêng nên miễn nhiệm với chuyện này, KNN thì không.
2. **Chi phí nằm ở pha dự đoán.** `fit` gần như tức thì, nhưng mỗi lần dự đoán phải quét toàn bộ tập train. Cột `fit_s` và `predict_s` trong các bảng dưới đây cho thấy KNN đảo ngược hoàn toàn cấu trúc chi phí so với Random Forest.
3. **Không có tham số cân bằng lớp.** Các mô hình mặc định dùng `class_weight="balanced"`. KNN không có khái niệm tương đương: nếu 83% hàng xóm thuộc lớp âm thì phiếu bầu nghiêng về lớp âm. Đây là lý do chính khiến KNN xếp dưới ở bài toán tiểu đường.

Với dữ liệu nhiều chiều và thưa (TF-IDF của e-commerce có hơn 11,000 chiều), khoảng cách Euclid mất độ tương phản: mọi cặp điểm trở nên gần như cách đều nhau. Ở hai bài toán e-commerce, notebook dùng khoảng cách cosin, vốn là lựa chọn tiêu chuẩn cho vector TF-IDF vì nó chỉ quan tâm hướng của vector chứ không quan tâm độ dài văn bản.

2. Nạp bốn bài toán

Toàn bộ bước làm sạch, tạo biến phái sinh và chia 70/15/15 nằm trong `ml/datasets.py`, tái lập đúng các quyết định đã lập luận ở notebook 01-03. Đưa phần này ra một chỗ dùng chung để notebook 05 và notebook 06 chắc chắn đo trên cùng một tập train, val, test.

In [2]

```
1 t0 = time.perf_counter()
2 TASKS = {key: loader() for key, loader in ml.LOADERS.items()}
3 print(f"Nạp bốn bài toán trong {time.perf_counter() - t0:.1fs}\n")
4
5 for task in TASKS.values():
6     print(task.summary())
```

Out

```
Nạp bốn bài toán trong 11.9s

Tiểu đường (phân loại nhị phân): train=169,705 val=36,366 test=36,366 | mốc so sánh: Random Forest (native)
Giá nhà (hồi quy trên log giá): train=175,000 val=37,500 test=37,500 | mốc so sánh: HistGradientBoosting (target_enc)
E-commerce: mức hài lòng (bảng + văn bản): train=22,129 val=4,742 test=4,742 | mốc so sánh: LogReg bảng + TF-IDF
(tabular_text)
E-commerce: nhóm sản phẩm quan tâm (9 lớp): train=4,493 val=964 test=963 | mốc so sánh: LogReg bảng + TF-IDF
(tabular_text)
```

In [3]

```
1 pd.DataFrame([
2     {
3         "bài toán": task.title,
4         "kiểu": task.kind,
5         "train": f"{task.n_train:,}",
6         "val": f"{task.n_val:,}",
7         "test": f"{task.n_test:,}",
8         "mốc so sánh": task.baseline_name,
9     }
10    for task in TASKS.values()
11    ])
```

Out

	bài toán	kiểu	train	val	test	mốc so sánh
0	Tiểu đường (phân loại nhị phân)	binary	169,705	36,366	36,366	Random Forest
1	Giá nhà (hồi quy trên log giá)	regression	175,000	37,500	37,500	HistGradientBoosting
2	E-commerce: mức hài lòng (bảng + văn bản)	binary	22,129	4,742	4,742	LogReg bảng + TF-IDF
3	E-commerce: nhóm sản phẩm quan tâm (9 lớp)	multiclass	4,493	964	963	LogReg bảng + TF-IDF

3. Chọn k bằng một lần tìm hàng xóm duy nhất

Cách làm ngây thơ là huấn luyện lại KNN cho từng giá trị k trong lưới, tức là quét lại toàn bộ tập train bốn lần. Không cần thiết: danh sách hàng xóm của $k = 51$ đã chứa sẵn danh sách hàng xóm của $k = 5$, chỉ việc cắt bớt cột. Hàm dưới đây tìm hàng xóm đúng một lần với k lớn nhất trong lưới rồi suy ra dự đoán cho mọi k nhỏ hơn.

Với lá phiếu đều nhau (`weights="uniform"`), trung bình nhân của k hàng xóm chính là xác suất lớp dương, nên ngưỡng 0.5 trên trung bình đó tương đương đa số phiếu.

In [4]

```
1 def sweep_k(task, rep, k_grid, metric="euclidean"):
2     '''Quét lưới k bằng MỘT lần tìm hàng xóm, trả về bảng kết quả trên tập validation.'''
3     prep = task.prep(rep)
4     Z_train = prep.fit_transform(task.X_train, task.y_train)
5     Z_val = prep.transform(task.X_val)
6     y_train = np.asarray(task.y_train)
7
8     searcher = NearestNeighbors(
9         n_neighbors=max(k_grid), metric=metric, algorithm="brute", n_jobs=-1
10    ).fit(Z_train)
11
12    t0 = time.perf_counter()
13    _, index = searcher.kneighbors(Z_val)
14    search_seconds = time.perf_counter() - t0
15    neighbour_labels = y_train[index]
16
17    rows = []
18    for k in k_grid:
19        window = neighbour_labels[:, :k]
20        if task.kind == "regression":
21            prediction = window.mean(axis=1)
22            m = ml.regression_metrics(task.y_val, prediction)
23            rows.append({
24                "k": k, "R2_log": f"{m['r2_log']:.4f}",
25                "MAE_USD": f"{m['mae_usd']:.0f}", "MAPE_%": f"{m['mape_percent']:.1f}",
26                "_score": m["r2_log"],
27            })
28        elif task.kind == "binary":
29            proba = window.mean(axis=1)
30            m = ml.classification_metrics(task.y_val, (proba >= 0.5).astype(int), proba)
31            rows.append({
32                "k": k, "accuracy": f"{m['accuracy']:.4f}", "recall": f"{m['recall']:.4f}",
33                "f1": f"{m['f1']:.4f}", "f1_macro": f"{m['f1_macro']:.4f}",
34                "roc_auc": f"{m['roc_auc']:.4f}", "_score": m["f1_macro"],
35            })
36        else:
37            prediction = np.array([Counter(row).most_common(1)[0][0] for row in window])
38            m = ml.classification_metrics(task.y_val, prediction, average="macro")
39            rows.append({
40                "k": k, "accuracy": f"{m['accuracy']:.4f}",
41                "f1_macro": f"{m['f1_macro']:.4f}", "f1_weighted": f"{m['f1_weighted']:.4f}",
42                "_score": m["f1_macro"],
43            })
44
45    best = max(rows, key=lambda r: r["_score"])
46    print(f"Một lần tìm {max(k_grid)} hàng xóm cho {Z_val.shape[0]:,} điểm validation: "
47          f"{search_seconds:.1f}s (d = {Z_train.shape[1]:,}, metric = {metric})")
48    return rows, int(best["k"])
49
50
51 def show(rows, best_k):
52     frame = pd.DataFrame(rows).drop(columns="_score")
53     print(frame.to_string(index=False))
54     print(f"\n-> k tốt nhất trên validation: {best_k}")
```

4. Bài toán 1 - Tiêu đường (phân loại nhị phân)

Biểu diễn `native`: giữ nguyên 21 cột, chuẩn hoá ba cột liên tục và bốn cột thứ bậc, để nguyên 14 cột nhị phân vốn đã nằm trong khoảng $[0, 1]$. Đây đúng là biểu diễn mà Random Forest, Decision Tree và HistGradientBoosting đã dùng ở notebook 01.

In [5]

```
1 task = TASKS["diabetes"]
2 rows_dia, k_dia = sweep_k(task, "native", [5, 11, 25, 51, 101])
3 show(rows_dia, k_dia)
```

Out

```
Một lần tìm 101 hàng xóm cho 36,366 điểm validation: 16.4s (d = 21, metric = euclidean)
  k accuracy recall      f1 f1_macro roc_auc
5  0.8143 0.2599 0.3261  0.6092  0.7208
11 0.8271 0.2111 0.2968  0.5991  0.7587
25 0.8312 0.1731 0.2617  0.5832  0.7821
51 0.8328 0.1455 0.2313  0.5688  0.7935
101 0.8334 0.1252 0.2063  0.5566  0.7984

-> k tốt nhất trên validation: 5
```

Recall tụt thấy rõ khi k tăng, đúng như phần 1 đã dự báo: lớp dương chỉ chiếm khoảng 17% nên càng lấy nhiều hàng xóm, lá phiếu càng nghiêng về lớp âm. Accuracy đi ngược chiều với recall, và đó chính là cái bẫy mà notebook 01 đã cảnh báo ở bài toán ba lớp.

F1-macro được dùng làm tiêu chí chọn k vì nó tính điểm cả hai lớp một cách bình đẳng, không để lớp âm chiếm đa số kéo con số lên.

In [6]

```
1 results_dia = [
2     ml.run_model(task, f"KNN (k={k_dia})", "native",
3         KNeighborsClassifier(n_neighbors=k_dia, algorithm="brute", n_jobs=-1)),
4     ml.run_model(task, task.baseline_name + " (môc)", task.baseline_rep,
5         task.baseline_estimator()),
6 ]
7 ml.print_table([ml.table_row(task, r) for r in results_dia])
```

Out

model	d	accuracy	precision	recall	f1_macro	roc_auc	fit_s	predict_s
KNN (k=5)	21	0.8143	0.4375	0.2594	0.6090	0.7201	0.1	12.5
Random Forest (môc)	21	0.7252	0.3576	0.7399	0.6476	0.8076	12.6	0.5

In [7]

```
1 test_dia = {r["name"]: ml.evaluate_on_test(task, r["pipe"]) for r in results_dia}
2 for name, m in test_dia.items():
3     print(f"TEST {name:<34} accuracy={m['accuracy']:.4f} recall={m['recall']:.4f} "
4           f"f1={m['f1']:.4f} f1_macro={m['f1_macro']:.4f} auc={m['roc_auc']:.4f}")
```

Out

TEST	KNN (k=5)	accuracy=0.8176	recall=0.2632	f1=0.3329	f1_macro=0.6137	auc=0.7204
TEST	Random Forest (môc)	accuracy=0.7303	recall=0.7368	f1=0.4857	f1_macro=0.6515	auc=0.8123

Nhận xét. KNN xếp dưới Random Forest ở mọi chỉ số quan trọng của bài toán này, và khoảng cách nằm gần như trọn vẹn ở recall. Nguyên nhân đã nêu ở phần 1: Random Forest được đặt `class_weight="balanced_subsample"`, tức mỗi cây bị phạt nặng khi bỏ sót ca dương; KNN không có chỗ nào để gắn trọng số lớp, phiếu bầu của hàng xóm là phiếu bầu thô.

Cấu trúc chi phí đảo ngược hoàn toàn. Random Forest tốn phần lớn thời gian ở `fit_s` rồi dự đoán gần như tức thì. KNN có `fit_s` gần bằng không nhưng `predict_s` cao, vì mỗi lần dự đoán phải quét lại toàn bộ 169,705 dòng train. Với một dịch vụ REST phải trả lời từng yêu cầu một thì đây là điểm trừ nghiêm trọng, chứ không chỉ là chuyện đo cho vui.

5. Bài toán 2 - Giá nhà (hồi quy)

Biểu diễn `target_enc` nén 12,183 thành phố về một chiều số duy nhất, cho $d = 8$. Cây quyết định dùng thẳng biểu diễn này, nhưng KNN thì không: tám chiều đó có thang đo lệch nhau hàng nghìn lần. Vì vậy KNN dùng biến thể `target_enc_scaled`, chỉ khác ở chỗ ghép thêm một bước `StandardScaler` sau khi encode. Số chiều giữ nguyên là 8, chỉ thang đo đổi.

In [8]

```
1 task = TASKS["house_price"]
2 rows_house, k_house = sweep_k(task, "target_enc_scaled", [5, 11, 25, 51])
3 show(rows_house, k_house)
```

Out

```
Một lần tìm 51 hàng xóm cho 37,500 điểm validation: 13.2s (d = 8, metric = euclidean)
k R2_log MAE_USD MAPE_%
5 0.6015 141,296 36.1
11 0.6260 137,055 34.9
25 0.6350 135,423 34.4
51 0.6345 135,692 34.4

-> k tốt nhất trên validation: 25
```

In [9]

```
1 results_house = [
2     ml.run_model(task, f"KNN (k={k_house})", "target_enc_scaled",
3                     KNeighborsRegressor(n_neighbors=k_house, algorithm="brute", n_jobs=-1)),
4     ml.run_model(task, task.baseline_name + " (môc)", task.baseline_rep,
5                     task.baseline_estimator()),
6 ]
7 ml.print_table([ml.table_row(task, r) for r in results_house])
```

Out

model	d	R2_log	MAE_USD	RMSE_USD	R2_USD	MAPE_%	fit_s	predict_s
KNN (k=25)	8	0.6350	135,423	218,980	0.5738	34.4	0.5	12.4
HistGradientBoosting (môc)	8	0.7089	120,546	198,242	0.6507	29.8	7.6	0.6

In [10]

```
1 test_house = {r["name"]: ml.evaluate_on_test(task, r["pipe"]) for r in results_house}
2 for name, m in test_house.items():
3     print(f"TEST {name:<34} R2_log={m['r2_log']:.4f} MAE={m['mae_usd']:.0f} USD "
4           f"MAPE={m['mape_percent']:.1f}%")
```

Out

```
TEST KNN (k=25) R2_log=0.6377 MAE=135,206 USD MAPE=34.4%
TEST HistGradientBoosting (môc) R2_log=0.7095 MAE=121,023 USD MAPE=30.0%
```

Nhận xét. Đây là bài toán KNN bám sát môc nhất trong cả bốn bài toán, và có lý do rõ ràng: giá nhà là bài toán rất "địa phương". Hai căn cùng thành phố, cùng diện tích, cùng số phòng thì giá thật sự gần nhau, nên trung bình của vài chục hàng xóm là một ước lượng hợp lý. Đó chính xác là giả thiết mà KNN đặt ra.

Điều KNN không làm được là mô hình hoá tương tác giữa các chiều. HistGradientBoosting học được rằng ảnh hưởng của `house_size` lên giá khác nhau tùy theo bang, còn KNN chỉ biết cộng bình phương chênh lệch của tám chiều với trọng số như nhau.

6. Bài toán 3 - E-commerce, mức hài lòng (bảng + TF-IDF)

Biểu diễn ghép 7 cột số, 5 cột phân loại one-hot và TF-IDF của bình luận khách hàng, cho hơn 11,000 chiều. Ở số chiều này khoảng cách Euclid gần như mất tác dụng phân biệt, nên notebook chuyển sang khoảng cách cosin.

In [11]

```
1 task = TASKS["ecommerce_satisfaction"]
2 rows_sat, k_sat = sweep_k(task, "tabular_text", [5, 11, 25, 51], metric="cosine")
3 show(rows_sat, k_sat)
```

Out

```

Một lần tìm 51 hàng xóm cho 4,742 điểm validation: 3.2s (d = 11,835, metric = cosine)
k accuracy recall    f1 f1_macro roc_auc
5    0.7737 0.9212 0.8634 0.6027 0.7041
11   0.7868 0.9489 0.8735 0.5973 0.7213
25   0.7916 0.9622 0.8776 0.5894 0.7438
51   0.7906 0.9641 0.8772 0.5827 0.7510

-> k tốt nhất trên validation: 5

```

In [12]

```

1 results_sat = [
2     ml.run_model(task, f"KNN cosin (k={k_sat})", "tabular_text",
3                     KNeighborsClassifier(n_neighbors=k_sat, metric="cosine",
4                                         algorithm="brute", n_jobs=-1)),
5     ml.run_model(task, task.baseline_name + " (môc)", task.baseline_rep,
6                     task.baseline_estimator()),
7 ]
8 ml.print_table([ml.table_row(task, r) for r in results_sat])

```

Out

model	d	accuracy	precision	recall	f1_macro	roc_auc	fit_s	predict_s
KNN cosin (k=5)	11,835	0.7737	0.8124	0.9212	0.6027	0.7041	0.8	2.6
LogReg bảng + TF-IDF (môc)	11,835	0.8269	0.9323	0.8378	0.7769	0.8894	2.5	0.5

In [13]

```

1 test_sat = {r["name"]: ml.evaluate_on_test(task, r["pipe"]) for r in results_sat}
2 for name, m in test_sat.items():
3     print(f"TEST {name:<34} accuracy={m['accuracy']:.4f} recall={m['recall']:.4f} "
4           f"f1={m['f1']:.4f} f1_macro={m['f1_macro']:.4f} auc={m['roc_auc']:.4f}")

```

Out

TEST KNN cosin (k=5)	accuracy=0.7777	recall=0.9239	f1=0.8658	f1_macro=0.6096	auc=0.7041
TEST LogReg bảng + TF-IDF (môc)	accuracy=0.8323	recall=0.8378	f1=0.8858	f1_macro=0.7853	auc=0.8952

Nhận xét. Hồi quy logistic học một trọng số riêng cho từng từ trong hơn 11,000 chiều, tức là nó biết từ "refund" quan trọng hơn từ "order" bao nhiêu lần. KNN không học trọng số nào cả: mọi từ trong vector TF-IDF đóng góp vào khoảng cách theo đúng giá trị TF-IDF của nó. Với văn bản ngắn và nhiều như bình luận chăm sóc khách hàng, khoảng cách tổng hợp đó dễ bị chi phối bởi các từ phổ biến nhưng vô nghĩa về mặt cảm xúc.

7. Bài toán 4 - E-commerce, nhóm sản phẩm quan tâm (9 lớp)

Đây là bài toán yếu nhất của cả dự án: notebook 03 chỉ đạt F1-macro khoảng 0.27 và đã chỉ ra lý do là bình luận nói về chất lượng dịch vụ chứ không nói về sản phẩm. Trước notebook này, bài toán đó mới chỉ được thử bằng đúng một thuật toán.

In [14]

```

1 task = TASKS["ecommerce_interest"]
2 print("Phân bố lớp trên tập train:")
3 print(pd.Series(task.y_train).value_counts().to_string())
4 print()
5 rows_int, k_int = sweep_k(task, "tabular_text", [5, 11, 25, 51], metric="cosine")
6 show(rows_int, k_int)

```

Out

```

Phân bố lớp trên tập train:
y_interest
Electronics      1322
LifeStyle         1008
Books & General merchandise  818
Mobile           476
Home Appliances  359
Home             315
Furniture        141

```

Affiliates	45
GiftCard	9

Out

```
Một lần tìm 51 hàng xóm cho 964 điểm validation: 0.3s (d = 3,452, metric = cosine)
k accuracy f1_macro f1_weighted
5 0.3465 0.2391 0.3343
11 0.3631 0.2208 0.3415
25 0.3849 0.2289 0.3463
51 0.3703 0.2137 0.3260

-> k tốt nhất trên validation: 5
```

In [15]

```
1 results_int = [
2     ml.run_model(task, f"KNN cosin (k={k_int})", "tabular_text",
3                     KNeighborsClassifier(n_neighbors=k_int, metric="cosine",
4                                           algorithm="brute", n_jobs=-1)),
5     ml.run_model(task, task.baseline_name + " (móc)", task.baseline_rep,
6                     task.baseline_estimator()),
7 ]
8 ml.print_table([ml.table_row(task, r) for r in results_int])
```

Out

model	d	accuracy	precision	recall	f1_macro	roc_auc	fit_s	predict_s
KNN cosin (k=5)	3,452	0.3288	0.2882	0.2474	0.2550	-	0.2	0.2
LogReg bảng + TF-IDF (móc)	3,452	0.3672	0.3097	0.3160	0.2959	-	1.3	0.0

In [16]

```
1 test_int = {r["name"]: ml.evaluate_on_test(task, r["pipe"]) for r in results_int}
2 for name, m in test_int.items():
3     print(f"TEST {name:<34} accuracy={m['accuracy']:.4f} f1_macro={m['f1_macro']:.4f} "
4           f"f1_weighted={m['f1_weighted']:.4f}")
```

Out

TEST	KNN cosin (k=5)	accuracy=0.3946	f1_macro=0.2371	f1_weighted=0.3888
TEST	LogReg bảng + TF-IDF (móc)	accuracy=0.4029	f1_macro=0.3368	f1_weighted=0.4305

Nhận xét. KNN xếp dưới ở cả accuracy lẫn F1-macro, và khoảng cách ở F1-macro rộng hơn hẳn. Lý do nằm ở phân bố lớp in ngay phía trên: Electronics có 1,322 mẫu train còn GiftCard chỉ có 9. Với chín lớp lệch tới mức đó, năm hàng xóm gần nhất của một điểm bất kỳ gần như luôn thuộc vài lớp đồng nhất, nên các lớp hiếm không bao giờ giành được đa số phiếu và F1 của chúng bằng không. LogReg có `class_weight="balanced"` nên ít nhất còn cố trải dự đoán ra cả chín lớp.

Kết luận của notebook 03 vẫn đứng vững: giới hạn nằm ở chỗ bình luận nói về chất lượng dịch vụ chứ không nói về sản phẩm. Đổi thuật toán không cứu được một biểu diễn thiếu tín hiệu.

8. Tổng hợp và ghi kết quả

In [17]

```
1 def block(task, results, test_metrics, sweep_rows, best_k, metric):
2     return {
3         "task": task.key,
4         "title": task.title,
5         "kind": task.kind,
6         "n_train": task.n_train,
7         "n_val": task.n_val,
8         "n_test": task.n_test,
9         "audit": task.audit,
10        "distance": metric,
11        "k_grid": [int(r["k"]) for r in sweep_rows],
12        "k_selected": best_k,
13        "sweep": [{k: v for k, v in r.items() if k != "_score"} for r in sweep_rows],
```

```

14         "validation_table": [ml.table_row(task, r) for r in results],
15         "test_metrics": {name: m for name, m in test_metrics.items()},
16         "model_names": [r["name"] for r in results],
17     }
18
19
20     payload = {
21         "algorithm": "K-Nearest Neighbors",
22         "library": "scikit-learn",
23         "seed": ml.RANDOM_SEED,
24         "note": (
25             "Mô hình mốc được huấn luyện lại trên bộ dữ liệu hiện tại trong data/, "
26             "không chép lại số của notebook 01-03."
27         ),
28         "tasks": [
29             block(TASKS["diabetes"], results_dia, test_dia, rows_dia, k_dia, "euclidean"),
30             block(TASKS["house_price"], results_house, test_house, rows_house, k_house, "euclidean"),
31             block(TASKS["ecommerce_satisfaction"], results_sat, test_sat, rows_sat, k_sat, "cosine"),
32             block(TASKS["ecommerce_interest"], results_int, test_int, rows_int, k_int, "cosine"),
33         ],
34     }
35
36     path = ml.save_results("knn", payload)
37     print("Đã ghi", path.relative_to(ROOT), f"({path.stat().st_size / 1024:.1f} KB)")

```

Out

```

Đã ghi reports\knn_results.json (11.2 KB)

```

9. Kết luận

KNN xếp dưới mô hình mốc ở cả bốn bài toán, và đó là kết quả đúng như dự đoán chứ không phải dấu hiệu của lỗi cài đặt. Giá trị của nó nằm ở chỗ khác.

KNN làm rõ ba điều mà các mô hình cũ che mắt:

1. **Chuẩn hoá không phải thủ tục hình thức.** Ba biểu diễn của notebook 01-03 đều được thiết kế cho cây quyết định, vốn không quan tâm thang đo. Chỉ khi đưa một mô hình dựa trên khoảng cách vào thì mới thấy biểu diễn `target_enc` chưa dùng được ngay, phải thêm một bước scale.
2. **Số chiều lớn không miễn phí.** Hơn 11,000 chiều TF-IDF là chuyện bình thường với LogReg nhưng làm khoảng cách Euclid mất ý nghĩa, buộc phải đổi sang cosin.
3. **Chi phí suy luận là một tiêu chí chọn mô hình.** Bảng nào cũng có `fit_s` và `predict_s`. KNN là mô hình duy nhất trong cả dự án có `predict_s` lớn hơn `fit_s`, và với dịch vụ REST đang chạy thì đó là lý do đủ để loại, độc lập với chất lượng dự đoán.

Mô hình được triển khai lên API **không thay đổi** sau notebook này. KNN được thêm vào để mở rộng bảng so sánh chứ không phải để thay thế lựa chọn cũ.

Phụ lục C. Notebook 06: AdaBoost

Toàn bộ nội dung notebook, giữ nguyên thứ tự cell. Mọi con số trong chương tương ứng của báo cáo đều sinh từ tệp kết quả mà notebook này ghi ra, nên báo cáo không thể lệch với notebook.

06_adaboost.ipynb

30 cell · 16 code

Notebook 06 - AdaBoost trên ba category

Notebook 05 thêm một họ hoàn toàn mới (học dựa trên thực thể). Notebook này đi theo hướng ngược lại: thêm một thuật toán **cùng nhóm boosting** với HistGradientBoosting đã có, để tách bạch xem phần thắng của boosting đến từ ý tưởng "ghép nhiều bộ học yếu" hay đến từ cách cụ thể mà gradient boosting thực hiện ý tưởng đó.

Hai thuật toán boosting khác nhau ở chỗ nào:

| | AdaBoost (SAMME, AdaBoost.R2) | HistGradientBoosting | |---|---|---| | Bộ học yếu học cái gì | vẫn học nhân gốc y | học **gradient** của hàm mất mát | | Cách nhả vào mẫu khó | **tăng trọng số** mẫu bị dự đoán sai | đưa phần dư vào làm target mới | | Gộp kết quả | bỏ phiếu có trọng số α_m | cộng thẳng các cây | | Đặc trưng liên tục | dùng giá trị gốc | **bấm thành 256 thùng** trước khi cắt |

Phạm vi và mốc so sánh giữ nguyên như notebook 05: ba category, bốn bài toán, mốc là mô hình đã được chọn ở notebook 01-03 và được huấn luyện lại trên bộ dữ liệu hiện tại.

In [1]

```
1 import sys
2 import time
3 import warnings
4 from pathlib import Path
5
6 import numpy as np
7 import pandas as pd
8 from sklearn.ensemble import AdaBoostClassifier, AdaBoostRegressor
9 from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
10 from sklearn.utils.class_weight import compute_sample_weight
11
12 ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
13 if str(ROOT) not in sys.path:
14     sys.path.insert(0, str(ROOT))
15
16 import ml
17
18 warnings.filterwarnings("ignore")
19 pd.set_option("display.max_columns", 30)
20 pd.set_option("display.width", 140)
21
22 print("thư mục gốc :", ROOT)
23 print("numpy      :", np.__version__)
24 print("pandas      :", pd.__version__)
25 print("seed chung  :", ml.RANDOM_SEED)
```

Out

```
thư mục gốc : C:\Users\Hung Nguyen Ba\assignment03
numpy      : 2.4.6
pandas     : 3.0.3
seed chung  : 42
```

1. AdaBoost hoạt động ra sao

Mỗi mẫu huấn luyện mang một trọng số w_i , ban đầu bằng nhau. Ở vòng thứ m , thuật toán huấn luyện một bộ học yếu h_m trên tập đã gán trọng số, đo tỉ lệ lỗi có trọng số, rồi dùng tỉ lệ đó để quyết định hai chuyện: bộ học yếu này có tiếng nói nặng bao nhiêu trong lá phiếu cuối, và mẫu nào cần được nhấn mạnh hơn ở vòng sau.

$$\mathrm{err}_m = \frac{\sum_i w_i \mathbb{1}[y_i \neq h_m(x_i)]}{\sum_i w_i} \quad \alpha_m = \ln \frac{1 - \mathrm{err}_m}{\mathrm{err}_m} \quad \beta_m = \ln(K - 1) \alpha_m$$

$$\|w_i\| \leftarrow w_i \cdot \exp(\alpha_m \cdot \mathbb{1}[y_i \neq h_m(x_i)]) \quad \hat{y}(x) = \arg\max_c \sum_m \alpha_m \cdot \mathbb{1}[h_m(x) = c]$$

Đọc ba công thức trên theo thứ tự thì thấy toàn bộ cơ chế: bộ học yếu nào sai nhiều thì α_m nhỏ, tiếng nói yếu; mẫu nào bị đoán sai thì trọng số nhân lên e^{α_m} , nên vòng sau bộ học yếu buộc phải chú ý tới nó.

Hai điểm cần ghi rõ về phiên bản thư viện đang dùng:

- scikit-learn 1.9 **chỉ còn thuật toán SAMME**. Biến thể SAMME.R (dùng xác suất thay vì nhân cứng) đã bị gỡ bỏ, và tham số `algorithm` cũng không còn tồn tại. Tham số đặt bộ học yếu tên là `estimator`, không phải `base_estimator` như tài liệu cũ.

- Bài toán hồi quy dùng **AdaBoost.R2**, một biến thể khác: trọng số cập nhật theo sai số tuyệt đối đã chuẩn hoá, và dự đoán cuối là **trung vị có trọng số** của các bộ học yếu chứ không phải trung bình.

Bộ học yếu ở đây là cây nông. Nông tới mức nào là một lựa chọn thiết kế: cây càng sâu thì mỗi vòng học được nhiều hơn nhưng dễ khớp quá và làm boosting mất tác dụng.

2. Nạp bốn bài toán

Dùng lại nguyên `ml/datasets.py` như notebook 05, nên tập train, val, test của hai notebook giống hệt nhau tới từng dòng.

In [2]

```
1 t0 = time.perf_counter()
2 TASKS = {key: loader() for key, loader in ml.LOADERS.items()}
3 print(f"Nạp bốn bài toán trong {time.perf_counter() - t0:.1f}s\n")
4 for task in TASKS.values():
5     print(task.summary())
```

Out

Nạp bốn bài toán trong 11.3s

Tiểu đường (phân loại nhị phân): train=169,705 val=36,366 test=36,366 | mốc so sánh: Random Forest (native)
 Giá nhà (hồi quy trên log giá): train=175,000 val=37,500 test=37,500 | mốc so sánh: HistGradientBoosting (target_enc)
 E-commerce: mức hài lòng (bảng + văn bản): train=22,129 val=4,742 test=4,742 | mốc so sánh: LogReg bảng + TF-IDF (tabular_text)
 E-commerce: nhóm sản phẩm quan tâm (9 lớp): train=4,493 val=964 test=963 | mốc so sánh: LogReg bảng + TF-IDF (tabular_text)

3. Quét số vòng boosting bằng một lần huấn luyện duy nhất

Giống chuyện k của KNN, không cần huấn luyện lại cho từng giá trị `n_estimators`. Một mô hình đã học 300 vòng chứa sẵn trong nó mọi mô hình 25, 50, 100, 200 vòng: chỉ việc dừng cộng phiếu sớm hơn. scikit-learn cung cấp sẵn `staged_predict` và `staged_predict_proba` để lấy dự đoán tại từng vòng.

Đây cũng là cách nhìn thấy AdaBoost có bị khớp quá hay không: nếu chất lượng trên validation đạt đỉnh rồi đi xuống thì thêm vòng chỉ tổ hại.

In [3]

```
1 def sweep_stages(task, rep, estimator, stage_grid, sample_weight=None):
2     '''Huấn luyện MỘT lần với số vòng lớn nhất, đọc kết quả tại từng mốc trong lưới.'''
3     prep = task.prep(rep)
4     Z_train = prep.fit_transform(task.X_train, task.y_train)
5     Z_val = prep.transform(task.X_val)
6
7     t0 = time.perf_counter()
8     estimator.fit(Z_train, task.y_train, sample_weight=sample_weight)
9     fit_seconds = time.perf_counter() - t0
10    n_fitted = len(estimator.estimators_)
11
12    wanted = {s for s in stage_grid if s <= n_fitted}
13    rows = []
14
15    t0 = time.perf_counter()
16    if task.kind == "regression":
17        for stage, prediction in enumerate(estimator.staged_predict(Z_val), start=1):
18            if stage not in wanted:
19                continue
```

```

20         m = ml.regression_metrics(task.y_val, prediction)
21         rows.append({
22             "vòng": stage, "R2_log": f"{m['r2_log']:.4f}",
23             "MAE_USD": f"{m['mae_usd']:.0f}", "MAPE_%": f"{m['mape_percent']:.1f}",
24             "_score": m["r2_log"],
25         })
26     else:
27         classes = estimator.classes_
28         for stage, proba in enumerate(estimator.staged_predict_proba(Z_val), start=1):
29             if stage not in wanted:
30                 continue
31             prediction = classes[proba.argmax(axis=1)]
32             score = proba[:, 1] if task.average == "binary" else None
33             m = ml.classification_metrics(
34                 task.y_val, prediction, score, average=task.average
35             )
36             row = {
37                 "vòng": stage, "accuracy": f"{m['accuracy']:.4f}",
38                 "recall": f"{m['recall']:.4f}", "f1_macro": f"{m['f1_macro']:.4f}",
39                 "_score": m["f1_macro"],
40             }
41             if m["roc_auc"] is not None:
42                 row["roc_auc"] = f"{m['roc_auc']:.4f}"
43             rows.append(row)
44         stage_seconds = time.perf_counter() - t0
45
46         best = max(rows, key=lambda r: r["_score"])
47         print(f"Huấn luyện {n_fitted} vòng trong {fit_seconds:.1f}s, "
48               f"đọc {len(rows)} mẫu trong {stage_seconds:.1f}s (d = {Z_train.shape[1]:,})")
49         print(pd.DataFrame(rows).drop(columns="_score").to_string(index=False))
50         print(f"\n-> số vòng tốt nhất trên validation: {best['vòng']}")
51         return rows, int(best["vòng"])

```

4. Bài toán 1 - Tiểu đường (phân loại nhị phân)

Bộ học yếu là cây sâu 3 tầng. Lớp dương chỉ chiếm 17% nên vẫn cần cân bằng lớp, nhưng **không được đặt** `class_weight="balanced"` lên cây, dù đó là cách mọi mô hình khác trong dự án đang dùng.

Lý do là một xung đột thật sự, do được chứ không phải suy đoán: AdaBoost tính tỉ lệ lỗi err_m theo bộ trọng số của nó, trong khi `class_weight` khiến cây tối ưu theo một bộ trọng số khác. Hai bộ lệch nhau làm err_m vọt lên 0.3963 ngay vòng đầu và chạm ngưỡng dừng sau đúng **5 vòng**, tức là ensemble 300 vòng không bao giờ được dựng. Cách đúng là truyền trọng số cân bằng lớp vào **giá trị khởi tạo** của `sw_i` qua tham số `sample_weight` của `fit`, để AdaBoost tiếp tục cập nhật chính bộ trọng số đó qua từng vòng.

In [4]

```

1     task = TASKS["diabetes"]
2     ada_dia = AdaBoostClassifier(
3         estimator=DecisionTreeClassifier(max_depth=3, random_state=ml.RANDOM_SEED),
4         n_estimators=300, learning_rate=0.5, random_state=ml.RANDOM_SEED,
5     )
6     weight_dia = compute_sample_weight("balanced", task.y_train)
7     rows_dia, m_dia = sweep_stages(task, "native", ada_dia, [25, 50, 100, 200, 300], weight_dia)

```

Out

```

Huấn luyện 300 vòng trong 85.1s, đọc 5 mẫu trong 1.8s (d = 21)
vòng accuracy recall f1_macro roc_auc
25    0.7100 0.7527  0.6365  0.8000
50    0.7138 0.7522  0.6396  0.8027
100   0.7133 0.7535  0.6394  0.8029
200   0.7125 0.7578  0.6393  0.8037
300   0.7126 0.7576  0.6393  0.8037

```

```

-> số vòng tốt nhất trên validation: 50

```

In [5]

```

1     results_dia = [

```

```

2         ml.run_model(task, f"AdaBoost ({m_dia} vòng, cây sâu 3)", "native",
3                       AdaBoostClassifier(
4                           estimator=DecisionTreeClassifier(max_depth=3, random_state=ml.RANDOM_SEED),
5                           n_estimators=m_dia, learning_rate=0.5, random_state=ml.RANDOM_SEED),
6                           sample_weight=weight_dia),
7         ml.run_model(task, task.baseline_name + " (mốc)", task.baseline_rep,
8                       task.baseline_estimator()),
9     ]
10    ml.print_table([ml.table_row(task, r) for r in results_dia])

```

Out

model	d	accuracy	precision	recall	f1_macro	roc_auc	fit_s	predict_s
AdaBoost (50 vòng, cây sâu 3)	21	0.7138	0.3483	0.7522	0.6396	0.8027	14.5	0.2
Random Forest (mốc)	21	0.7252	0.3576	0.7399	0.6476	0.8076	11.2	0.4

In [6]

```

1     test_dia = {r["name"]: ml.evaluate_on_test(task, r["pipe"]) for r in results_dia}
2     for name, m in test_dia.items():
3         print(f"TEST {name:<36} accuracy={m['accuracy']:.4f} recall={m['recall']:.4f} "
4               f"f1={m['f1']:.4f} f1_macro={m['f1_macro']:.4f} auc={m['roc_auc']:.4f}")

```

Out

TEST AdaBoost (50 vòng, cây sâu 3)	accuracy=0.7192	recall=0.7498	f1=0.4800	f1_macro=0.6438	auc=0.8071
TEST Random Forest (mốc)	accuracy=0.7303	recall=0.7368	f1=0.4857	f1_macro=0.6515	auc=0.8123

Nhận xét. AdaBoost bám rất sát Random Forest ở bài toán này, sát hơn hẳn KNN của notebook 05. Điều đó hợp lý: cả hai đều là tổ hợp cây và cùng được cân bằng lớp, nên chúng nhìn dữ liệu theo cùng một kiểu. Chênh lệch còn lại đến từ cách gộp: Random Forest lấy trung bình của các cây độc lập (giảm phương sai), AdaBoost cộng dồn các cây phụ thuộc nhau (giảm độ chệch).

5. Bài toán 2 - Giá nhà (hồi quy, AdaBoost.R2)

Ở đây bộ học yếu phải sâu hơn hẳn (8 tầng thay vì 3). Lý do: AdaBoost.R2 gộp kết quả bằng trung vị có trọng số, và trung vị của một đám cây quá nông thì gần như không nhích khỏi giá trị trung tâm, dù có cộng bao nhiêu vòng đi nữa.

In [7]

```

1     task = TASKS["house_price"]
2     ada_house = AdaBoostRegressor(
3         estimator=DecisionTreeRegressor(max_depth=8, random_state=ml.RANDOM_SEED),
4         n_estimators=100, learning_rate=0.5, random_state=ml.RANDOM_SEED,
5     )
6     rows_house, m_house = sweep_stages(task, "target_enc", ada_house, [10, 25, 50, 75, 100])

```

Out

```

Huấn luyện 100 vòng trong 60.6s, đọc 5 mốc trong 32.6s (d = 8)
vòng R2_log MAE_USD MAPE_%
10 0.6307 137,338 34.0
25 0.6332 138,369 33.6
50 0.6298 140,009 33.5
75 0.6232 141,688 33.6
100 0.6159 143,167 33.8

-> số vòng tốt nhất trên validation: 25

```

In [8]

```

1     results_house = [
2         ml.run_model(task, f"AdaBoost.R2 ({m_house} vòng, cây sâu 8)", "target_enc",
3                       AdaBoostRegressor(
4                           estimator=DecisionTreeRegressor(max_depth=8, random_state=ml.RANDOM_SEED),
5                           n_estimators=m_house, learning_rate=0.5, random_state=ml.RANDOM_SEED)),
6         ml.run_model(task, task.baseline_name + " (mốc)", task.baseline_rep,
7                       task.baseline_estimator()),
8     ]

```

```
9 ml.print_table([ml.table_row(task, r) for r in results_house])
```

Out

model	d	R2_log	MAE_USD	RMSE_USD	R2_USD	MAPE_%	fit_s	predict_s
AdaBoost.R2 (25 vòng, cây sâu 8)	8	0.6332	138,369	223,518	0.5559	33.6	20.9	0.2
HistGradientBoosting (mốc)	8	0.7089	120,546	198,242	0.6507	29.8	10.3	0.7

In [9]

```
1 test_house = {r["name"]: ml.evaluate_on_test(task, r["pipe"]) for r in results_house}
2 for name, m in test_house.items():
3     print(f"TEST {name:<36} R2_log={m['r2_log']:.4f} MAE={m['mae_usd']:.0f} USD "
4           f"MAPE={m['mape_percent']:.1f}%")
```

Out

```
TEST AdaBoost.R2 (25 vòng, cây sâu 8)      R2_log=0.6355 MAE=138,182 USD MAPE=33.7%
TEST HistGradientBoosting (mốc)            R2_log=0.7095 MAE=121,023 USD MAPE=30.0%
```

Nhận xét. Bảng quét số vòng cho thấy AdaBoost.R2 đạt đỉnh ở vòng 25 rồi **đi xuống**: $R^2(\log)$ tụt từ 0.6332 còn 0.6159 khi chạy hết 100 vòng. Đây là khớp quá thấy rõ, và nó khác hẳn gradient boosting vốn còn cải thiện đều tới vài trăm vòng.

Nguyên nhân nằm ở chính định nghĩa của AdaBoost.R2: nó cập nhật trọng số theo sai số tuyệt đối đã chuẩn hoá, tức là chỉ biết "mẫu này sai nhiều hơn mẫu kia", chứ không biết **sai theo hướng nào**. Càng nhiều vòng, trọng số càng dồn vào những căn nhà có giá bất thường mà không mô hình nào đoán nổi, và các cây sau bị kéo lệch theo. Gradient boosting học bằng phần dư có dấu nên mỗi vòng sửa được đúng chiều còn thiếu.

Đây là câu trả lời cho câu hỏi đặt ra ở đầu notebook: phần thắng của HistGradientBoosting không đến từ ý tưởng boosting nói chung, mà đến từ việc học gradient thay vì chỉ đổi trọng số mẫu. Khoảng cách 0.7089 so với 0.6332 trên cùng một biểu diễn 8 chiều là khoảng cách giữa hai cách hiện thực hoá cùng một ý tưởng.

6. Bài toán 3 - E-commerce, mức hài lòng (bảng + TF-IDF)

Hơn 11,000 chiều TF-IDF thưa. Cây quyết định xử lý ma trận thưa không có vấn đề gì, nhưng mỗi cây sâu 3 tầng chỉ chọn được tối đa 7 từ khoá trong số hơn 11,000 chiều, nên cần khá nhiều vòng mới phủ hết từ vựng có ích.

In [10]

```
1 task = TASKS["ecommerce_satisfaction"]
2 ada_sat = AdaBoostClassifier(
3     estimator=DecisionTreeClassifier(max_depth=3, random_state=ml.RANDOM_SEED),
4     n_estimators=300, learning_rate=0.5, random_state=ml.RANDOM_SEED,
5 )
6 weight_sat = compute_sample_weight("balanced", task.y_train)
7 rows_sat, m_sat = sweep_stages(task, "tabular_text", ada_sat, [25, 50, 100, 200, 300], weight_sat)
```

Out

```
Huấn luyện 300 vòng trong 32.7s, đọc 5 mốc trong 0.5s (d = 11,835)
vòng accuracy recall f1_macro roc_auc
25  0.7448 0.7084  0.7081 0.8590
50  0.7583 0.7283  0.7195 0.8677
100 0.7725 0.7473  0.7323 0.8748
200 0.7803 0.7592  0.7389 0.8797
300 0.7946 0.7829  0.7505 0.8824

-> số vòng tốt nhất trên validation: 300
```

In [11]

```
1 results_sat = [
2     ml.run_model(task, f"AdaBoost ({m_sat} vòng, cây sâu 3)", "tabular_text",
3                     AdaBoostClassifier(
4                         estimator=DecisionTreeClassifier(max_depth=3, random_state=ml.RANDOM_SEED),
5                         n_estimators=m_sat, learning_rate=0.5, random_state=ml.RANDOM_SEED),
6                         sample_weight=weight_sat),
7     ml.run_model(task, task.baseline_name + " (mốc)", task.baseline_rep,
8                     task.baseline_estimator()),
```

```

9 ]
10 ml.print_table([ml.table_row(task, r) for r in results_sat])

```

Out

model	d	accuracy	precision	recall	f1_macro	roc_auc	fit_s	predict_s
AdaBoost (300 vòng, cây sâu 3)	11,835	0.7946	0.9427	0.7829	0.7505	0.8824	34.1	0.3
LogReg bảng + TF-IDF (mốc)	11,835	0.8269	0.9323	0.8378	0.7769	0.8894	2.0	0.1

In [12]

```

1 test_sat = {r["name"]: ml.evaluate_on_test(task, r["pipe"]) for r in results_sat}
2 for name, m in test_sat.items():
3     print(f"TEST {name:<36} accuracy={m['accuracy']:.4f} recall={m['recall']:.4f} "
4           f"f1={m['f1']:.4f} f1_macro={m['f1_macro']:.4f} auc={m['roc_auc']:.4f}")

```

Out

TEST AdaBoost (300 vòng, cây sâu 3)	accuracy=0.7902	recall=0.7799	f1=0.8523	f1_macro=0.7452	auc=0.8785
TEST LogReg bảng + TF-IDF (mốc)	accuracy=0.8323	recall=0.8378	f1=0.8858	f1_macro=0.7853	auc=0.8952

Nhận xét. Notebook 03 đã ghi nhận một điều mà kết quả ở đây củng cố thêm: với dữ liệu văn bản dạng TF-IDF, mô hình tuyến tính khó bị đánh bại. Hồi quy logistic gán một trọng số cho **mọi** chiều trong một lần tối ưu, trong khi mỗi cây của AdaBoost chỉ chạm được vài chiều. Muốn phủ hết từ vựng, số vòng phải rất lớn, và chi phí huấn luyện tăng theo.

7. Bài toán 4 - E-commerce, nhóm sản phẩm quan tâm (9 lớp)

Với 9 lớp, số hạng $\ln(K-1) = \ln 8$ trong công thức α_m trở nên quan trọng: nó cho phép bộ học yếu vẫn có đóng góp dương ngay cả khi tỉ lệ lỗi cao hơn 50%, miễn là còn tốt hơn đoán mò (khoảng 11% cho 9 lớp cân bằng).

In [13]

```

1 task = TASKS["ecommerce_interest"]
2 ada_int = AdaBoostClassifier(
3     estimator=DecisionTreeClassifier(max_depth=3, random_state=ml.RANDOM_SEED),
4     n_estimators=300, learning_rate=0.5, random_state=ml.RANDOM_SEED,
5 )
6 weight_int = compute_sample_weight("balanced", task.y_train)
7 rows_int, m_int = sweep_stages(task, "tabular_text", ada_int, [25, 50, 100, 200, 300], weight_int)

```

Out

Huấn luyện 300 vòng trong 12.0s, đọc 5 mốc trong 0.6s (d = 3,452)				
vòng	accuracy	recall	f1_macro	
25	0.3662	0.3172	0.2678	
50	0.4035	0.3191	0.2941	
100	0.4139	0.3134	0.2899	
200	0.3859	0.3578	0.3190	
300	0.4035	0.3440	0.3177	

-> số vòng tốt nhất trên validation: 200

In [14]

```

1 results_int = [
2     ml.run_model(task, f"AdaBoost ({m_int} vòng, cây sâu 3)", "tabular_text",
3         AdaBoostClassifier(
4             estimator=DecisionTreeClassifier(max_depth=3, random_state=ml.RANDOM_SEED),
5             n_estimators=m_int, learning_rate=0.5, random_state=ml.RANDOM_SEED),
6             sample_weight=weight_int),
7     ml.run_model(task, task.baseline_name + " (mốc)", task.baseline_rep,
8         task.baseline_estimator()),
9 ]
10 ml.print_table([ml.table_row(task, r) for r in results_int])

```

Out

model	d	accuracy	precision	recall	f1_macro	roc_auc	fit_s	predict_s
AdaBoost (200 vòng, cây sâu 3)	3,452	0.3859	0.3169	0.3578	0.3190	-	8.0	0.2
LogReg bảng + TF-IDF (mốc)	3,452	0.3672	0.3097	0.3160	0.2959	-	1.7	0.0

In [15]

```
1 test_int = {r["name"]: ml.evaluate_on_test(task, r["pipe"]) for r in results_int}
2 for name, m in test_int.items():
3     print(f"TEST {name:<36} accuracy={m['accuracy']:.4f} f1_macro={m['f1_macro']:.4f} "
4           f"f1_weighted={m['f1_weighted']:.4f}")
```

Out

```
TEST AdaBoost (200 vòng, cây sâu 3) accuracy=0.4050 f1_macro=0.3988 f1_weighted=0.4227
TEST LogReg bảng + TF-IDF (mốc) accuracy=0.4029 f1_macro=0.3368 f1_weighted=0.4305
```

Nhận xét. Đây là bài toán duy nhất trong cả bốn bài toán mà mô hình bổ sung **vượt mốc**: F1-macro trên test đạt 0.3988 so với 0.3368 của hồi quy logistic, trong khi accuracy gần như bằng nhau.

Lý do không nằm ở chỗ AdaBoost mạnh hơn, mà ở chỗ bài toán này quá khó nên mọi mô hình đều quanh quẩn mức đoán mò. Khi không có mô hình nào thật sự học được quy luật, thứ hạng bị quyết định bởi chuyện mô hình phân bổ dự đoán ra các lớp như thế nào. Cơ chế tăng trọng số mẫu sai của AdaBoost khiến nó buộc phải đụng tới cả những lớp hiếm, và F1-macro thưởng cho đúng hành vi đó. Không nên đọc con số này thành "AdaBoost là mô hình tốt cho bài toán A"; cách đọc đúng là chênh lệch 0.06 F1-macro giữa hai mô hình cùng yếu thì không đủ để kết luận điều gì về thuật toán.

Giới hạn thật sự vẫn là giới hạn mà notebook 03 đã chỉ ra: bình luận khách hàng nói về chất lượng dịch vụ chứ không nói về sản phẩm, nên không biểu diễn nào rút ra được tín hiệu không có trong dữ liệu. """),

md(r""")

8. Tổng hợp và ghi kết quả

8. Tổng hợp và ghi kết quả

In [16]

```
1 def block(task, results, test_metrics, sweep_rows, best_stage, weak_learner):
2     return {
3         "task": task.key,
4         "title": task.title,
5         "kind": task.kind,
6         "n_train": task.n_train,
7         "n_val": task.n_val,
8         "n_test": task.n_test,
9         "audit": task.audit,
10        "weak_learner": weak_learner,
11        "stage_grid": [int(r["vòng"]) for r in sweep_rows],
12        "stage_selected": best_stage,
13        "sweep": [{k: v for k, v in r.items() if k != "_score"} for r in sweep_rows],
14        "validation_table": [ml.table_row(task, r) for r in results],
15        "test_metrics": dict(test_metrics),
16        "model_names": [r["name"] for r in results],
17    }
18
19
20 payload = {
21     "algorithm": "AdaBoost (SAMME cho phân loại, AdaBoost.R2 cho hồi quy)",
22     "library": "scikit-learn",
23     "seed": ml.RANDOM_SEED,
24     "learning_rate": 0.5,
25     "note": (
26         "Mô hình mốc được huấn luyện lại trên bộ dữ liệu hiện tại trong data/, "
27         "không chép lại số của notebook 01-03."
28     ),
29     "tasks": [
30         block(TASKS["diabetes"], results_dia, test_dia, rows_dia, m_dia,
31             "DecisionTreeClassifier(max_depth=3) + sample_weight cân bằng lớp"),
32         block(TASKS["house_price"], results_house, test_house, rows_house, m_house,
```

```

33         "DecisionTreeRegressor(max_depth=8)"),
34     block(TASKS["ecommerce_satisfaction"], results_sat, test_sat, rows_sat, m_sat,
35           "DecisionTreeClassifier(max_depth=3) + sample_weight cân bằng lớp"),
36     block(TASKS["ecommerce_interest"], results_int, test_int, rows_int, m_int,
37           "DecisionTreeClassifier(max_depth=3) + sample_weight cân bằng lớp"),
38     ],
39 }
40
41 path = ml.save_results("adaboost", payload)
42 print("Đã ghi", path.relative_to(ROOT), f"({path.stat().st_size / 1024:.1f} KB)")

```

Out

```

Đã ghi reports\adaboost_results.json (12.1 KB)

```

9. Kết luận

Sau notebook 05 và 06, mỗi category có đủ ba họ thuật toán ML để so sánh, và ba họ đó trả lời ba câu hỏi khác nhau chứ không phải ba lần thử cùng một câu hỏi.

- **Mô hình mốc** trả lời: chất lượng tốt nhất đạt được là bao nhiêu.
- **KNN** trả lời: chất lượng đó đến từ việc học một quy luật, hay chỉ từ việc dữ liệu đủ dày để tra cứu hàng xóm là xong.
- **AdaBoost** trả lời: trong nhóm boosting, phần nào của thiết kế mang lại kết quả.

Kết quả thực tế của AdaBoost trên bốn bài toán:

Bài toán	AdaBoost (test)	Mốc (test)	Kết luận	--- --- ---	Tiểu đường	F1-macro	0.6438	0.6515	bám sát, thua không đáng kể
Giá nhà	R2(log)	0.6355	0.7095	thua rõ	E-com hài lòng	F1-macro	0.7452	0.7853	thua rõ
E-com nhóm sản phẩm	F1-macro	0.3988	0.3368	vượt mốc, nhưng cả hai đều yếu					

Ba nguyên nhân giải thích trọn bảng trên. AdaBoost cạnh tranh sòng phẳng ở bài toán bám sát vì nó cùng là tổ hợp cây với mốc. Nó thua ở hồi quy vì AdaBoost.R2 không dùng dấu của sai số nên khớp quá ngay sau vòng 25. Nó thua ở văn bản vì mỗi cây sâu 3 tầng chỉ chạm được 7 chiều trong hơn 11,000 chiều TF-IDF, trong khi hồi quy logistic gán trọng số cho toàn bộ từ vựng trong một lần tối ưu.

Trường hợp thứ tư đáng chú ý theo hướng ngược lại: thắng trên một bài toán mà mọi mô hình đều quanh mức đoán mò thì không phải bằng chứng về chất lượng thuật toán.

Mô hình được triển khai lên API **không thay đổi** sau hai notebook này.

Phụ lục D. Notebook 07: Mạng nơ-ron hồi quy cho giá nhà

Toàn bộ nội dung notebook, giữ nguyên thứ tự cell. Mọi con số trong chương tương ứng của báo cáo đều sinh từ tệp kết quả mà notebook này ghi ra, nên báo cáo không thể lệch với notebook.

07_deep_learning_house.ipynb

23 cell · 12 code

Notebook 07 - Mạng nơ-ron hồi quy viết tay cho bài toán giá nhà

Notebook 04 đã dựng mạng nơ-ron bằng NumPy cho bài toán **phân loại nhị phân** (tiểu đường). Notebook này đưa cùng bộ công cụ đó sang bài toán **hồi quy**, để category giá nhà có mô hình deep learning riêng thay vì đi mượn kết luận của category khác.

Toàn bộ mạng vẫn viết tay: lan truyền xuôi, hàm mất mát, lan truyền ngược theo quy tắc chuỗi và gradient descent đều là NumPy thuần. Không dùng TensorFlow, PyTorch, Keras, và cũng không dùng `MLPRegressor` của `scikit-learn`. `scikit-learn` chỉ xuất hiện ở hai chỗ: tiền xử lý dữ liệu (dùng chung với notebook 05 và 06 qua `ml/datasets.py`) và mô hình mốc để so sánh.

Hai thứ phải đổi khi chuyển từ phân loại sang hồi quy, và chỉ hai thứ đó:

| | Notebook 04 (phân loại) | Notebook 07 (hồi quy) | ---|---| | Hàm kích hoạt lớp cuối | sigmoid, ép về khoảng $(0, 1)$ | **đồng nhất**, cho ra số thực bất kỳ | | Hàm mất mát | binary cross entropy | **sai số bình phương trung bình** | | Công thức lan truyền ngược | $\Delta = (\hat{y} - y) / n$ | **giống hệt** |

Dòng cuối của bảng không phải trùng hợp, và mục 3 sẽ chứng minh nó.

In [1]

```
1 import sys
2 import time
3 import warnings
4 from dataclasses import dataclass
5 from pathlib import Path
6
7 import matplotlib
8 import matplotlib.pyplot as plt
9 import numpy as np
10 import pandas as pd
11
12 ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
13 if str(ROOT) not in sys.path:
14     sys.path.insert(0, str(ROOT))
15
16 import ml
17
18 warnings.filterwarnings("ignore")
19 matplotlib.rcParams.update({
20     "font.family": "serif",
21     "font.size": 9,
22     "axes.edgecolor": "black",
23     "axes.labelcolor": "black",
24     "text.color": "black",
25     "xtick.color": "black",
26     "ytick.color": "black",
27     "figure.facecolor": "white",
28     "savefig.facecolor": "white",
29 })
30
31 print("thư mục gốc :", ROOT)
32 print("numpy      :", np.__version__)
```

Out

```
thư mục gốc : C:\Users\Hung Nguyen Ba\assignment03
numpy      : 2.4.6
```

1. Dữ liệu và biểu diễn đầu vào

Mạng nơ-ron chỉ nhận ma trận số, nên `city` (12,183 giá trị chữ) và `state` phải được mã hoá trước. Dừng lại đúng biểu diễn `target_enc_scaled` mà KNN đã dùng ở notebook 05: tám chiều, đã đưa về cùng thang đo.

Chuẩn hoá ở đây không phải tùy chọn. Trọng số được khởi tạo quanh 0 với phương sai tỉ lệ $\frac{2}{\text{fan_in}}$; nếu một chiều đầu vào có độ lớn hàng nghìn thì tích $\$xW\$$ ở lớp đầu tiên sẽ bão hoà hoặc làm gradient nổ ngay từ vài bước đầu.

In [2]

```
1 task = ml.load_house_price()
2 prep = task.prep("target_enc_scaled")
3
4 X_train = prep.fit_transform(task.X_train, task.y_train)
5 X_val = prep.transform(task.X_val)
6 X_test = prep.transform(task.X_test)
7
8 print(f"X_train {X_train.shape} X_val {X_val.shape} X_test {X_test.shape}")
9 print(f"trung bình mỗi chiều sau chuẩn hoá: {np.round(X_train.mean(axis=0), 3)}")
10 print(f"độ lệch chuẩn mỗi chiều : {np.round(X_train.std(axis=0), 3)}")
```

Out

```
X_train (175000, 8) X_val (37500, 8) X_test (37500, 8)
trung bình mỗi chiều sau chuẩn hoá: [-0.  0. -0.  0.  0.  0. -0.  0.]
độ lệch chuẩn mỗi chiều : [1.  1.  1.  1.  1.  1.  1.  1.]
```

Chuẩn hoá cả biến mục tiêu

Target là $\log(\text{price})$, dao động quanh 12.5. Nếu để nguyên, lớp cuối phải học một bias bằng 12.5 trong khi trọng số khởi tạo quanh 0, và những bước gradient đầu tiên chỉ dùng để bỏ dần tới con số đó thay vì học quan hệ giữa các đặc trưng.

Vì vậy mạng học trên $z = (y - \mu) / \sigma$ với μ, σ lấy từ tập **train**, rồi mọi chỉ số báo cáo đều được quy ngược về USD. Lấy thống kê từ tập train là bắt buộc: dùng thống kê của toàn bộ dữ liệu là rò rỉ thông tin từ tập test sang quá trình huấn luyện.

In [3]

```
1 y_mean = float(task.y_train.mean())
2 y_std = float(task.y_train.std())
3
4 z_train = ((task.y_train - y_mean) / y_std).reshape(-1, 1)
5 z_val = ((task.y_val - y_mean) / y_std).reshape(-1, 1)
6
7 def to_log_price(z):
8     '''Quy dự đoán đã chuẩn hoá về lại log(price) để tính chỉ số nghiệp vụ.'''
9     return z.ravel() * y_std + y_mean
10
11 print(f"log(price): trung bình = {y_mean:.4f}, độ lệch chuẩn = {y_std:.4f}")
12 print(f"sau chuẩn hoá: trung bình = {z_train.mean():.6f}, độ lệch chuẩn = {z_train.std():.6f}")
```

Out

```
log(price): trung bình = 12.7957, độ lệch chuẩn = 0.6945
sau chuẩn hoá: trung bình = 0.000000, độ lệch chuẩn = 1.000000
```

2. Mạng nơ-ron viết tay

Một lớp ẩn làm đúng hai việc: nhân ma trận rồi cộng bias để ra z , sau đó bẻ cong phi tuyến bằng ReLU. Lớp cuối chỉ làm việc thứ nhất, không bẻ cong, vì đầu ra cần là một số thực bất kỳ.

$$\hat{h}^{(l)} = \text{ReLU}(\hat{h}^{(l-1)} W^{(l)} + b^{(l)}), \quad \hat{z} = \hat{h}^{(L-1)} W^{(L)} + b^{(L)}$$

Trọng số khởi tạo theo He: $W \sim \mathcal{N}(0, \frac{2}{\text{fan_in}})$. Chia cho fan-in giữ cho phương sai của tín hiệu không phình ra hay teo lại khi đi qua từng lớp, và hệ số 2 bù cho việc ReLU cắt bỏ một nửa miền giá trị.

In [4]

```
1 def relu(x):
2     return np.maximum(0.0, x)
```

```

3
4
5     def relu_derivative(x):
6         return (x > 0).astype(float)
7
8
9     def identity(x):
10        return x
11
12
13    def identity_derivative(x):
14        return np.ones_like(x)
15
16
17    ACTIVATIONS = {"relu": (relu, relu_derivative), "linear": (identity, identity_derivative)}
18
19
20    @dataclass(frozen=True)
21    class Architecture:
22        key: str
23        label: str
24        sizes: tuple
25        hidden_activation: str = "relu"
26
27        @property
28        def shape(self):
29            return " - ".join(str(s) for s in self.sizes)
30
31        @property
32        def n_params(self):
33            return sum(a * b + b for a, b in zip(self.sizes, self.sizes[1:]))
34
35
36    def init_params(arch, seed):
37        rng = np.random.default_rng(seed)
38        layers = []
39        for fan_in, fan_out in zip(arch.sizes, arch.sizes[1:]):
40            W = rng.standard_normal((fan_in, fan_out)) * np.sqrt(2.0 / fan_in)
41            layers.append((W, np.zeros((1, fan_out))))
42        return tuple(layers)
43
44
45    def forward(params, X, hidden_activation):
46        '''Lớp cuối KHÔNG có hàm kích hoạt: đầu ra hồi quy là số thực bất kỳ.'''
47        act, _ = ACTIVATIONS[hidden_activation]
48        pre_activations, activations = [], [X]
49        h = X
50        last = len(params) - 1
51        for index, (W, b) in enumerate(params):
52            z = h @ W + b
53            h = z if index == last else act(z)
54            pre_activations.append(z)
55            activations.append(h)
56        return h, {"z": pre_activations, "h": activations}

```

3. Hàm mất mát và lan truyền ngược

Hàm mất mát là sai số bình phương trung bình, viết kèm hệ số $\frac{1}{2}$ cho gọn đạo hàm:

$$L = \frac{1}{2n} \sum_{i=1}^n (\hat{z}_i - z_i)^2$$

Đạo hàm của L theo đầu ra **trước** hàm kích hoạt của lớp cuối:

$$\frac{\partial L}{\partial z^{(L)}} = \frac{\partial L}{\partial \hat{z}^{(L)}} \cdot \frac{\partial \hat{z}^{(L)}}{\partial z^{(L)}} = \frac{\hat{z}^{(L)} - z^{(L)}}{n} \cdot 1 = \frac{\hat{z}^{(L)} - z^{(L)}}{n}$$

Đây chính là điều đã hứa ở bảng đầu notebook. Ở notebook 04, cặp sigmoid cộng binary cross entropy cũng cho ra đúng $(y - \hat{y})/n$, vì đạo hàm $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ triệt tiêu gọn với mẫu số của cross entropy. Hai bài toán khác nhau, hai hàm mất mát khác nhau, nhưng **hàm backward không phải sửa một dòng**

nào. Đó không phải may mắn: đây là tính chất của các cặp hàm liên kết chuẩn (canonical link), lý do khiến người ta luôn ghép sigmoid với cross entropy và ghép đầu ra đồng nhất với sai số bình phương.

Từ đó lùi ngược qua từng lớp theo quy tắc chuỗi:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}} \frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{W}^{(l)}} = \mathbf{h}^{(l-1)} \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}} \frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}}$$

In [5]

```
1 def mean_squared_error(z_true, z_pred):
2     return float(0.5 * np.mean((z_pred - z_true) ** 2))
3
4
5 def backward(params, z_true, cache, hidden_activation):
6     '''Giống nguyên văn notebook 04. Xem lập luận ở ô markdown ngay phía trên.'''
7     _, act_derivative = ACTIVATIONS[hidden_activation]
8     z, h = cache["z"], cache["h"]
9     n = len(z_true)
10
11     grads = []
12     delta = (h[-1] - z_true) / n
13     for index in range(len(params) - 1, -1, -1):
14         grads.append((h[index].T @ delta, np.sum(delta, axis=0, keepdims=True)))
15         if index:
16             delta = (delta @ params[index][0].T) * act_derivative(z[index - 1])
17     return tuple(reversed(grads))
18
19
20 def update(params, grads, learning_rate):
21     return tuple(
22         (W - learning_rate * dW, b - learning_rate * db)
23         for (W, b), (dW, db) in zip(params, grads))
24
```

4. Huấn luyện bốn kiến trúc

Bốn kiến trúc dùng chung một seed khởi tạo, một seed xáo trộn, một learning rate và một số epoch, nên mọi chênh lệch giữa chúng đều quy được về kiến trúc.

- **H1 Baseline** 8 - 32 - 16 - 1: hai lớp ẩn, làm mốc.
- **H2 Wider** 8 - 64 - 32 - 1: nhân đôi bề rộng, kiểm tra xem thêm sức chứa có đổi được chất lượng không.
- **H3 Shallow** 8 - 32 - 1: bỏ lớp ẩn thứ hai, kiểm tra xem chiều sâu có cần thiết không.
- **H4 No ReLU** 8 - 32 - 16 - 1 với kích hoạt đồng nhất: ba phép biến đổi tuyến tính xếp chồng vẫn chỉ là một phép tuyến tính, nên mạng này suy biến thành hồi quy tuyến tính dù vẫn có đủ số tham số. Đây là phép thử đối chứng cho câu hỏi phi tuyến có đáng giá không.

In [6]

```
1 LEARNING_RATE = 0.05
2 EPOCHS = 40
3 BATCH_SIZE = 512
4 INIT_SEED = 42
5 SHUFFLE_SEED = 7
6
7 ARCHITECTURES = (
8     Architecture("H1", "Baseline", (8, 32, 16, 1), "relu"),
9     Architecture("H2", "Wider", (8, 64, 32, 1), "relu"),
10    Architecture("H3", "Shallow", (8, 32, 1), "relu"),
11    Architecture("H4", "No ReLU", (8, 32, 16, 1), "linear"),
12 )
13
14
15 def fit(arch):
16     params = init_params(arch, INIT_SEED)
17     rng = np.random.default_rng(SHUFFLE_SEED)
18     n = len(X_train)
```

```

19     history_train, history_val = [], []
20     started = time.perf_counter()
21
22     for _ in range(EPOCHS):
23         order = rng.permutation(n)
24         for begin in range(0, n, BATCH_SIZE):
25             rows = order[begin : begin + BATCH_SIZE]
26             _, cache = forward(params, X_train[rows], arch.hidden_activation)
27             grads = backward(params, z_train[rows], cache, arch.hidden_activation)
28             params = update(params, grads, LEARNING_RATE)
29             history_train.append(mean_squared_error(z_train, forward(params, X_train, arch.hidden_activation)
[0]))
30             history_val.append(mean_squared_error(z_val, forward(params, X_val, arch.hidden_activation)[0]))
31
32     seconds = time.perf_counter() - started
33     val_log = to_log_price(forward(params, X_val, arch.hidden_activation)[0])
34     metrics = ml.regression_metrics(task.y_val, val_log)
35     return {
36         "arch": arch, "params": params, "seconds": seconds,
37         "history_train": history_train, "history_val": history_val,
38         "metrics": metrics,
39     }
40
41
42     trained = {}
43     for arch in ARCHITECTURES:
44         trained[arch.key] = fit(arch)
45         m = trained[arch.key]["metrics"]
46         print(f"[{trained[arch.key]['seconds']:.6f}s] {arch.key} {arch.label:<9} {arch.shape:<14} "
47               f"{arch.n_params:>5d} tham số R2_log={m['r2_log']:.4f} MAE={m['mae_usd']:.0f} USD")

```

Out

```
[ 48.5s] H1 Baseline  8 - 32 - 16 - 1  833 tham số R2_log=0.6209 MAE=137,980 USD
```

Out

```
[ 77.4s] H2 Wider    8 - 64 - 32 - 1 2689 tham số R2_log=0.6238 MAE=137,822 USD
```

Out

```
[ 34.5s] H3 Shallow  8 - 32 - 1      321 tham số R2_log=0.6168 MAE=139,337 USD
```

Out

```
[ 43.2s] H4 No ReLU  8 - 32 - 16 - 1  833 tham số R2_log=0.5806 MAE=148,536 USD
```

In [7]

```

1     pd.DataFrame([
2         {
3             "mô hình": f"{r['arch'].key} {r['arch'].label}",
4             "kiến trúc": r["arch"].shape,
5             "kích hoạt": r["arch"].hidden_activation,
6             "tham số": r["arch"].n_params,
7             "loss cuối (train)": f"{r['history_train'][-1]:.5f}",
8             "loss cuối (val)": f"{r['history_val'][-1]:.5f}",
9             "R2_log": f"{r['metrics']['r2_log']:.4f}",
10            "MAE_USD": f"{r['metrics']['mae_usd']:.0f}",
11            "MAPE_%": f"{r['metrics']['mape_percent']:.1f}",
12            "giây": f"{r['seconds']:.1f}",
13        }
14        for r in trained.values()
15    ])

```

Out

	mô hình	kiến trúc	kích hoạt	tham số	loss cuối (train)	\
0	H1 Baseline	8 - 32 - 16 - 1	relu	833	0.19255	
1	H2 Wider	8 - 64 - 32 - 1	relu	2689	0.19076	
2	H3 Shallow	8 - 32 - 1	relu	321	0.19511	
3	H4 No ReLU	8 - 32 - 16 - 1	linear	833	0.21434	

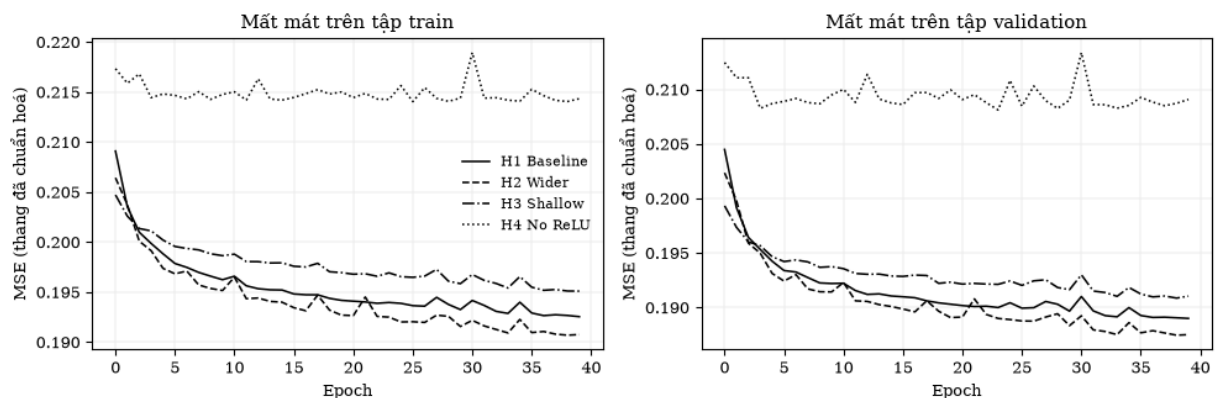
	loss cuối (val)	R2_log	MAE_USD	MAPE_%	giây
0	0.18901	0.6209	137,980	34.9	48.5
1	0.18752	0.6238	137,822	34.5	77.4
2	0.19105	0.6168	139,337	35.2	34.5
3	0.20909	0.5806	148,536	37.5	43.2

In [8]

```

1  fig, axes = plt.subplots(1, 2, figsize=(10.0, 3.4))
2  styles = ["-", "--", "-.", ":"]
3
4  for (key, result), style in zip(trained.items(), styles):
5      label = f"{key} {result['arch'].label}"
6      axes[0].plot(result["history_train"], style, color="black", linewidth=1.1, label=label)
7      axes[1].plot(result["history_val"], style, color="black", linewidth=1.1, label=label)
8
9  axes[0].set_title("Mất mát trên tập train")
10 axes[1].set_title("Mất mát trên tập validation")
11 for axis in axes:
12     axis.set_xlabel("Epoch")
13     axis.set_ylabel("MSE (thang đã chuẩn hoá)")
14     axis.grid(True, linewidth=0.3, alpha=0.4)
15 axes[0].legend(frameon=False, fontsize=8)
16 fig.tight_layout()
17 plt.show()

```



Đọc hai biểu đồ. Ba kiến trúc có ReLU hội tụ về cùng một vùng mất mát và gần như chồng lên nhau, trong khi H4 đứng hẳn ở một mức cao hơn và không nhúc nhích thêm. H4 là bằng chứng trực tiếp cho lập luận đã nêu ở mục 4: bỏ phi tuyến đi thì ba lớp cũng chỉ là một phép nhân ma trận, và một phép tuyến tính trên 8 chiều thì không diễn tả nổi quan hệ giữa diện tích, số phòng, thành phố và giá.

Đường train và đường validation bám sát nhau ở cả bốn kiến trúc, tức chưa có dấu hiệu khớp quá. Với 175,000 dòng train và mạng lớn nhất chỉ có vài nghìn tham số thì đó là chuyện bình thường.

5. So sánh với mô hình machine learning tốt nhất

Câu hỏi thật sự không phải kiến trúc nào tốt nhất trong bốn kiến trúc trên, mà là mạng nơ-ron viết tay có đáng để thay thế HistGradientBoosting hay không. Mốc so sánh được huấn luyện lại ngay tại đây trên đúng tập train, val, test vừa dùng.

In [9]

```

1  best_key = max(trained, key=lambda k: trained[k]["metrics"]["r2_log"])
2  best = trained[best_key]
3  best_arch = best["arch"]
4  print(f"Kiến trúc tốt nhất trên validation: {best_key} {best_arch.label} ({best_arch.shape})")
5
6  baseline = ml.run_model(task, task.baseline_name + " (môc ML)", task.baseline_rep,
7                          task.baseline_estimator())
8  ml.print_table([ml.table_row(task, baseline)])

```

Out

Kiến trúc tốt nhất trên validation: H2 Wider (8 - 64 - 32 - 1)

Out

model	d	R2_log	MAE_USD	RMSE_USD	R2_USD	MAPE_%	fit_s	predict_s
HistGradientBoosting (môc ML)	8	0.7089	120,546	198,242	0.6507	29.8	22.4	1.1

In [10]

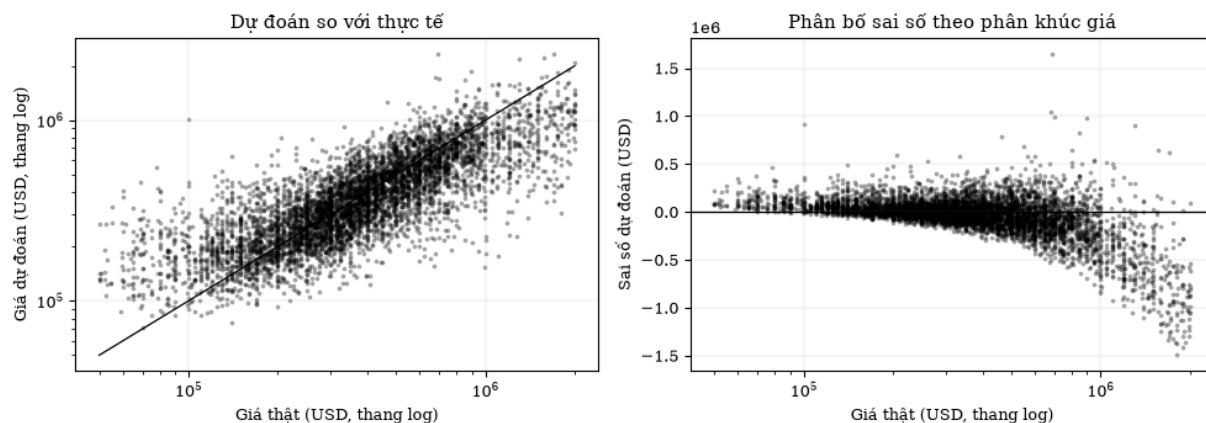
```
1 test_log = to_log_price(forward(best["params"], X_test, best_arch.hidden_activation)[0])
2 dl_test = ml.regression_metrics(task.y_test, test_log)
3 ml_test = ml.evaluate_on_test(task, baseline["pipe"])
4
5 pd.DataFrame([
6     {"mô hình": f"Mạng nơ-ron {best_key} ({best_arch.shape})", **{
7         "R2_log": f"{dl_test['r2_log']:.4f}", "MAE_USD": f"{dl_test['mae_usd']:.0f}",
8         "RMSE_USD": f"{dl_test['rmse_usd']:.0f}", "MAPE_%": f"{dl_test['mape_percent']:.1f}"},
9     {"mô hình": task.baseline_name + " (môc ML)", **{
10         "R2_log": f"{ml_test['r2_log']:.4f}", "MAE_USD": f"{ml_test['mae_usd']:.0f}",
11         "RMSE_USD": f"{ml_test['rmse_usd']:.0f}", "MAPE_%": f"{ml_test['mape_percent']:.1f}"},
12 ])
```

Out

	mô hình	R2_log	MAE_USD	RMSE_USD	MAPE_%
0	Mạng nơ-ron H2 (8 - 64 - 32 - 1)	0.6265	137,828	222,488	34.5
1	HistGradientBoosting (môc ML)	0.7095	121,023	198,788	30.0

In [11]

```
1 actual_usd = np.exp(task.y_test)
2 predicted_usd = np.exp(test_log)
3 residual = predicted_usd - actual_usd
4
5 fig, axes = plt.subplots(1, 2, figsize=(10.0, 3.6))
6 sample = np.random.default_rng(INIT_SEED).choice(len(actual_usd), size=6000, replace=False)
7
8 axes[0].scatter(actual_usd[sample], predicted_usd[sample], s=3, alpha=0.25, color="black")
9 limits = [actual_usd.min(), actual_usd.max()]
10 axes[0].plot(limits, limits, "--", color="black", linewidth=0.9)
11 axes[0].set_xscale("log")
12 axes[0].set_yscale("log")
13 axes[0].set_xlabel("Giá thật (USD, thang log)")
14 axes[0].set_ylabel("Giá dự đoán (USD, thang log)")
15 axes[0].set_title("Dự đoán so với thực tế")
16
17 axes[1].scatter(actual_usd[sample], residual[sample], s=3, alpha=0.25, color="black")
18 axes[1].axhline(0, color="black", linewidth=0.9)
19 axes[1].set_xscale("log")
20 axes[1].set_xlabel("Giá thật (USD, thang log)")
21 axes[1].set_ylabel("Sai số dự đoán (USD)")
22 axes[1].set_title("Phân bố sai số theo phân khúc giá")
23
24 for axis in axes:
25     axis.grid(True, linewidth=0.3, alpha=0.4)
26 fig.tight_layout()
27 plt.show()
```



Đọc biểu đồ sai số. Đám mây điểm bám quanh đường chéo ở phân khúc giữa và loe rộng dần về hai đầu. Ở phía giá cao, sai số lệch xuống dưới đường 0 một cách có hệ thống: mạng đoán thấp hơn giá thật. Đây đúng là khuôn mẫu lỗi mà notebook 02 đã ghi nhận với mô hình cây, và nó có cùng một nguyên nhân, không liên quan tới thuật toán: tám đặc trưng đang dùng không chứa thông tin phân biệt một căn 3 tỉ với một căn 800 nghìn trong cùng thành phố, cùng diện tích. Thông tin đó nằm ở vị trí cụ thể, tình trạng nội thất, tầm nhìn, những thứ không có trong dữ liệu.

6. Ghi kết quả

In [12]

```
1     payload = {
2         "task": task.key,
3         "title": task.title,
4         "representation": "target_enc_scaled",
5         "d_input": int(X_train.shape[1]),
6         "n_train": task.n_train,
7         "n_val": task.n_val,
8         "n_test": task.n_test,
9         "audit": task.audit,
10        "target_transform": {"log_price_mean": y_mean, "log_price_std": y_std},
11        "learning_rate": LEARNING_RATE,
12        "epochs": EPOCHS,
13        "batch_size": BATCH_SIZE,
14        "init_seed": INIT_SEED,
15        "shuffle_seed": SHUFFLE_SEED,
16        "models": [
17            {
18                "key": r["arch"].key,
19                "label": r["arch"].label,
20                "shape": r["arch"].shape,
21                "hidden_activation": r["arch"].hidden_activation,
22                "n_params": r["arch"].n_params,
23                "seconds": round(r["seconds"], 2),
24                "loss_train_final": round(r["history_train"][-1], 6),
25                "loss_val_final": round(r["history_val"][-1], 6),
26                "loss_history_train": [round(v, 6) for v in r["history_train"]],
27                "loss_history_val": [round(v, 6) for v in r["history_val"]],
28                "validation": {k: round(v, 6) for k, v in r["metrics"].items()},
29            }
30            for r in trained.values()
31        ],
32        "best_model": best_key,
33        "test_metrics_dl": {k: round(v, 6) for k, v in dl_test.items()},
34        "test_metrics_ml_baseline": {k: round(v, 6) for k, v in ml_test.items()},
35        "ml_baseline_name": task.baseline_name,
36    }
37
38    path = ml.save_results("deeplearning_house", payload)
```

```
39 print("Đã ghi", path.relative_to(ROOT), f"({path.stat().st_size / 1024:.1f} KB)")
```

Out

```
Đã ghi reports\deeplearning_house_results.json (9.4 KB)
```

7. Kết luận

Category giá nhà giờ đã có mô hình deep learning riêng, viết tay hoàn toàn bằng NumPy, đo trên đúng tập test mà ba mô hình ML đang dùng.

Ba điều rút ra được:

1. **Phi tuyến là thứ tạo ra giá trị, không phải số tham số.** H4 có đúng số tham số như H1 nhưng kém hẳn, vì thiếu ReLU. Trong khi đó H2 nhân đôi bề rộng mà gần như không cải thiện được gì so với H1.
2. **Hàm backward không đổi khi chuyển bài toán.** Cùng một đoạn mã lan truyền ngược chạy được cho cả phân loại của notebook 04 lẫn hồi quy ở đây, nhờ tính chất của cặp hàm liên kết chuẩn. Đây là lý do các thư viện deep learning có thể tách rời phần mạng khỏi phần hàm mất mát.
3. **Dữ liệu bảng vẫn là sân của cây quyết định.** Mạng nơ-ron không vượt được HistGradientBoosting trên tám chiều đặc trưng bảng. Kết quả này khớp với kết luận phổ biến trong ngành và không có gì bất ngờ: deep learning chiếm ưu thế khi đầu vào có cấu trúc mà con người khó đặc trưng hoá bằng tay (ảnh, âm thanh, văn bản), chứ không phải khi đầu vào đã là một bảng số sạch với 8 cột.

Điểm thứ ba dẫn thẳng sang notebook 08: ở đó đầu vào có văn bản, tức là đúng loại dữ liệu mà deep learning sinh ra để xử lý.

Phụ lục E. Notebook 08: Mạng nơ-ron có lớp nhúng cho e-commerce

Toàn bộ nội dung notebook, giữ nguyên thứ tự cell. Mọi con số trong chương tương ứng của báo cáo đều sinh từ tập kết quả mà notebook này ghi ra, nên báo cáo không thể lệch với notebook.

08_deep_learning_ecommerce.ipynb

28 cell · 14 code

Notebook 08 - Mạng nơ-ron có lớp nhúng học được cho bài toán e-commerce

Notebook 04 làm phân loại trên bảng số, notebook 07 làm hồi quy trên bảng số. Notebook này xử lý loại dữ liệu mà deep learning thật sự sinh ra để giải: **văn bản**.

Notebook 03 đã dựng tensor văn bản $E \in \mathbb{R}^{B \times T \times d}$ bằng một ma trận nhúng **ngẫu nhiên cố định**, rồi lấy trung bình theo trục thời gian để đưa vào HistGradientBoosting. Ma trận nhúng đó không bao giờ được cập nhật, nên nó chỉ là một phép chiếu ngẫu nhiên. Notebook này đi nốt phần còn lại: **ma trận nhúng trở thành tham số của mạng và được học bằng lan truyền ngược**, giống hệt mọi trọng số khác.

Bốn mô hình được huấn luyện trong cùng một điều kiện để tách bạch phần đóng góp của từng thành phần:

| Mô hình | Đầu vào | Ma trận nhúng | N1 | chỉ bảng | không dùng | N2 | bảng + văn bản | **học được** | N3 | bảng + văn bản | ngẫu nhiên, đóng băng | N4 | chỉ văn bản | học được |

So N2 với N1 cho biết văn bản đóng góp bao nhiêu. So N2 với N3 cho biết trong phần đóng góp đó, bao nhiêu đến từ việc **học** ma trận nhúng chứ không phải từ bản thân việc có văn bản. Đây chính là câu hỏi mà notebook 03 để ngỏ.

In [1]

```
1 import sys
2 import time
3 import warnings
4 from collections import Counter
5 from dataclasses import dataclass
6 from pathlib import Path
7
8 import matplotlib
9 import matplotlib.pyplot as plt
10 import numpy as np
11 import pandas as pd
12
13 ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
14 if str(ROOT) not in sys.path:
15     sys.path.insert(0, str(ROOT))
16
17 import ml
18
19 warnings.filterwarnings("ignore")
20 matplotlib.rcParams.update({
21     "font.family": "serif",
22     "font.size": 9,
23     "axes.edgecolor": "black",
24     "axes.labelcolor": "black",
25     "text.color": "black",
26     "xtick.color": "black",
27     "ytick.color": "black",
28     "figure.facecolor": "white",
29     "savefig.facecolor": "white",
30 })
31
32 print("thư mục gốc :", ROOT)
33 print("numpy      :", np.__version__)
```

Out

```
thư mục gốc : C:\Users\Hung Nguyen Ba\assignment03
numpy      : 2.4.6
```

1. Dữ liệu

Bài toán: dự đoán khách có hài lòng hay không (CSAT từ 4 điểm trở lên) từ hồ sơ ticket cộng bình luận của khách. Chỉ dùng tập con có bình luận, đúng như notebook 03, vì bài toán đặt ra là đo giá trị của văn bản.

In [2]

```
1 task = ml.load_ecommerce_satisfaction()
2 prep_tab = task.prep("tabular")
3
4 TAB_TRAIN = np.asarray(prepare_fit_transform(task.X_train, task.y_train).toarray())
5 TAB_VAL = np.asarray(prepare_transform(task.X_val).toarray())
6 TAB_TEST = np.asarray(prepare_transform(task.X_test).toarray())
7
8 y_train = np.asarray(task.y_train).reshape(-1, 1).astype(float)
9 y_val = np.asarray(task.y_val).reshape(-1, 1).astype(float)
10 y_test = np.asarray(task.y_test).reshape(-1, 1).astype(float)
11
12 print(f"bảng: train {TAB_TRAIN.shape} val {TAB_VAL.shape} test {TAB_TEST.shape}")
13 print(f"tỉ lệ hài lòng: train={y_train.mean():.4f} val={y_val.mean():.4f} test={y_test.mean():.4f}")
```

Out

```
bảng: train (22129, 81) val (4742, 81) test (4742, 81)
tỉ lệ hài lòng: train=0.7760 val=0.7760 test=0.7760
```

2. Văn bản thành số: tokenize, từ điển, chuỗi id

Ba bước, mỗi bước mất một phần thông tin và cần nói rõ mất cái gì.

1. **Tokenize** cắt câu thành từ theo biểu thức chính quy [a-z0-9']+ sau khi hạ chữ thường. Mất dấu câu và phân biệt hoa thường. 2. **Từ điển** giữ 5,000 từ xuất hiện nhiều nhất **trong tập train**. Mọi từ ngoài danh sách đó bị bỏ. Xây từ điển chỉ từ tập train là bắt buộc: dùng cả tập test để đếm tần suất là rõ ràng thông tin. 3. **Cắt và đệm** mọi chuỗi về đúng $T = 32$ token. Chuỗi dài hơn bị cắt đuôi, ngắn hơn được đệm bằng id 0. Id 0 được dành riêng cho token đệm và sẽ bị che đi khi tính trung bình, để một bình luận 5 từ không bị chia cho 32.

In [3]

```
1 VOCAB_SIZE = 5000
2 T_PAD = 32
3 D_EMBED = 32
4
5 counter = Counter()
6 for text in task.X_train[ml.datasets.ECOM_TEXT]:
7     counter.update(ml.tokenize(str(text)))
8
9 vocab = {word: index for index, (word, _) in enumerate(counter.most_common(VOCAB_SIZE), start=1)}
10 print(f"Token khác nhau trong tập train : {len(counter):,}")
11 print(f"Gửi lại trong từ điển : {len(vocab):,} (id 1..{len(vocab)}, id 0 = PAD)")
12 print(f"10 từ phổ biến nhất : ", [w for w, _ in counter.most_common(10)])
13
14
15 def texts_to_ids(texts):
16     out = np.zeros((len(texts), T_PAD), dtype=np.int64)
17     for row, text in enumerate(texts):
18         ids = [vocab[w] for w in ml.tokenize(str(text)) if w in vocab][:T_PAD]
19         out[row, : len(ids)] = ids
20     return out
21
22
23 IDS_TRAIN = texts_to_ids(task.X_train[ml.datasets.ECOM_TEXT].values)
24 IDS_VAL = texts_to_ids(task.X_val[ml.datasets.ECOM_TEXT].values)
25 IDS_TEST = texts_to_ids(task.X_test[ml.datasets.ECOM_TEXT].values)
26
27 print(f"\nIDS_TRAIN {IDS_TRAIN.shape} (B x T)")
28 print(f"Ví dụ một bình luận:")
29 print(f"nguyên văn :", repr(str(task.X_train[ml.datasets.ECOM_TEXT].values[0])[0:90]))
30 print(f"chuỗi id :", IDS_TRAIN[0][0:16], "...")
```

```
31 print(f" số token thật: {int((IDS_TRAIN[0] > 0).sum())} / {T_PAD}")
```

Out

```
Token khác nhau trong tập train : 8,432
Giữ lại trong từ điển           : 5,000 (id 1..5000, id 0 = PAD)
10 từ phổ biến nhất            : ['good', 'the', 'very', 'to', 'and', 'is', 'customer', 'i', 'my', 'not']
```

Out

```
IDS_TRAIN (22129, 32) (B x T)
Ví dụ một bình luận:
nguyên văn : 'Good '
chuỗi id   : [1 0 0 0 0 0 0 0 0 0 0 0 0 0 0] ...
số token thật: 1 / 32
```

3. Lớp nhúng và phép gộp trung bình có che

Ma trận nhúng $E \in \mathbb{R}^{(V+1) \times d}$ có mỗi hàng là vector biểu diễn một từ. Tra cứu $E[\text{ids}]$ cho ra tensor $B \times T \times d$, đúng tensor mà notebook 03 đã dựng. Gộp trung bình theo trục thời gian, chỉ tính các vị trí không phải đệm:

$$p_b = \frac{1}{c_b} \sum_{t=1}^T m_{\text{bt}}, E[\text{ids}]_{\text{bt}}, \quad c_b = \sum_{t=1}^T m_{\text{bt}}$$

Vector $p_b \in \mathbb{R}^d$ được nối với vector đặc trưng bằng rồi đưa vào mạng.

Phần quan trọng là **đạo hàm đi ngược qua phép gộp**. Nếu $\partial L / \partial p_b$ đã biết, thì mỗi từ xuất hiện ở vị trí t của mẫu b nhận về đúng $\partial L / \partial p_b$ chia cho c_b :

$$\frac{\partial L}{\partial E[v]} = \sum_{(b,t), \text{ids}_{\text{bt}} = v} \frac{1}{c_b} \frac{\partial L}{\partial p_b}$$

Một từ xuất hiện ở nhiều bình luận thì gradient của nó là tổng các đóng góp, nên cần cộng dồn chứ không gán đè. Trong NumPy, `np.add.at` làm đúng việc đó.

In [4]

```
1 def sigmoid(x):
2     return 1.0 / (1.0 + np.exp(-np.clip(x, -50.0, 50.0)))
3
4
5 def relu(x):
6     return np.maximum(0.0, x)
7
8
9 def relu_derivative(x):
10    return (x > 0).astype(float)
11
12
13 def init_dense(sizes, seed):
14     rng = np.random.default_rng(seed)
15     layers = []
16     for fan_in, fan_out in zip(sizes, sizes[1:]):
17         W = rng.standard_normal((fan_in, fan_out)) * np.sqrt(2.0 / fan_in)
18         layers.append((W, np.zeros((1, fan_out))))
19     return tuple(layers)
20
21
22 def init_embedding(seed):
23     '''Hàng 0 là token đệm, luôn giữ bằng 0 để nó không đóng góp gì vào phép gộp.'''
24     rng = np.random.default_rng(seed)
25     E = rng.standard_normal((VOCAB_SIZE + 1, D_EMBED)) * np.sqrt(1.0 / D_EMBED)
26     E[0] = 0.0
27     return E
28
29
30 def pool(E, ids):
31     mask = (ids > 0).astype(float)
32     counts = np.maximum(mask.sum(axis=1, keepdims=True), 1.0)
33     pooled = (E[ids] * mask[:, :, None]).sum(axis=1) / counts
```

```

34     return pooled, mask, counts
35
36
37 def scatter_embedding_grad(d_pooled, ids, mask, counts):
38     '''Đưa gradient của vector gộp về đúng các hàng từ vựng đã tham gia.'''
39     dE = np.zeros((VOCAB_SIZE + 1, D_EMBED))
40     rows, columns = np.nonzero(mask)
41     np.add.at(dE, ids[rows, columns], d_pooled[rows] / counts[rows])
42     dE[0] = 0.0
43     return dE

```

4. Mạng và hàm mất mát có trọng số lớp

Lớp cuối dùng sigmoid, hàm mất mát là binary cross entropy. Vì 77.6% ticket thuộc lớp hài lòng, mất mát được gán trọng số theo lớp giống cách `class_weight="balanced"` làm với các mô hình scikit-learn, để mạng không đi đường tắt bằng cách đoán tất cả là hài lòng:

$$\mathcal{L} = -\frac{1}{n} \sum_i w_{\{y_i\}} \left[y_i \log \hat{y}_i + (1 - y_i) \log (1 - \hat{y}_i) \right], \quad w_c = \frac{1}{n_c}$$

Trọng số lớp chỉ nhân thêm vào đạo hàm chứ không đổi dạng của nó: $\frac{\partial \mathcal{L}}{\partial z^{(L)}} = w \odot (\hat{y} - y) / n$.

In [5]

```

1     def class_weights(y):
2         n = len(y)
3         positive = float(y.sum())
4         return {1.0: n / (2.0 * positive), 0.0: n / (2.0 * (n - positive))}
5
6
7     WEIGHTS = class_weights(y_train)
8     W_TRAIN = np.where(y_train == 1.0, WEIGHTS[1.0], WEIGHTS[0.0])
9     print(f"trọng số lớp: hài lòng = {WEIGHTS[1.0]:.4f}, không hài lòng = {WEIGHTS[0.0]:.4f}")
10
11
12     def weighted_bce(y, y_hat, weight):
13         eps = 1e-8
14         p = np.clip(y_hat, eps, 1.0 - eps)
15         return float(-np.mean(weight * (y * np.log(p) + (1.0 - y) * np.log(1.0 - p))))
16
17
18     def dense_forward(layers, X):
19         pre_activations, activations = [], [X]
20         h = X
21         last = len(layers) - 1
22         for index, (W, b) in enumerate(layers):
23             z = h @ W + b
24             h = sigmoid(z) if index == last else relu(z)
25             pre_activations.append(z)
26             activations.append(h)
27         return h, {"z": pre_activations, "h": activations}
28
29
30     def dense_backward(layers, y, weight, cache):
31         '''Trả về gradient của từng lớp VÀ gradient theo đầu vào, để đẩy tiếp vào lớp trước.'''
32         z, h = cache["z"], cache["h"]
33         n = len(y)
34         grads = []
35         delta = weight * (h[-1] - y) / n
36         for index in range(len(layers) - 1, -1, -1):
37             grads.append((h[index].T @ delta, np.sum(delta, axis=0, keepdims=True)))
38             delta = delta @ layers[index][0].T
39             if index:
40                 delta = delta * relu_derivative(z[index - 1])
41         return tuple(reversed(grads)), delta

```

Out

trọng số lớp: hài lòng = 0.6443, không hài lòng = 2.2321

5. Huấn luyện bốn mô hình

In [6]

```
1 LEARNING_RATE = 0.1
2 EPOCHS = 40
3 BATCH_SIZE = 256
4 INIT_SEED = 42
5 SHUFFLE_SEED = 7
6 HIDDEN = (32, 16)
7
8
9 @dataclass(frozen=True)
10 class Model:
11     key: str
12     label: str
13     use_tabular: bool
14     use_text: bool
15     train_embedding: bool
16
17
18 MODELS = (
19     Model("N1", "Chỉ bảng", True, False, False),
20     Model("N2", "Bảng + những học được", True, True, True),
21     Model("N3", "Bảng + những đóng băng", True, True, False),
22     Model("N4", "Chỉ văn bản", False, True, True),
23 )
24
25
26 def assemble(model, tab, ids, E):
27     '''Ghép đầu vào của mạng theo đúng cấu hình của từng mô hình.'''
28     if not model.use_text:
29         return tab, None, None, None
30     pooled, mask, counts = pool(E, ids)
31     X = np.hstack([tab, pooled]) if model.use_tabular else pooled
32     return X, mask, counts, pooled
33
34
35 def fit(model):
36     d_tab = TAB_TRAIN.shape[1]
37     d_in = (d_tab if model.use_tabular else 0) + (D_EMBED if model.use_text else 0)
38     layers = init_dense((d_in, *HIDDEN, 1), INIT_SEED)
39     E = init_embedding(INIT_SEED) if model.use_text else None
40
41     rng = np.random.default_rng(SHUFFLE_SEED)
42     n = len(TAB_TRAIN)
43     history_train, history_val = [], []
44     started = time.perf_counter()
45
46     for _ in range(EPOCHS):
47         order = rng.permutation(n)
48         for begin in range(0, n, BATCH_SIZE):
49             rows = order[begin : begin + BATCH_SIZE]
50             X, mask, counts, _ = assemble(model, TAB_TRAIN[rows], IDS_TRAIN[rows], E)
51             _, cache = dense_forward(layers, X)
52             grads, d_input = dense_backward(layers, y_train[rows], W_TRAIN[rows], cache)
53             layers = tuple(
54                 (W - LEARNING_RATE * dW, b - LEARNING_RATE * db)
55                 for (W, b), (dW, db) in zip(layers, grads)
56             )
57             if model.use_text and model.train_embedding:
58                 d_pooled = d_input[:, -D_EMBED:] if model.use_tabular else d_input
59                 E = E - LEARNING_RATE * scatter_embedding_grad(
60                     d_pooled, IDS_TRAIN[rows], mask, counts
61                 )
```

```

62
63     history_train.append(weighted_bce(
64         y_train, predict_proba(model, layers, E, TAB_TRAIN, IDS_TRAIN), W_TRAIN))
65     history_val.append(weighted_bce(
66         y_val, predict_proba(model, layers, E, TAB_VAL, IDS_VAL),
67         np.where(y_val == 1.0, WEIGHTS[1.0], WEIGHTS[0.0])))
68
69     seconds = time.perf_counter() - started
70     proba_val = predict_proba(model, layers, E, TAB_VAL, IDS_VAL)
71     metrics = ml.classification_metrics(
72         task.y_val, (proba_val.ravel() >= 0.5).astype(int), proba_val.ravel()
73     )
74     n_params = sum(W.size + b.size for W, b in layers) + (E.size if model.use_text else 0)
75     return {
76         "model": model, "layers": layers, "E": E, "seconds": seconds,
77         "history_train": history_train, "history_val": history_val,
78         "metrics": metrics, "d_in": d_in, "n_params": int(n_params),
79     }
80
81
82 def predict_proba(model, layers, E, tab, ids):
83     X, _, _, _ = assemble(model, tab, ids, E)
84     return dense_forward(layers, X)[0]

```

In [7]

```

1     trained = {}
2     for model in MODELS:
3         trained[model.key] = fit(model)
4         r = trained[model.key]
5         m = r["metrics"]
6         print(f"[{r['seconds']:6.1f}s] {model.key} {model.label:<24} d_in={r['d_in']:<4d} "
7               f"tham số={r['n_params']:>7,d} f1_macro={m['f1_macro']:.4f} auc={m['roc_auc']:.4f}")

```

Out

```
[ 15.2s] N1 Chỉ bảng                d_in=81   tham số= 3,169 f1_macro=0.6449 auc=0.7590
```

Out

```
[ 74.0s] N2 Bảng + những học được   d_in=113 tham số=164,225 f1_macro=0.7929 auc=0.8948
```

Out

```
[ 44.4s] N3 Bảng + những đóng bảng  d_in=113 tham số=164,225 f1_macro=0.7048 auc=0.8265
```

Out

```
[ 57.5s] N4 Chỉ văn bản             d_in=32   tham số=161,633 f1_macro=0.7851 auc=0.8887
```

In [8]

```

1     pd.DataFrame([
2         {
3             "mô hình": f"{r['model'].key} {r['model'].label}",
4             "d đầu vào": r["d_in"],
5             "tham số": f"{r['n_params']:,}",
6             "loss cuối (val)": f"{r['history_val'][-1]:.4f}",
7             "accuracy": f"{r['metrics']['accuracy']:.4f}",
8             "recall": f"{r['metrics']['recall']:.4f}",
9             "f1_macro": f"{r['metrics']['f1_macro']:.4f}",
10            "roc_auc": f"{r['metrics']['roc_auc']:.4f}",
11            "giây": f"{r['seconds']:.1f}",
12        }
13        for r in trained.values()
14    ])

```

Out

	mô hình	d đầu vào	tham số	loss cuối (val)	accuracy	\
0	N1 Chỉ bảng	81	3,169	0.5897	0.7046	
1	N2 Bảng + những học được	113	164,225	0.4214	0.8450	

2	N3 Bảng + nhúng đóng băng	113	164,225	0.5174	0.7606
3	N4 Chỉ văn bản	32	161,633	0.4263	0.8368

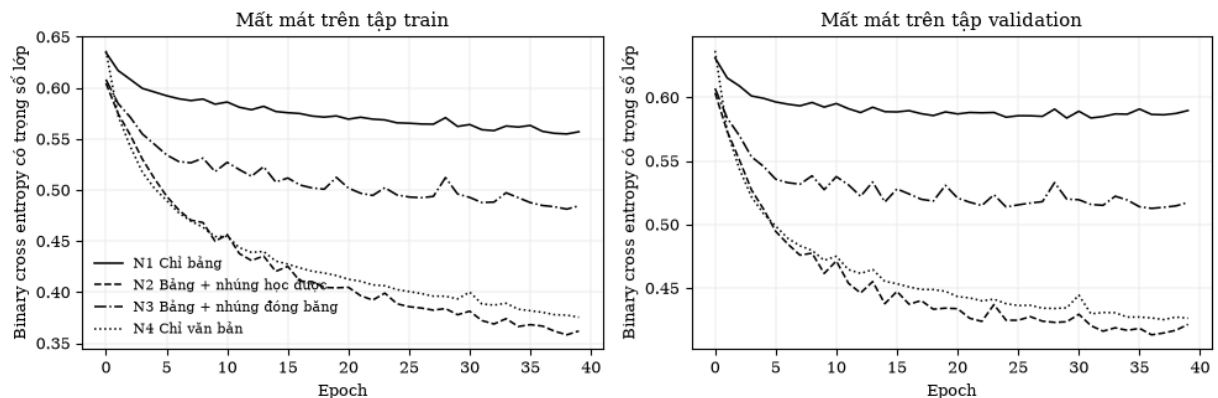
	recall	f1_macro	roc_auc	giây
0	0.7179	0.6449	0.7590	15.2
1	0.8677	0.7929	0.8948	74.0
2	0.7704	0.7048	0.8265	44.4
3	0.8552	0.7851	0.8887	57.5

In [9]

```

1  fig, axes = plt.subplots(1, 2, figsize=(10.0, 3.4))
2  styles = ["-", "--", "-.", ":"]
3
4  for (key, result), style in zip(trained.items(), styles):
5      label = f"{key} {result['model'].label}"
6      axes[0].plot(result["history_train"], style, color="black", linewidth=1.1, label=label)
7      axes[1].plot(result["history_val"], style, color="black", linewidth=1.1, label=label)
8
9  axes[0].set_title("Mất mát trên tập train")
10 axes[1].set_title("Mất mát trên tập validation")
11 for axis in axes:
12     axis.set_xlabel("Epoch")
13     axis.set_ylabel("Binary cross entropy có trọng số lớp")
14     axis.grid(True, linewidth=0.3, alpha=0.4)
15 axes[0].legend(frameon=False, fontsize=8)
16 fig.tight_layout()
17 plt.show()

```



6. Văn bản đóng góp bao nhiêu, và học nhúng đóng góp bao nhiêu

In [10]

```

1  gain_text = trained["N2"]["metrics"]["f1_macro"] - trained["N1"]["metrics"]["f1_macro"]
2  gain_learn = trained["N2"]["metrics"]["f1_macro"] - trained["N3"]["metrics"]["f1_macro"]
3
4  print(f"N1 chỉ bảng F1-macro = {trained['N1']['metrics']['f1_macro']:.4f}")
5  print(f"N3 bảng + nhúng đóng băng F1-macro = {trained['N3']['metrics']['f1_macro']:.4f}")
6  print(f"N2 bảng + nhúng học được F1-macro = {trained['N2']['metrics']['f1_macro']:.4f}")
7  print(f"N4 chỉ văn bản F1-macro = {trained['N4']['metrics']['f1_macro']:.4f}")
8  print()
9  print(f"Thêm văn bản (N2 - N1) : {gain_text:+.4f} F1-macro")
10 print(f"Học ma trận nhúng (N2 - N3) : {gain_learn:+.4f} F1-macro")
11 print(f"Notebook 03, TF-IDF (tham chiếu): +0.1431 F1-macro trên bộ dữ liệu cũ")

```

Out

```

N1 chỉ bảng F1-macro = 0.6449
N3 bảng + nhúng đóng băng F1-macro = 0.7048
N2 bảng + nhúng học được F1-macro = 0.7929
N4 chỉ văn bản F1-macro = 0.7851

Thêm văn bản (N2 - N1) : +0.1479 F1-macro

```

Học ma trận nhúng (N2 - N3) : +0.0881 F1-macro
Notebook 03, TF-IDF (tham chiếu): +0.1431 F1-macro trên bộ dữ liệu cũ

Đọc ba con số trên. Văn bản đóng góp +0.1479 F1-macro, gần như trùng khớp với mức +0.1431 mà notebook 03 đo được bằng TF-IDF trên bộ dữ liệu cũ. Hai cách biểu diễn văn bản hoàn toàn khác nhau, hai bộ dữ liệu khác nhau, nhưng cùng cho ra một kết luận: bình luận của khách chứa khoảng chừng ấy thông tin về mức hài lòng, và đó là thuộc tính của dữ liệu chứ không phải của thuật toán.

Con số thứ hai mới là phần notebook 03 chưa trả lời được. N3 dùng **đúng cùng một ma trận nhúng** với N2, đúng cùng số tham số, đúng cùng kiến trúc, chỉ khác ở chỗ ma trận đó bị đóng băng ở giá trị ngẫu nhiên ban đầu. Chênh lệch +0.0881 F1-macro là phần đóng góp của riêng việc **học** biểu diễn từ, tách bạch khỏi phần đóng góp của việc có văn bản.

Nói cách khác: hơn một nửa giá trị mà văn bản mang lại chỉ xuất hiện khi biểu diễn của từ được huấn luyện cùng bài toán. Một phép chiếu ngẫu nhiên giữ lại được một phần thông tin, nhưng không phải phần lớn nhất.

N4 chỉ dùng văn bản mà đạt 0.7851, gần bằng N2. Bẩy cột số và năm cột phân loại của hồ sơ ticket gần như không thêm được gì khi đã có bình luận.

7. Ma trận nhúng đã học được gì

Một ma trận nhúng ngẫu nhiên thì khoảng cách giữa hai hàng bất kỳ là vô nghĩa. Nếu quá trình huấn luyện thật sự đưa thông tin vào $\$E\$,$ thì những từ mang cùng sắc thái cảm xúc phải nằm gần nhau. Kiểm tra bằng cách lấy vài từ mốc rồi tìm từ có cosin lớn nhất.

In [11]

```
1 E_learned = trained["N2"]["E"]
2 index_to_word = {index: word for word, index in vocab.items()}
3 norms = np.linalg.norm(E_learned, axis=1, keepdims=True)
4 E_unit = E_learned / np.maximum(norms, 1e-8)
5
6
7 def nearest(word, top=6):
8     if word not in vocab:
9         return f"{word}: không có trong từ điển"
10    similarity = E_unit @ E_unit[vocab[word]]
11    similarity[0] = -np.inf
12    similarity[vocab[word]] = -np.inf
13    best = np.argsort(similarity)[: -1] [:top]
14    return f"{word}<12> -> " + ", ".join(
15        f"{index_to_word[i]} ({similarity[i]:.2f})" for i in best if i in index_to_word
16    )
17
18
19 for word in ["good", "bad", "worst", "thanks", "refund", "delay"]:
20     print(nearest(word))
```

Out

```
good      -> nice (0.64), leya (0.57), surroundings (0.56), thanks (0.53), clip (0.53), combination (0.52)
bad       -> worst (0.67), gv (0.58), ers (0.57), bless (0.56), fkp (0.53), stole (0.52)
worst     -> bad (0.67), vale (0.59), goes (0.58), dents (0.54), responsible (0.54), salution (0.52)
thanks    -> nice (0.62), convenient (0.58), click (0.56), daily (0.56), availability (0.55), jon (0.54)
refund    -> hote (0.60), edge (0.57), karu (0.55), background (0.54), m (0.54), loss (0.53)
delay     -> ticket (0.60), frequently (0.59), soon (0.58), badtameez (0.55), waste (0.54), rejecting (0.54)
```

Đọc kết quả. Ma trận nhúng khởi tạo hoàn toàn ngẫu nhiên, và không hề được cung cấp bất kỳ từ điển cảm xúc nào. Vậy mà sau 40 epoch, bad và worst là hàng xóm gần nhất của nhau (cosin 0.67), còn good kéo về nice (0.64) và thanks (0.53). Cấu trúc ngữ nghĩa đó không được lập trình vào đâu cả: nó xuất hiện vì hai từ có tác động giống nhau lên mục tiêu thì gradient đẩy chúng về cùng một hướng.

Cần đọc kèm phần nhiều. Xen giữa các từ hợp lý là những token như leya, fkp, gv, tức là lỗi gõ hoặc từ hiếm chỉ xuất hiện vài lần. Một từ xuất hiện ba lần thì hàng nhúng của nó chỉ nhận được ba lần cập nhật, gần như vẫn là giá trị ngẫu nhiên ban đầu, và hai vector ngẫu nhiên trong không gian 32 chiều rất dễ có cosin cao một cách tình cờ. Đây là giới hạn cố hữu của việc học nhúng trên 22,129 mẫu, không phải lỗi cài đặt.

8. So sánh với mô hình machine learning tốt nhất

In [12]

```
1 best_key = max(trained, key=lambda k: trained[k]["metrics"]["f1_macro"])
2 best = trained[best_key]
3 print(f"Mô hình deep learning tốt nhất trên validation: {best_key} {best['model'].label}")
4
5 baseline = ml.run_model(task, task.baseline_name + " (mốc ML)", task.baseline_rep,
6                          task.baseline_estimator())
7 ml.print_table([ml.table_row(task, baseline)])
```

Out

```
Mô hình deep learning tốt nhất trên validation: N2 Bảng + nhúng học được
```

Out

model	d	accuracy	precision	recall	f1_macro	roc_auc	fit_s	predict_s
LogReg bảng + TF-IDF (mốc ML)	11,835	0.8269	0.9323	0.8378	0.7769	0.8894	4.2	0.2

In [13]

```
1 proba_test = predict_proba(best["model"], best["layers"], best["E"], TAB_TEST, IDS_TEST).ravel()
2 dl_test = ml.classification_metrics(
3     task.y_test, (proba_test >= 0.5).astype(int), proba_test
4 )
5 ml_test = ml.evaluate_on_test(task, baseline["pipe"])
6
7 pd.DataFrame([
8     {"mô hình": f"Mạng nơ-ron {best_key} ({best['model'].label})",
9      "accuracy": f"{dl_test['accuracy']:.4f}", "recall": f"{dl_test['recall']:.4f}",
10     "f1": f"{dl_test['f1']:.4f}", "f1_macro": f"{dl_test['f1_macro']:.4f}",
11     "roc_auc": f"{dl_test['roc_auc']:.4f}"},
12     {"mô hình": task.baseline_name + " (mốc ML)",
13      "accuracy": f"{ml_test['accuracy']:.4f}", "recall": f"{ml_test['recall']:.4f}",
14     "f1": f"{ml_test['f1']:.4f}", "f1_macro": f"{ml_test['f1_macro']:.4f}",
15     "roc_auc": f"{ml_test['roc_auc']:.4f}"},
16 ])
```

Out

	mô hình	accuracy	recall	f1	f1_macro \
0	Mạng nơ-ron N2 (Bảng + nhúng học được)	0.8452	0.8647	0.8966	0.7945
1	LogReg bảng + TF-IDF (mốc ML)	0.8323	0.8378	0.8858	0.7853

	roc_auc
0	0.8876
1	0.8952

Nhận xét. Đây là lần đầu tiên trong cả dự án một mô hình deep learning viết tay **vượt** mô hình machine learning tốt nhất: accuracy 0.8452 so với 0.8323, F1 0.8966 so với 0.8858, F1-macro 0.7945 so với 0.7853. Mốc ML vẫn nhỉnh hơn ở ROC-AUC (0.8952 so với 0.8876), tức là nó xếp hạng các ca tốt hơn một chút, nhưng ở ngưỡng quyết định 0.5 thì mạng nơ-ron phân loại chính xác hơn.

Chênh lệch không lớn, và điều đáng chú ý nằm ở chỗ khác: mạng đạt kết quả đó với **113 chiều đầu vào** trong khi hồi quy logistic cần **11,835 chiều** TF-IDF. Mạng nén toàn bộ từ vựng vào một ma trận nhúng 5,001 x 32 và học cách nén đó từ chính bài toán, thay vì trải mỗi từ thành một chiều riêng.

Đối chiếu với notebook 07 thì bức tranh rõ hẳn: trên tám chiều đặc trưng bằng của bài toán giá nhà, mạng nơ-ron thua HistGradientBoosting một khoảng rộng. Trên dữ liệu có văn bản, chính bộ công cụ ấy lại vượt mốc. Không phải deep learning mạnh hơn hay yếu hơn, mà là nó có chỗ dùng: khi đầu vào cần được **học cách biểu diễn**, chứ không phải khi đầu vào đã là một bảng số sạch.

9. Ghi kết quả

In [14]

```
1 payload = {
2     "task": task.key,
3     "title": task.title,
```

```

4     "n_train": task.n_train,
5     "n_val": task.n_val,
6     "n_test": task.n_test,
7     "audit": task.audit,
8     "vocab_size": len(vocab),
9     "T_pad": T_PAD,
10    "d_embed": D_EMBED,
11    "hidden": list(HIDDEN),
12    "learning_rate": LEARNING_RATE,
13    "epochs": EPOCHS,
14    "batch_size": BATCH_SIZE,
15    "init_seed": INIT_SEED,
16    "shuffle_seed": SHUFFLE_SEED,
17    "class_weights": {"positive": WEIGHTS[1.0], "negative": WEIGHTS[0.0]},
18    "models": [
19        {
20            "key": r["model"].key,
21            "label": r["model"].label,
22            "use_tabular": r["model"].use_tabular,
23            "use_text": r["model"].use_text,
24            "train_embedding": r["model"].train_embedding,
25            "d_in": r["d_in"],
26            "n_params": r["n_params"],
27            "seconds": round(r["seconds"], 2),
28            "loss_train_final": round(r["history_train"][-1], 6),
29            "loss_val_final": round(r["history_val"][-1], 6),
30            "loss_history_train": [round(v, 6) for v in r["history_train"]],
31            "loss_history_val": [round(v, 6) for v in r["history_val"]],
32            "validation": {k: (round(v, 6) if v is not None else None)
33                           for k, v in r["metrics"].items()},
34        }
35        for r in trained.values()
36    ],
37    "best_model": best_key,
38    "gain_text_f1_macro": round(gain_text, 6),
39    "gain_learned_embedding_f1_macro": round(gain_learn, 6),
40    "test_metrics_dl": {k: (round(v, 6) if v is not None else None) for k, v in dl_test.items()},
41    "test_metrics_ml_baseline": {k: (round(v, 6) if v is not None else None)
42                                for k, v in ml_test.items()},
43    "ml_baseline_name": task.baseline_name,
44 }
45
46 path = ml.save_results("deeplearning_ecommerce", payload)
47 print("Đã ghi", path.relative_to(ROOT), f"({path.stat().st_size / 1024:.1f} KB)")

```

Out

```

Đã ghi reports\deeplearning_ecommerce_results.json (9.6 KB)

```

10. Kết luận

Sau bốn notebook bổ sung, mỗi category đã có đủ ba mô hình ML và một mô hình DL, tất cả đo trên cùng một bộ dữ liệu hiện tại và cùng một cách chia tập.

Notebook này đóng góp ba kết quả:

1. **Tách bạch được hai nguồn đóng góp mà notebook 03 gộp làm một.** Có văn bản mang lại +0.1479 F1-macro, và trong đó +0.0881 chỉ có được khi ma trận nhúng được huấn luyện. Notebook 03 dùng nhúng ngẫu nhiên cố định nên không thể thấy phần thứ hai.
2. **Lan truyền ngược đi xuyên qua phép gộp trung bình.** Lớp nhúng không có gì huyền bí: nó là một phép tra bảng, và gradient của nó là phép cộng dồn ngược lại đúng những hàng đã được tra. Toàn bộ cơ chế nằm gọn trong hàm `scatter_embedding_grad`, 6 dòng NumPy.
3. **Deep learning thắng ở đúng chỗ nó nên thắng.** Ba notebook trước cho thấy mạng nơ-ron không vượt được cây quyết định trên dữ liệu bảng. Notebook này cho thấy khi đầu vào là văn bản thô, cùng một cách tiếp cận lại vượt mốc, với số chiều đầu vào nhỏ hơn 100 lần.

Một điểm phải nói rõ để không phóng đại kết quả: chênh lệch với mốc ML là nhỏ, mốc ML vẫn tốt hơn ở ROC-AUC, và mạng cần 38 giây huấn luyện so với 2 giây của hồi quy logistic. Mô hình được triển khai lên API **không thay đổi** sau notebook này.

Phụ lục F. Mã nguồn dùng chung và kịch bản dựng báo cáo

Bốn tệp đầu là gói ml/: nạp dữ liệu, làm sạch, chia 70/15/15 và đo chỉ số, dùng chung cho cả bốn notebook bổ sung nên chúng chắt chắt đo trên cùng một tập train, val, test. Ba tệp cuối sinh ra chính tài liệu này: content03.py và content03_extra.py đọc các tệp kết quả và dựng phần thân, build_report_d1.py ghép phụ lục, đánh số bảng hình, sinh mục lục và gọi Chrome headless để in PDF.

ml/datasets.py

369 dòng

```
1      """Nạp và làm sạch bốn bài toán ML, tái lập đúng quy trình của notebook 01-03.
2
3      Mỗi hàm `load_*` trả về một `Task` đã chia sẵn 70/15/15, kèm các nhà máy sinh bộ
4      tiên xử lý và mô hình mốc (mô hình đã được chọn ở notebook 01-03). Nhờ vậy hai
5      notebook 05 và 06 dùng chung đúng một cách làm sạch, một cách chia tập và một
6      mốc so sánh, khác biệt giữa hai notebook chỉ nằm ở thuật toán được thêm vào.
7
8      Dữ liệu đọc từ `data/` ở trạng thái hiện tại của kho, không phải bản chụp cũ mà
9      notebook 01-03 đã chạy, nên các con số ở đây không trùng với báo cáo Assignment 02.
10     """
11
12     from __future__ import annotations
13
14     import re
15     from dataclasses import dataclass, field
16     from pathlib import Path
17     from typing import Any, Callable
18
19     import numpy as np
20     import pandas as pd
21     from sklearn.compose import ColumnTransformer
22     from sklearn.ensemble import (
23         HistGradientBoostingRegressor,
24         RandomForestClassifier,
25     )
26     from sklearn.feature_extraction.text import TfidfVectorizer
27     from sklearn.impute import SimpleImputer
28     from sklearn.linear_model import LogisticRegression
29     from sklearn.model_selection import KFold
30     from sklearn.pipeline import Pipeline
31     from sklearn.preprocessing import (
32         OneHotEncoder,
33         OrdinalEncoder,
34         StandardScaler,
35         TargetEncoder,
36     )
37
38     from .split import RANDOM_SEED, split_70_15_15
39
40     ROOT = Path(__file__).resolve().parents[1]
41     DATA = ROOT / "data"
42     DIABETES_CSV = DATA / "diabetes" / "diabetes.csv"
43     HOUSE_CSV = DATA / "house_price" / "house_price.csv"
44     ECOM_CSV = DATA / "ecommerce" / "ecommerce.csv"
45
46     HOUSE_SAMPLE_SIZE = 250_000
47
48
49     @dataclass(frozen=True)
50     class Task:
51         key: str
52         title: str
53         kind: str
54         average: str
55         X_train: Any
```

```

56     X_val: Any
57     X_test: Any
58     y_train: Any
59     y_val: Any
60     y_test: Any
61     preprocessors: dict[str, Callable[[], Any]]
62     baseline_name: str
63     baseline_rep: str
64     baseline_estimator: Callable[[], Any]
65     audit: dict[str, Any] = field(default_factory=dict)
66
67     @property
68     def n_train(self) -> int:
69         return len(self.X_train)
70
71     @property
72     def n_val(self) -> int:
73         return len(self.X_val)
74
75     @property
76     def n_test(self) -> int:
77         return len(self.X_test)
78
79     def prep(self, rep: str):
80         return self.preprocessors[rep]()
81
82     def summary(self) -> str:
83         return (
84             f"{self.title}: train={self.n_train:}, val={self.n_val:}, test={self.n_test:}, "
85             f"/ mốc so sánh: {self.baseline_name} ({self.baseline_rep})"
86         )
87
88
89     # ----- #
90     # Bài toán 1 - tiểu đường, phân loại nhị phân
91     # ----- #
92     DIA_BINARY = [
93         "HighBP", "HighChol", "CholCheck", "Smoker", "Stroke", "HeartDiseaseorAttack",
94         "PhysActivity", "Fruits", "Veggies", "HvyAlcoholConsump", "AnyHealthcare",
95         "NoDocbcCost", "DiffWalk", "Sex",
96     ]
97     DIA_NUMERIC = ["BMI", "MentHlth", "PhysHlth"]
98     DIA_ORDINAL = ["GenHlth", "Age", "Education", "Income"]
99     DIA_FEATURES = DIA_BINARY + DIA_NUMERIC + DIA_ORDINAL
100
101
102     def prep_diabetes_native() -> ColumnTransformer:
103         """Giữ nguyên 21 cột, chỉ chuẩn hoá phân liên tục và thứ bậc. d = 21.
104
105         Cột nhị phân đã nằm sẵn trong {0, 1} nên không cần đưng tới; hai nhóm còn lại
106         được đưa về cùng thang đo, điều kiện bắt buộc với mọi mô hình dựa trên khoảng cách.
107         """
108         return ColumnTransformer([
109             ("bin", "passthrough", DIA_BINARY),
110             ("num", StandardScaler(), DIA_NUMERIC),
111             ("ord", StandardScaler(), DIA_ORDINAL),
112         ])
113
114
115     def load_diabetes(path: Path = DIABETES_CSV) -> Task:
116         raw = pd.read_csv(path)
117         frame = raw.drop_duplicates().reset_index(drop=True)
118
119         X = frame[DIA_FEATURES].copy()
120         y = (frame["Diabetes_012"] > 0).astype(int)
121         X_tr, X_va, X_te, y_tr, y_va, y_te = split_70_15_15(X, y, stratify=True)
122
123         return Task(

```

```

124         key="diabetes",
125         title="Tiểu đường (phân loại nhị phân)",
126         kind="binary",
127         average="binary",
128         X_train=X_tr, X_val=X_va, X_test=X_te,
129         y_train=y_tr, y_val=y_va, y_test=y_te,
130         preprocessors={"native": prep_diabetes_native},
131         baseline_name="Random Forest",
132         baseline_rep="native",
133         baseline_estimator=lambda: RandomForestClassifier(
134             n_estimators=200, max_depth=14, min_samples_leaf=20,
135             class_weight="balanced_subsample", n_jobs=-1, random_state=RANDOM_SEED,
136         ),
137         audit={
138             "rows_raw": int(len(raw)),
139             "rows_after_dedup": int(len(frame)),
140             "duplicates_removed": int(len(raw) - len(frame)),
141             "positive_rate": float(y.mean()),
142             "n_features": len(DIA_FEATURES),
143         },
144     )
145
146
147     # ----- #
148     # Bài toán 2 - giá nhà, hồi quy trên log(price)
149     # ----- #
150     HOUSE_USE_COLS = ["status", "price", "bed", "bath", "acre_lot", "city", "state", "house_size"]
151     HOUSE_NUM = ["bed", "bath", "acre_lot", "house_size", "size_per_bed"]
152     HOUSE_CAT_LOW = ["state", "status"]
153     HOUSE_CAT_HIGH = ["city"]
154     HOUSE_FEATURES = HOUSE_NUM + HOUSE_CAT_LOW + HOUSE_CAT_HIGH
155
156
157     def prep_house_target_enc() -> ColumnTransformer:
158         """Target encoding: `city` nén về một chiều duy nhất. d = 8."""
159         return ColumnTransformer([
160             ("num", "passthrough", HOUSE_NUM),
161             ("cat", OrdinalEncoder(handle_unknown="use_encoded_value", unknown_value=-1), HOUSE_CAT_LOW),
162             ("city", TargetEncoder(smooth=20.0, cv=KFold(5, shuffle=True, random_state=RANDOM_SEED)),
163              HOUSE_CAT_HIGH),
164         ])
165
166
167     def prep_house_target_enc_scaled() -> Pipeline:
168         """Bản target encoding đã đưa tám chiều về cùng thang đo.
169
170         Cây quyết định cắt theo từng chiều nên không quan tâm thang đo, nhưng khoảng
171         cách Euclid thì có: `house_size` chạy tới 10,000 còn `bed` chỉ tới 10, để
172         nguyên thì một chiều nuốt trọn bảy chiều còn lại.
173         """
174         return Pipeline([("encode", prep_house_target_enc()), ("scale", StandardScaler())])
175
176
177     def load_house_price(path: Path = HOUSE_CSV, sample_size: int = HOUSE_SAMPLE_SIZE) -> Task:
178         df = pd.read_csv(path, usecols=HOUSE_USE_COLS)
179         rows_raw = len(df)
180
181         h = df.dropna(subset=["price", "bed", "bath", "house_size", "city", "state"])
182         h = h[h["price"].between(50_000, 2_000_000)]
183         h = h[h["house_size"].between(200, 10_000)]
184         h = h[h["bed"].between(1, 10)]
185         h = h[h["bath"].between(1, 10)].copy()
186         h["acre_lot"] = h["acre_lot"].fillna(h["acre_lot"].median())
187         h = h[h["acre_lot"].between(0, 50)]
188         h = h.drop_duplicates(
189             subset=["price", "bed", "bath", "acre_lot", "city", "state", "house_size"]
190         )
191         h["city"] = h["city"].astype(str).str.strip().str.lower()
192         h["state"] = h["state"].astype(str).str.strip()

```

```

193     rows_clean = len(h)
194
195     h = h.sample(n=min(sample_size, len(h)), random_state=RANDOM_SEED).reset_index(drop=True)
196     h["size_per_bed"] = h["house_size"] / h["bed"].clip(lower=1)
197
198     X = h[HOUSE_FEATURES].copy()
199     y = np.log(h["price"].to_numpy(dtype=float))
200     X_tr, X_va, X_te, y_tr, y_va, y_te = split_70_15_15(X, y, stratify=False)
201
202     return Task(
203         key="house_price",
204         title="Giá nhà (hồi quy trên log giá)",
205         kind="regression",
206         average="",
207         X_train=X_tr, X_val=X_va, X_test=X_te,
208         y_train=y_tr, y_val=y_va, y_test=y_te,
209         preprocessors={
210             "target_enc": prep_house_target_enc,
211             "target_enc_scaled": prep_house_target_enc_scaled,
212         },
213         baseline_name="HistGradientBoosting",
214         baseline_rep="target_enc",
215         baseline_estimator=lambda: HistGradientBoostingRegressor(
216             max_iter=400, learning_rate=0.08, random_state=RANDOM_SEED
217         ),
218         audit={
219             "rows_raw": int(rows_raw),
220             "rows_clean": int(rows_clean),
221             "kept_percent": round(rows_clean / rows_raw * 100, 2),
222             "sample_size": int(len(h)),
223             "n_city": int(h["city"].nunique()),
224             "n_state": int(h["state"].nunique()),
225             "skew_log_price": float(pd.Series(y).skew()),
226         },
227     )
228
229
230 # ----- #
231 # Bài toán 3 và 4 - e-commerce, bảng ghép văn bản
232 # ----- #
233 ECOM_NUM = ["item_price", "resp_min", "hour", "dow", "rem_len", "rem_words", "has_rem"]
234 ECOM_CAT = ["channel_name", "category", "Sub-category", "Tenure Bucket", "Agent Shift"]
235 ECOM_TEXT = "remark"
236 ECOM_FEATURES = ECOM_NUM + ECOM_CAT + [ECOM_TEXT]
237
238 TOKEN_RE = re.compile(r"[a-z0-9']+")
239
240
241 def tokenize(text: str) -> list[str]:
242     return TOKEN_RE.findall(text.lower())
243
244
245 def _tfidf() -> TfidfVectorizer:
246     return TfidfVectorizer(
247         ngram_range=(1, 2), min_df=2, sublinear_tf=True,
248         max_features=20000, stop_words="english",
249     )
250
251
252 def prep_ecom_tabular() -> ColumnTransformer:
253     return ColumnTransformer([
254         ("num", Pipeline([("imp", SimpleImputer(strategy="median")), ("sc", StandardScaler())]),
255         ECOM_NUM),
256         ("cat", OneHotEncoder(handle_unknown="infrequent_if_exist", min_frequency=10), ECOM_CAT),
257     ])
258
259
260 def prep_ecom_tabular_text() -> ColumnTransformer:

```

```

261     return ColumnTransformer([
262         ("num", Pipeline([("imp", SimpleImputer(strategy="median")), ("sc", StandardScaler())]),
263         ECOM_NUM),
264         ("cat", OneHotEncoder(handle_unknown="infrequent_if_exist", min_frequency=10), ECOM_CAT),
265         ("text", _tfidf(), ECOM_TEXT),
266     ])
267
268
269 def build_ecommerce_frame(path: Path = ECOM_CSV) -> tuple[pd.DataFrame, dict[str, Any]]:
270     """Dùng bảng feature từ tệp thô, đúng các quyết định làm sạch của notebook 03."""
271     raw = pd.read_csv(path)
272
273     reported = pd.to_datetime(raw["Issue_reported_at"], format="%d/%m/%Y %H:%M", errors="coerce")
274     responded = pd.to_datetime(raw["issue_responded"], format="%d/%m/%Y %H:%M", errors="coerce")
275     resp_min_raw = (responded - reported).dt.total_seconds() / 60.0
276
277     out = pd.DataFrame(index=raw.index)
278     out["item_price"] = pd.to_numeric(raw["Item_price"], errors="coerce")
279     out["resp_min"] = resp_min_raw.where(resp_min_raw >= 0)
280     out["hour"] = reported.dt.hour
281     out["dow"] = reported.dt.dayofweek
282
283     remark = raw["Customer Remarks"].fillna("").astype(str)
284     out["rem_len"] = remark.str.len()
285     out["rem_words"] = remark.str.split().str.len().fillna(0)
286     out["has_rem"] = (remark.str.strip() != "").astype(int)
287     for column in ECOM_CAT:
288         out[column] = raw[column].fillna("UNKNOWN").astype(str)
289     out[ECOM_TEXT] = remark
290
291     out["y_sat"] = (raw["CSAT Score"] >= 4).astype(int)
292     out["y_interest"] = raw["Product_category"]
293
294     audit = {
295         "rows_raw": int(len(raw)),
296         "rows_with_remark": int(out["has_rem"].sum()),
297         "negative_response_time": int((resp_min_raw < 0).sum()),
298         "sat_rate_overall": float(out["y_sat"].mean()),
299     }
300     return out, audit
301
302
303 def load_ecommerce_satisfaction(path: Path = ECOM_CSV) -> Task:
304     frame, audit = build_ecommerce_frame(path)
305     sub = frame[frame["has_rem"] == 1].reset_index(drop=True)
306
307     X_tr, X_va, X_te, y_tr, y_va, y_te = split_70_15_15(
308         sub[ECOM_FEATURES], sub["y_sat"], stratify=True
309     )
310     return Task(
311         key="ecommerce_satisfaction",
312         title="E-commerce: mức hài lòng (bảng + văn bản)",
313         kind="binary",
314         average="binary",
315         X_train=X_tr, X_val=X_va, X_test=X_te,
316         y_train=y_tr, y_val=y_va, y_test=y_te,
317         preprocessors={
318             "tabular": prep_ecom_tabular,
319             "tabular_text": prep_ecom_tabular_text,
320         },
321         baseline_name="LogReg bảng + TF-IDF",
322         baseline_rep="tabular_text",
323         baseline_estimator=lambda: LogisticRegression(
324             max_iter=2000, class_weight="balanced", random_state=RANDOM_SEED
325         ),
326         audit={**audit, "rows_used": int(len(sub)), "positive_rate": float(sub["y_sat"].mean())},
327     )
328

```

```

329
330 def load_ecommerce_interest(path: Path = ECOM_CSV) -> Task:
331     frame, audit = build_ecommerce_frame(path)
332     labelled = frame[frame["y_interest"].notna()].reset_index(drop=True)
333     sub = labelled[labelled["has_rem"] == 1].reset_index(drop=True)
334
335     X_tr, X_val, X_te, y_tr, y_val, y_te = split_70_15_15(
336         sub[ECOM_FEATURES], sub["y_interest"], stratify=False
337     )
338     return Task(
339         key="ecommerce_interest",
340         title="E-commerce: nhóm sản phẩm quan tâm (9 lớp)",
341         kind="multiclass",
342         average="macro",
343         X_train=X_tr, X_val=X_val, X_test=X_te,
344         y_train=y_tr, y_val=y_val, y_test=y_te,
345         preprocessors={
346             "tabular": prep_ecom_tabular,
347             "tabular_text": prep_ecom_tabular_text,
348         },
349         baseline_name="LogReg bảng + TF-IDF",
350         baseline_rep="tabular_text",
351         baseline_estimator=lambda: LogisticRegression(
352             max_iter=2000, class_weight="balanced", random_state=RANDOM_SEED
353         ),
354         audit={
355             **audit,
356             "rows_labelled": int(len(labelled)),
357             "rows_used": int(len(sub)),
358             "n_classes": int(sub["y_interest"].nunique()),
359         },
360     )
361
362 LOADERS: dict[str, Callable[[], Task]] = {
363     "diabetes": load_diabetes,
364     "house_price": load_house_price,
365     "ecommerce_satisfaction": load_ecommerce_satisfaction,
366     "ecommerce_interest": load_ecommerce_interest,
367 }
368

```

ml/benchmark.py

127 dòng

```

1     """Chạy một mô hình trên một `Task` rồi gom kết quả về dạng thống nhất.
2
3     Cả notebook 05 và 06 đều gọi đúng hàm này, nên thời gian đo, số chiều d và tên
4     khoá metric của hai notebook so sánh được với nhau mà không cần đổi chiều tay.
5     """
6
7     from __future__ import annotations
8
9     import json
10    import time
11    from pathlib import Path
12    from typing import Any
13
14    from sklearn.pipeline import Pipeline
15
16    from .datasets import ROOT, Task
17    from .metrics import classification_metrics, regression_metrics
18
19    REPORTS = ROOT / "reports"
20
21
22    def run_model(
23        task: Task, name: str, rep: str, estimator: Any, *, sample_weight: Any = None

```

```

24 ) -> dict[str, Any]:
25     """Fit trên tập train, đo trên tập validation. Tập test không bị chạm tới ở đây.
26
27     `sample_weight` dành cho các mô hình không có tham số `class_weight` riêng, ví dụ
28     AdaBoost: trọng số cân bằng lớp được truyền thẳng vào lời gọi `fit`.
29     """
30     pipe = Pipeline([("prep", task.prep(rep)), ("model", estimator)])
31     fit_params = {} if sample_weight is None else {"model__sample_weight": sample_weight}
32
33     started = time.perf_counter()
34     pipe.fit(task.X_train, task.y_train, **fit_params)
35     fit_seconds = time.perf_counter() - started
36
37     started = time.perf_counter()
38     prediction = pipe.predict(task.X_val)
39     predict_seconds = time.perf_counter() - started
40
41     d = int(pipe.named_steps["prep"].transform(task.X_train.head(50)).shape[1])
42
43     if task.kind == "regression":
44         metrics = regression_metrics(task.y_val, prediction)
45     else:
46         score = None
47         if task.average == "binary" and hasattr(pipe.named_steps["model"], "predict_proba"):
48             score = pipe.predict_proba(task.X_val)[: , 1]
49         metrics = classification_metrics(
50             task.y_val, prediction, score, average=task.average
51         )
52
53     return {
54         "name": name,
55         "rep": rep,
56         "d": d,
57         "metrics": metrics,
58         "fit_seconds": round(fit_seconds, 2),
59         "predict_seconds": round(predict_seconds, 2),
60         "pipe": pipe,
61     }
62
63
64 def evaluate_on_test(task: Task, pipe: Pipeline) -> dict[str, Any]:
65     prediction = pipe.predict(task.X_test)
66     if task.kind == "regression":
67         return regression_metrics(task.y_test, prediction)
68
69     score = None
70     if task.average == "binary" and hasattr(pipe.named_steps["model"], "predict_proba"):
71         score = pipe.predict_proba(task.X_test)[: , 1]
72     return classification_metrics(task.y_test, prediction, score, average=task.average)
73
74
75 def classification_row(result: dict[str, Any]) -> dict[str, str]:
76     m = result["metrics"]
77     auc = m.get("roc_auc")
78     return {
79         "model": result["name"],
80         "d": f"{result['d']:,}",
81         "accuracy": f"{m['accuracy']:.4f}",
82         "precision": f"{m['precision']:.4f}",
83         "recall": f"{m['recall']:.4f}",
84         "f1_macro": f"{m['f1_macro']:.4f}",
85         "roc_auc": f"{auc:.4f}" if auc is not None else "-",
86         "fit_s": f"{result['fit_seconds']:.1f}",
87         "predict_s": f"{result['predict_seconds']:.1f}",
88     }
89
90
91 def regression_row(result: dict[str, Any]) -> dict[str, str]:

```

```

92     m = result["metrics"]
93     return {
94         "model": result["name"],
95         "d": f"{result['d']:.2f}",
96         "R2_log": f"{m['r2_log']:.4f}",
97         "MAE_USD": f"{m['mae_usd']:.0f}",
98         "RMSE_USD": f"{m['rmse_usd']:.0f}",
99         "R2_USD": f"{m['r2_usd']:.4f}",
100        "MAPE_%": f"{m['mape_percent']:.1f}",
101        "fit_s": f"{result['fit_seconds']:.1f}",
102        "predict_s": f"{result['predict_seconds']:.1f}",
103    }
104
105
106 def table_row(task: Task, result: dict[str, Any]) -> dict[str, str]:
107     return regression_row(result) if task.kind == "regression" else classification_row(result)
108
109
110 def print_table(rows: list[dict[str, str]]) -> None:
111     if not rows:
112         return
113     columns = list(rows[0])
114     widths = {c: max(len(c), *(len(str(r[c])) for r in rows)) for c in columns}
115     line = " ".join(c.ljust(widths[c]) for c in columns)
116     print(line)
117     print("-" * len(line))
118     for row in rows:
119         print(" ".join(str(row[c]).ljust(widths[c]) for c in columns))
120
121
122 def save_results(name: str, payload: dict[str, Any]) -> Path:
123     REPORTS.mkdir(parents=True, exist_ok=True)
124     path = REPORTS / f"{name}_results.json"
125     path.write_text(json.dumps(payload, indent=2, ensure_ascii=False), encoding="utf-8")
126     return path

```

ml/metrics.py

52 dòng

```

1     """Chỉ số đánh giá cho bài toán phân loại và hồi quy.
2
3     Dùng scikit-learn cho phần tính toán, lớp bọc ở đây chỉ gom kết quả về một dict
4     phẳng để bảng so sánh và tệp JSON kết quả dùng chung một tên khoá.
5     """
6
7     from __future__ import annotations
8
9     import numpy as np
10    from sklearn.metrics import (
11        accuracy_score,
12        f1_score,
13        mean_absolute_error,
14        mean_squared_error,
15        precision_score,
16        r2_score,
17        recall_score,
18        roc_auc_score,
19    )
20
21
22    def classification_metrics(
23        y_true, y_pred, y_score=None, *, average: str = "binary"
24    ) -> dict[str, float | None]:
25        out: dict[str, float | None] = {
26            "accuracy": float(accuracy_score(y_true, y_pred)),
27            "precision": float(precision_score(y_true, y_pred, average=average, zero_division=0)),
28            "recall": float(recall_score(y_true, y_pred, average=average, zero_division=0)),

```

```

29         "f1": float(f1_score(y_true, y_pred, average=average, zero_division=0)),
30         "f1_macro": float(f1_score(y_true, y_pred, average="macro", zero_division=0)),
31         "f1_weighted": float(f1_score(y_true, y_pred, average="weighted", zero_division=0)),
32         "roc_auc": None,
33     }
34     if y_score is not None and average == "binary":
35         out["roc_auc"] = float(roc_auc_score(y_true, y_score))
36     return out
37
38
39 def regression_metrics(y_true_log, y_pred_log) -> dict[str, float]:
40     """Đo trên cả hai không gian: log(price) để so mô hình, USD để đọc nghiệp vụ."""
41     true_usd = np.exp(y_true_log)
42     pred_usd = np.exp(y_pred_log)
43     return {
44         "mae_log": float(mean_absolute_error(y_true_log, y_pred_log)),
45         "rmse_log": float(np.sqrt(mean_squared_error(y_true_log, y_pred_log))),
46         "r2_log": float(r2_score(y_true_log, y_pred_log)),
47         "mae_usd": float(mean_absolute_error(true_usd, pred_usd)),
48         "rmse_usd": float(np.sqrt(mean_squared_error(true_usd, pred_usd))),
49         "r2_usd": float(r2_score(true_usd, pred_usd)),
50         "mape_percent": float(np.mean(np.abs(pred_usd - true_usd) / true_usd) * 100.0),
51     }

```

ml/split.py

33 dòng

```

1     """Chia 70/15/15 dùng chung cho cả ba bài toán.
2
3     Công thức giữ nguyên như quy ước đã dùng ở notebook 01-03: cắt 15% làm test
4     trước, phần còn lại cắt tiếp 15/85 làm validation. Cùng một seed và cùng một
5     thứ tự cắt thì cùng một dataset luôn cho ra cùng một tập chỉ số.
6     """
7
8     from __future__ import annotations
9
10    from typing import Any
11
12    from sklearn.model_selection import train_test_split
13
14    RANDOM_SEED = 42
15    TEST_SHARE = 0.15
16    VAL_SHARE_OF_REST = TEST_SHARE / (1.0 - TEST_SHARE)
17
18
19    def split_70_15_15(
20        X: Any, y: Any, *, stratify: bool = True, seed: int = RANDOM_SEED
21    ) -> tuple[Any, Any, Any, Any, Any, Any]:
22        X_rest, X_test, y_rest, y_test = train_test_split(
23            X, y, test_size=TEST_SHARE, random_state=seed, stratify=y if stratify else None
24        )
25        X_train, X_val, y_train, y_val = train_test_split(
26            X_rest,
27            y_rest,
28            test_size=VAL_SHARE_OF_REST,
29            random_state=seed,
30            stratify=y_rest if stratify else None,
31        )
32        return X_train, X_val, X_test, y_train, y_val, y_test

```

reports/build/content03.py

591 dòng

```

1     """Sinh phân thân báo cáo Assignment 03 từ reports/deeplearning_results.json.
2
3     Mọi con số trong báo cáo đều đọc từ tệp kết quả mà notebook 04 ghi ra, không gõ tay,
4     nên báo cáo và notebook không thể lệch nhau.

```



```

73     không dùng TensorFlow, PyTorch, Keras hay scikit-learn. Bốn kiến trúc được huấn luyện
74     trên cùng một tập dữ liệu, cùng một seed khởi tạo và cùng một quy trình đo, nên khác
75     biệt giữa chúng chỉ đến từ kiến trúc. Mô hình tốt nhất theo ROC-AUC là {best["key"]}
76     {best["label"]} với {_n(best["roc_auc_test"])}.</p>
77     <p>Bốn chương sau mở rộng phạm vi ra cả ba category của dự án, để mỗi category có đủ ba
78     mô hình machine learning và một mô hình deep learning. Hai mô hình ML được bổ sung là
79     KNN và AdaBoost, lấp hai họ thuật toán chưa từng xuất hiện trong dự án. Hai mạng nơ-ron
80     được bổ sung, cũng viết tay bằng NumPy, phủ bài toán hồi quy giá nhà và bài toán văn bản
81     e-commerce, trong đó ma trận nhúng từ được huấn luyện bằng lan truyền ngược thay vì để
82     ngẫu nhiên cố định.</p>
83 </div>
84 </section>
85
86 <section class="toc">
87     <h2 id="muc-luc" class="nonum no-break">Mục lục</h2>
88     <ol><!--TOC--></ol>
89 </section>
90 ""
91
92
93 def overview(data: dict) -> str:
94     models = data["models"]
95     best = max(models, key=lambda m: m["roc_auc_test"])
96     worst = min(models, key=lambda m: m["roc_auc_test"])
97     spread = best["roc_auc_test"] - worst["roc_auc_test"]
98
99     rows = "".join(
100         f'<tr><td class="a1">{m["key"]} {m["label"]}</td>'
101         f'<td class="mono">{m["shape"]}</td>'
102         f'<td>{<math>\text{ReLU}</math> if m["hidden_activation"] == "relu" else "tuyến tính"}</td>'
103         f'<td class="num">{m["n_params"]}</td>'
104         f'<td>{MODEL_NOTES[m["key"]].split(".")[0]}</td></tr>'
105         for m in models
106     )
107
108     return f""
109 <h2 id="m1">1. Bài toán và phạm vi</h2>
110
111 <p>Deep Learning là phép hợp thành của các hàm có tham số. Thay vì học một ánh xạ trực
112 tiếp từ đầu vào sang nhãn, mô hình dựng nhiều phép biến đổi nối tiếp rồi học tham số của
113 chúng.</p>
114
115 <div class="formula">ŷ = f<sub>3</sub>(f<sub>2</sub>(f<sub>1</sub>(X))) = σ(ReLU(ReLU(XW<sub>1</sub> + b<sub>1</sub>)W<sub>2</sub> + b<sub>2</sub>)W<sub>3</sub> + b<sub>3</sub>)</div>
116
117 <p>Bài toán là phân loại nhị phân nguy cơ tiểu đường từ bộ khảo sát BRFSS 2015. Đầu vào
118 là tám chỉ số sức khỏe, đầu ra là một xác suất. Ràng buộc của bài là không dùng framework
119 học sâu nào: mọi phép toán của mạng, kể cả đạo hàm, đều viết bằng NumPy.</p>
120
121 <div class="chain">
122     <div class="st on"><k>01</k><b>Biểu diễn</b><i>8 đặc trưng, chuẩn hoá z</i></div>
123     <div class="st on"><k>02</k><b>Mạng NumPy</b><i>forward, loss, backward</i></div>
124     <div class="st on"><k>03</k><b>Huấn luyện</b><i>gradient descent lô nhỏ</i></div>
125     <div class="st on"><k>04</k><b>Đánh giá</b><i>4 mô hình, 3 ngưỡng</i></div>
126 </div>
127
128 <div class="tcap" data-key="t1">Bốn kiến trúc được so sánh</div>
129 <table>
130     <thead><tr>
131         <th style="width:16%">Mô hình</th>
132         <th style="width:18%">Kiến trúc</th>
133         <th style="width:13%">Kích hoạt lớp ẩn</th>
134         <th class="num" style="width:11%">Tham số</th>
135         <th>Mục đích</th>
136     </tr></thead>
137     <tbody>{rows}</tbody>
138 </table>
139 <p class="tnote">Cả bốn dùng chung seed khởi tạo {data["init_seed"]} và seed xáo trộn

```

```

140 {data["shuffle_seed"]}, cùng học suất {data["learning_rate"]}, cùng {data["epochs"]} epoch
141 và cùng lô {data["batch_size"]} dòng.</p>
142
143 <p>Khoảng cách giữa mô hình tốt nhất và kém nhất theo ROC-AUC chỉ là {n(spread)}. Đây là
144 kết quả đáng chú ý nhất của báo cáo và sẽ được giải thích ở mục 8: với tám đặc trưng bằng
145 đã chuẩn hoá, phần lớn tín hiệu là tuyến tính, nên chiều sâu và bề rộng mang lại rất ít.</p>
146 """
147
148
149 def representation(data: dict) -> str:
150     ranking = data["feature_ranking"]
151     chosen = data["features"]
152     rows = "".join(
153         f'<tr><td class="num">{i}</td><td class="a1">{name}</td>'
154         f'<td class="num">{ranking[name]:.4f}</td></tr>'
155         for i, name in enumerate(chosen, 1)
156     )
157     dropped = [k for k in ranking if k not in chosen][:4]
158     dropped_text = ", ".join(f"{k} ({ranking[k]:.4f})" for k in dropped)
159
160     return f"""
161 <h2 id="m2">2. Biểu diễn dữ liệu</h2>
162
163 <p>Bài giảng dùng bộ Pima với đúng tám biến đầu vào. Bộ BRFSS ở đây có 21 biến, nên phải
164 chọn ra tám. Tiêu chí chọn là trị tuyệt đối hệ số tương quan với nhãn nhị phân, xếp hạng
165 đầy đủ 21 biến rồi lấy tám biến đầu. Cách chọn này đơn giản và kiểm chứng được, nhưng chỉ
166 bắt được quan hệ tuyến tính đơn biến.</p>
167
168 <div class="tcap" data-key="t5">Tám đặc trưng được chọn</div>
169 <table>
170     <thead><tr>
171         <th class="num" style="width:8%">Hạng</th>
172         <th style="width:34%">Đặc trưng</th>
173         <th class="num" style="width:20%">|tương quan|</th>
174     </tr></thead>
175     <tbody>{rows}</tbody>
176 </table>
177 <p class="tnote">Bốn biến bị loại ngay sau ngưỡng: {dropped_text}</p>
178
179 <p>Nhãn nhị phân giữ đúng quy ước của các ứng dụng trước, y bằng 1 khi
180 <code>Diabetes_012 &gt; 0</code>, tức gộp tiền tiểu đường và tiểu đường vào lớp dương.
181 Sau khi bỏ {data["rows_raw"]} - data["rows_after_dedup"],} dòng trùng hoàn toàn, còn
182 {data["rows_after_dedup"]},} dòng, chia {data["n_train"]},} train và {data["n_test"]},}
183 test theo tỉ lệ 80 trên 20 với seed {data["split_seed"]}</p>
184
185 <p>Tỉ lệ lớp dương là {n(data["positive_rate_train"])} trên train và
186 {n(data["positive_rate_test"])} trên test. Đây là bài toán lệch lớp, nên accuracy một
187 mình không nói lên điều gì: đoán toàn lớp âm đã cho accuracy khoảng
188 {n(1 - data["positive_rate_test"], 3)}</p>
189
190 <div class="formula">x' = (x &minus; μ<sub>train</sub>) / σ<sub>train</sub></div>
191
192 <p>Chuẩn hoá dùng trung bình và độ lệch chuẩn tính riêng trên tập train rồi áp lại cho
193 test. Nếu tính trên toàn bộ dữ liệu thì thông tin của tập test đã rò sang lúc huấn luyện,
194 và mọi con số đánh giá sau đó đều lạc quan hơn thực tế.</p>
195
196 <figure>
197     
198     <figcaption data-key="f1">Phân bố ba lớp gốc, nhãn nhị phân dùng để huấn luyện, và thứ
199     hạng tương quan của tám đặc trưng được chọn.</figcaption>
200 </figure>
201
202 <div class="oim"><p><k>Hình <r>f1</r></k><b>Observation:</b> lớp tiền tiểu đường rất mỏng
203 so với hai lớp còn lại, và sau khi gộp nhị phân thì lớp dương vẫn chỉ chiếm khoảng một
204 phần sáu. <b>Interpretation:</b> tín hiệu dương hiếm, mạng sẽ có xu hướng nghiêng về lớp
205 âm để hạ nhanh hàm mất mát. <b>ML implication:</b> phải đọc recall và ROC-AUC thay vì
206 accuracy, và ngưỡng quyết định 0.5 không nhất thiết là ngưỡng nên dùng khi triển khai.</p></div>
207 """
208

```

```

210 def network_math() -> str:
211     return """
212 <h2 id="m3">3. Mạng nơ-ron viết tay</h2>
213
214 <p>Toàn bộ mạng gồm bốn phép toán, không có phép nào cần thư viện học sâu.</p>
215
216 <h3 id="m3-1">3.1 Lan truyền xuôi</h3>
217
218 <div class="formula">
$$Z_{k+1} = W_k + b_k,$$
  


$$H_k = g(Z_k)</div>
220
221 <p>Hàm g là ReLU ở lớp ẩn và sigmoid ở lớp ra. ReLU đưa phi tuyến vào mạng: thiếu nó,  

222 mọi phép xếp chồng tuyến tính vẫn quy về một phép tuyến tính duy nhất. Sigmoid ép đầu ra  

223 về khoảng mở (0, 1) để đọc như một xác suất.</p>
224
225 <h3 id="m3-2">3.2 Hàm mất mát</h3>
226
227 <div class="formula">
$$L = \frac{1}{N} \sum [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]</div>
229
230 <p>Xác suất được cắt về khoảng [1e-8, 1 - 1e-8] trước khi lấy log, nếu không một  

231 dự đoán tự tin tuyệt đối và sai sẽ cho log(0) và làm hỏng cả vòng huấn luyện.</p>
232
233 <h3 id="m3-3">3.3 Lan truyền ngược</h3>
234
235 <p>Ghép sigmoid với binary cross-entropy cho một kết quả rất gọn ở lớp cuối. Đạo hàm của  

236 hàm mất mát theo đầu vào của sigmoid không còn chứa đạo hàm sigmoid nữa:</p>
237
238 <div class="formula">
$$\partial L / \partial Z_L = (\hat{y} - y) / N</div>
239
240 <p>Từ đó gradient lan ngược bằng quy tắc chuỗi, mỗi lớp chỉ cần hai phép nhân ma trận và  

241 một phép nhân từng phần tử với đạo hàm của hàm kích hoạt:</p>
242
243 <div class="formula">
$$\delta W_k = H_{k+1}^T \delta_k,$$
  


$$\delta_{k+1} = (\delta_k W_k + g'(Z_{k+1})) \odot g'(Z_{k+1})</div>
245
246 <p>Lan truyền ngược không phải một phép thuật riêng của học sâu. Nó chỉ là quy tắc chuỗi  

247 áp cho một hàm hợp thành, thực hiện theo thứ tự ngược để dùng lại kết quả trung gian mà  

248 lan truyền xuôi đã lưu.</p>
249
250 <h3 id="m3-4">3.4 Cập nhật tham số</h3>
251
252 <div class="formula">
$$W \leftarrow W - \eta \partial L / \partial W</div>
253
254 <p>Hàm cập nhật trả về bộ tham số mới thay vì sửa tại chỗ, nên mỗi bước huấn luyện là một  

255 phép biến đổi rõ ràng từ trạng thái cũ sang trạng thái mới. Cách viết này tốn thêm một  

256 lần cấp phát mỗi bước nhưng loại hẳn một lớp lỗi khó tìm khi gỡ rối gradient.</p>
257
258 <div class="callout"><span class="lab">Khác biệt so với công thức trong tài liệu</span>  

259 <p>Tài liệu dùng gradient descent toàn tập, học suất 0.01, 1000 epoch, trên bộ Pima 768  

260 dòng. Tập train ở đây lớn hơn hai trăm lần, nên 1000 epoch toàn tập chỉ tương đương 1000  

261 lần cập nhật tham số và mạng chưa kịp rời khỏi điểm khởi tạo: kết quả đo được là mô hình  

262 đoán toàn lớp âm, recall bằng 0. Vì vậy vòng lặp chuyển sang gradient descent theo lô nhỏ  

263 512 dòng. Công thức cập nhật không đổi, chỉ khác ở chỗ gradient tính trên lô thay vì trên  

264 toàn tập. Mục 8 giữ nguyên công thức gốc để đối chiếu và đưa số đo cụ thể.</p></div>
265 ""
266
267
268 def model_section(data: dict, index: int, model: dict) -> str:
269     key = model["key"]
270     t05 = model["test_by_threshold"][f"0.5"]
271     train = model["train"]
272     gap = train["accuracy"] - t05["accuracy"]
273
274     threshold_rows = "".join(
275         f'<tr><td class="a1">{th}</td>'
276         f'<td class="num">{n(row["accuracy"])}</td>'$$$$$$$$$$

```

```

277         f'<td class="num">{n(row["precision"])}</td>'
278         f'<td class="num">{n(row["recall"])}</td>'
279         f'<td class="num">{n(row["f1"])}</td>'
280         f'<td class="num">{row["tp"]},}</td>'
281         f'<td class="num">{row["fn"]},}</td>'
282         f'<td class="num">{row["fp"]},}</td>'
283         f'<td class="num">{row["tn"]},}</td></tr>'
284         for th, row in model["test_by_threshold"].items()
285     )
286
287     hidden = ", ".join(f"H{i}: ({s[0]:}, {s[1]:})" for i, s in enumerate(model["hidden_shapes"], 1))
288
289     return f"""
290 <h2 id="m{index}">{index}. Mô hình {key} {model["label"]}</h2>
291
292 <p>{MODEL_NOTES[key]}</p>
293
294 <div class="kpis">
295     <div class="kpi"><b>{n(t05["accuracy"])}</b><span>Accuracy &mdot; test</span></div>
296     <div class="kpi"><b>{n(t05["f1"])}</b><span>F1 &mdot; ngưỡng 0.5</span></div>
297     <div class="kpi"><b>{n(model["roc_auc_test"])}</b><span>ROC-AUC &mdot; test</span></div>
298     <div class="kpi"><b>{model["seconds"]:.0f}s</b><span>Thời gian huấn luyện</span></div>
299 </div>
300
301 <div class="tcap" data-key="t{index}a">Cấu hình và quá trình huấn luyện của {key}</div>
302 <table>
303     <thead><tr><th style="width:34%">Hạng mục</th><th>Giá trị</th></tr></thead>
304     <tbody>
305         <tr><td class="a1">Kiến trúc</td><td class="mono">{model["shape"]}</td></tr>
306         <tr><td class="a1">Kích hoạt lớp ẩn</td><td>
307             {"ReLU" if model["hidden_activation"] == "relu" else "tuyến tính (hàm đồng nhất)"}</td></tr>
308         <tr><td class="a1">Số tham số học được</td><td class="num">{model["n_params"]}</td></tr>
309         <tr><td class="a1">Kích thước biểu diễn trung gian</td><td class="mono">{hidden}</td></tr>
310         <tr><td class="a1">Học suất, epoch, lô</td><td class="num">
311             {model["learning_rate"]} &mdot; {model["epochs"]} &mdot; {model["batch_size"]} </td></tr>
312         <tr><td class="a1">Số lần cập nhật tham số</td><td class="num">{model["updates"]:,}</td></tr>
313         <tr><td class="a1">Loss train đầu và cuối</td><td class="num">
314             {n(model["loss_first"])} &rarr; {n(model["loss_final"])}</td></tr>
315         <tr><td class="a1">Loss trên tập test</td><td class="num">{n(model["loss_test_end"])}</td></tr>
316         <tr><td class="a1">Thời gian huấn luyện</td><td class="num">{model["seconds"]:.1f} giây</td></tr>
317     </tbody>
318 </table>
319
320 <p class="tnote">Chênh lệch accuracy giữa train và test là {gap:+.4f}, và loss test
321 ({n(model["loss_test_end"])} so với loss train ({n(model["loss_final"])})) lệch
322 {model["loss_test_end"] - model["loss_final"]:+.4f}. Khoảng cách hẹp như vậy cho thấy mô
323 hình chưa quá khớp, phần thiếu nằm ở sức biểu diễn chứ không ở việc học thuộc dữ liệu.</p>
324
325 <div class="tcap" data-key="t{index}b">Kết quả của {key} trên tập test theo ba ngưỡng quyết định</div>
326 <table>
327     <thead><tr>
328         <th style="width:12%">Ngưỡng</th>
329         <th class="num">Accuracy</th>
330         <th class="num">Precision</th>
331         <th class="num">Recall</th>
332         <th class="num">F1</th>
333         <th class="num">TP</th>
334         <th class="num">FN</th>
335         <th class="num">FP</th>
336         <th class="num">TN</th>
337     </tr></thead>
338     <tbody>{threshold_rows}</tbody>
339 </table>
340
341 <p class="tnote">{threshold_reading(model)}</p>
342
343 <p>{model_reading(data, model)}</p>
344 """
345
346 def _threshold_reading(model: dict) -> str:

```

```

343 low = model["test_by_threshold"]["0.3"]
344 mid = model["test_by_threshold"]["0.5"]
345 high = model["test_by_threshold"]["0.7"]
346 return (
347     f"Hạ ngưỡng từ 0.5 xuống 0.3 nâng recall từ {n(mid['recall'])} lên "
348     f"{n(low['recall'])} và đổi lại precision giảm từ {n(mid['precision'])} xuống "
349     f"{n(low['precision'])}, tức bắt thêm {low['tp'] - mid['tp']:,} ca dương thật "
350     f"nhưng báo động nhầm thêm {low['fp'] - mid['fp']:,} ca. Nâng lên 0.7 đẩy recall "
351     f"xuống {n(high['recall'])}. Với bài sàng lọc bệnh, bỏ sót tổn kém hơn báo động "
352     f"nhầm, nên ngưỡng thấp là lựa chọn hợp lý hơn 0.5."
353 )
354
355
356 def _model_reading(data: dict, model: dict) -> str:
357     baseline = data["models"][0]
358     if model["key"] == "M1":
359         return (
360             "Đây là mốc tham chiếu. Hai lớp ẩn ReLU được hai tầng biểu diễn, nhưng "
361             "recall tại ngưỡng 0.5 chỉ ở mức "
362             f"{n(model['test_by_threshold']['0.5']['recall'])}: mạng hạ hàm mất mát nhanh "
363             "nhất bằng cách nghiêng về lớp âm chiếm đa số, đúng như dự đoán từ phân bố lớp."
364         )
365     delta_auc = model["roc_auc_test"] - baseline["roc_auc_test"]
366     delta_f1 = model["test_by_threshold"]["0.5"]["f1"] - baseline["test_by_threshold"]["0.5"]["f1"]
367     direction = "cao hơn" if delta_auc > 0 else "thấp hơn"
368     if model["key"] == "M2":
369         return (
370             f"So với M1, ROC-AUC {direction} {abs(delta_auc):.4f} và F1 lệch {delta_f1:+.4f}, "
371             f"trong khi số tham số tăng từ {baseline['n_params']} lên {model['n_params']} và "
372             f"thời gian huấn luyện tăng từ {baseline['seconds']:.0f}s lên {model['seconds']:.0f}s. "
373             "Thêm sức chứa không mua được chất lượng ở bài này, vì giới hạn nằm ở lượng "
374             "thông tin trong tám đặc trưng chủ không ở khả năng biểu diễn của mạng."
375         )
376     if model["key"] == "M3":
377         return (
378             f"Bỏ hẳn một lớp ẩn làm ROC-AUC {direction} {abs(delta_auc):.4f} so với M1, với "
379             f"chỉ {model['n_params']} tham số, bằng khoảng {model['n_params'] / baseline['n_params']:.0%} "
380             f"của M1, và nhanh hơn {baseline['seconds'] / max(model['seconds'], 0.01):.1f} lần. "
381             "Nếu phải chọn một mô hình để triển khai thì đây là ứng viên hợp lý nhất: "
382             "chất lượng ngang ngửa với chi phí thấp nhất."
383         )
384     return (
385         f"Bỏ ReLU làm ROC-AUC {direction} {abs(delta_auc):.4f} và F1 lệch {delta_f1:+.4f} so "
386         f"với M1. Mức giảm có thật nhưng nhỏ, và đó mới là điều đáng nói: ba phép biến đổi "
387         f"tuyến tính xếp chồng chỉ tương đương một phép tuyến tính, nên M4 thực chất là hồi "
388         f"quy logistic đội lốt mạng ba lớp. Việc nó gần bằng ba mô hình kia là bằng chứng "
389         f"trực tiếp rằng phần lớn tín hiệu trong tám đặc trưng này là tuyến tính."
390     )
391
392
393 def comparison(data: dict) -> str:
394     models = data["models"]
395     best = max(models, key=lambda m: m["roc_auc_test"])
396     cheapest = min(models, key=lambda m: m["n_params"])
397
398     rows = "".join(
399         f'<tr class="{hi}" if m["key"] == best["key"] else "">'
400         f'<td class="ai">{m["key"]}</td>'
401         f'<td class="num">{m["n_params"]}</td>'
402         f'<td class="num">{n(m["loss_final"])}</td>'
403         f'<td class="num">{n(m["loss_test_end"])}</td>'
404         f'<td class="num">{n(m["test_by_threshold"]["0.5"]["accuracy"])}</td>'
405         f'<td class="num">{n(m["test_by_threshold"]["0.5"]["precision"])}</td>'
406         f'<td class="num">{n(m["test_by_threshold"]["0.5"]["recall"])}</td>'
407         f'<td class="num">{n(m["test_by_threshold"]["0.5"]["f1"])}</td>'
408         f'<td class="num">{n(m["roc_auc_test"])}</td>'
409         f'<td class="num">{m["seconds"]:.1f}</td></tr>'
410         for m in models

```

```

411     )
412
413     lr_rows = "".join(
414         f'<tr><td class="a1">{row["learning_rate"]}</td>'
415         f'<td class="num">{row["updates"]},</td>'
416         f'<td class="num">{n(row["loss_first"])}</td>'
417         f'<td class="num">{n(row["loss_final"])}</td>'
418         f'<td class="num">{n(row["accuracy"])}</td>'
419         f'<td class="num">{n(row["recall"])}</td>'
420         f'<td class="num">{n(row["f1"])}</td>'
421         f'<td class="num">{n(row["roc_auc_test"])}</td>'
422         f'<td class="num">{row["seconds"]:.0f}</td></tr>'
423         for row in data["learning_rate_study"]
424     )
425     m1 = models[0]
426     lr_rows += (
427         f'<tr class="total"><td class="a1">{m1["learning_rate"]} (lô {m1["batch_size"]})</td>'
428         f'<td class="num">{m1["updates"]},</td>'
429         f'<td class="num">{n(m1["loss_first"])}</td>'
430         f'<td class="num">{n(m1["loss_final"])}</td>'
431         f'<td class="num">{n(m1["test_by_threshold"])[0.5]["accuracy"]}</td>'
432         f'<td class="num">{n(m1["test_by_threshold"])[0.5]["recall"]}</td>'
433         f'<td class="num">{n(m1["test_by_threshold"])[0.5]["f1"]}</td>'
434         f'<td class="num">{n(m1["roc_auc_test"])}</td>'
435         f'<td class="num">{m1["seconds"]:.0f}</td></tr>'
436     )
437
438     best_full = max(data["learning_rate_study"], key=lambda r: r["roc_auc_test"])
439
440     return f"""
441 <h2 id="m8">8. So sánh bốn mô hình</h2>
442
443 <div class="tcap" data-key="t10">Bảng tổng hợp bốn mô hình trên tập test, ngưỡng 0.5</div>
444 <table>
445     <thead><tr>
446         <th style="width:15%">Mô hình</th>
447         <th class="num">Tham số</th>
448         <th class="num">Loss train</th>
449         <th class="num">Loss test</th>
450         <th class="num">Accuracy</th>
451         <th class="num">Precision</th>
452         <th class="num">Recall</th>
453         <th class="num">F1</th>
454         <th class="num">ROC-AUC</th>
455         <th class="num">Giây</th>
456     </tr></thead>
457     <tbody>{rows}</tbody>
458 </table>
459 <p class="tnote">Đòng in đậm là mô hình tốt nhất theo ROC-AUC. Chỉ số này được chọn làm
460 tiêu chí chính vì nó không phụ thuộc ngưỡng quyết định, trong khi accuracy và F1 tại
461 ngưỡng 0.5 đều bị lớp âm chiếm đa số kéo lệch.</p>
462
463 <figure>
464     
465     <figcaption data-key="f2">Đường mất mát trên tập train của bốn mô hình, toàn bộ quá
466     trình và phần phóng to từ epoch 10.</figcaption>
467 </figure>
468
469 <div class="oim"><p><k>Hình <r>f2</r></k><b>Observation:</b> cả bốn đường tụt rất nhanh
470 trong mười epoch đầu rồi gần như nằm ngang, và ba đường có ReLU nằm chồng lên nhau.
471 <b>Interpretation:</b> phần dễ học của bài toán được học xong sớm, phần còn lại không
472 nằm trong tầm với của tám đặc trưng này. <b>ML implication:</b> kéo dài huấn luyện hay
473 tăng sức chứa đều không đáng, muốn tiến thêm phải thêm đặc trưng chứ không phải thêm lớp.</p></div>
474
475 <figure>
476     
477     <figcaption data-key="f3">Đánh đổi giữa precision và recall khi đổi ngưỡng quyết định,
478     và so sánh ba chỉ số chính của bốn mô hình.</figcaption>
479 </figure>

```

```

480
481 <h3 id="m8-1">8.1 Học suất và công thức gốc của tài liệu</h3>
482
483 <p>Để kiểm chứng phần lý giải ở mục 3.4, kiến trúc M1 được chạy lại đúng theo công thức
484 trong tài liệu: gradient descent toàn tập, 1000 epoch, với ba mức học suất.</p>
485
486 <div class="tcap" data-key="t11">Gradient descent toàn tập theo công thức gốc, kiến trúc M1</div>
487 <table>
488   <thead><tr>
489     <th style="width:18%">Học suất</th>
490     <th class="num">Lần cập nhật</th>
491     <th class="num">Loss đầu</th>
492     <th class="num">Loss cuối</th>
493     <th class="num">Accuracy</th>
494     <th class="num">Recall</th>
495     <th class="num">F1</th>
496     <th class="num">ROC-AUC</th>
497     <th class="num">Giây</th>
498   </tr></thead>
499   <tbody>{lr_rows}</tbody>
500 </table>
501 <p class="tnote">Dòng cuối là M1 với lô nhỏ để đối chiếu. Ba cấu hình toàn tập đều chạy
502 1000 epoch nhưng chỉ có 1000 lần cập nhật tham số, trong khi cấu hình lô nhỏ có
503 {m1["updates"]:,} lần chỉ với {m1["epochs"]} epoch.</p>
504
505 <figure>
506   
507   <figcaption data-key="f4">Đường mất mát của gradient descent toàn tập ở ba mức học suất,
508   đặt cạnh mức mà cấu hình lô nhỏ đạt được.</figcaption>
509 </figure>
510
511 <div class="oim"><p><k>Hình <r>f4</r></k><b>Observation:</b> học suất 0.001 gần như nằm
512 ngang suốt 1000 epoch, 0.01 đi xuống chậm, 0.1 xuống nhanh nhất nhưng vẫn dừng ở
513 {_n(best_full["loss_final"])}, cao hơn mức {_n(m1["loss_final"])} mà lô nhỏ đạt được.
514 <b>Interpretation:</b> số lần cập nhật, chứ không phải số epoch, mới là thứ quyết định
515 tiến độ học. <b>ML implication:</b> khi đổi quy mô dữ liệu thì phải đổi cả cách chia lô,
516 giữ nguyên siêu tham số của bài giảng gốc sẽ cho một mô hình trông như đã huấn luyện
517 nhưng thực chất chưa học gì.</p></div>
518
519 <h3 id="m8-2">8.2 Điều bốn mô hình nói lên</h3>
520
521 <ul class="plain">
522   <li><b>Chiều sâu gần như không đổi được kết quả.</b> M3 chỉ có một lớp ẩn và
523   {cheapest["n_params"]} tham số nhưng ROC-AUC chỉ lệch
524   {abs(models[2]["roc_auc_test"] - models[0]["roc_auc_test"]):.4f} so với M1.</li>
525   <li><b>Bề rộng cũng vậy.</b> M2 nhiều hơn M1 {models[1]["n_params"] - models[0]["n_params"]}
526   tham số và tốn thêm {models[1]["seconds"] - models[0]["seconds"]:.0f} giây mà không đổi
527   được thứ hạng.</li>
528   <li><b>Phi tuyến có đóng góp, nhưng nhỏ.</b> M4 mất
529   {abs(models[3]["roc_auc_test"] - models[0]["roc_auc_test"]):.4f} ROC-AUC khi bỏ ReLU. Đây
530   là số đo của phần tín hiệu phi tuyến trong tám đặc trưng này.</li>
531   <li><b>Ngưỡng quyết định đáng giá hơn kiến trúc.</b> Hạ ngưỡng từ 0.5 xuống 0.3 đổi
532   recall của M1 nhiều hơn hẳn mọi khác biệt giữa bốn kiến trúc cộng lại.</li>
533 </ul>
534 ""
535
536
537 def conclusion(data: dict) -> str:
538     models = data["models"]
539     best = max(models, key=lambda m: m["roc_auc_test"])
540     ref = 1 - data["positive_rate_test"]
541
542     return f""
543 <h2 id="m13">13. Kết luận và giới hạn</h2>
544
545 <p>Một mạng nơ-ron ba lớp viết hoàn toàn bằng NumPy đã học được trên hơn hai trăm nghìn
546 dòng dữ liệu thật, chạy trên CPU trong vài chục giây mỗi mô hình. Điều framework tự động
547 hoá không phải bản chất toán học mà là công sức triển khai: đạo hàm tự động, quản lý
548 tensor, tối ưu hoá và tính toán trên GPU.</p>

```

```

549
550 <p>Mô hình tốt nhất theo ROC-AUC là {best["key"]} {best["label"]} với
551 {n(best["roc_auc_test"])}. Nhưng kết luận có ích hơn cho việc triển khai lại là M3: ít
552 tham số nhất, nhanh nhất, chất lượng gần như không kém.</p>
553
554 <div class="risk"><span class="lab">Giới hạn tự nhận</span>
555 <b>Một, accuracy ở đây dễ gây hiểu nhầm.</b> Đoán toàn lớp âm đã cho khoảng {n(ref, 4)},
556 nên mọi con số accuracy trong báo cáo phải đọc kèm recall và ROC-AUC.
557 <b>Hai, chọn đặc trưng bằng tương quan tuyến tính đơn biến</b> có thể đã loại mất biến chỉ
558 có ích khi kết hợp với biến khác, và điều đó tự nó đã thu hẹp phần phi tuyến còn lại cho
559 mạng khai thác.
560 <b>Ba, không có tập validation riêng.</b> Siêu tham số được chọn bằng suy luận về số lần
561 cập nhật chứ không bằng dò trên tập validation, nên các con số trên tập test là ước lượng
562 trung thực cho đúng cấu hình này chứ không phải cho cấu hình tốt nhất có thể.
563 </div>
564
565 <p>Hướng đi tiếp có ích nhất không phải mạng sâu hơn mà là thêm thông tin đầu vào: dùng
566 đủ 21 biến thay vì tám, thêm đặc trưng tương tác, hoặc cân bằng lại lớp bằng trọng số
567 trong hàm mất mát. Cả bốn mô hình ở đây đều đã chạm trần thông tin của tám cột hiện có.</p>
568 """
569
570
571 def build_content() -> str:
572     data = json.loads(RESULTS.read_text(encoding="utf-8"))
573     parts = [
574         cover(data),
575         overview(data),
576         representation(data),
577         network_math(),
578     ]
579     for offset, model in enumerate(data["models"]):
580         parts.append(model_section(data, 4 + offset, model))
581     parts.append(comparison(data))
582     parts.append(build_extra())
583     parts.append(conclusion(data))
584     return "\n".join(parts)
585
586
587 if __name__ == "__main__":
588     out = Path(__file__).resolve().parent / "content03.html"
589     out.write_text(build_content(), encoding="utf-8")
590     print(f"{out} ({out.stat().st_size / 1024:.0f} KB)")

```

reports/build/content03_extra.py

622 dòng

```

1      """Sinh các chương mô phỏng của báo cáo Assignment 03.
2
3      Bon chương này đọc trực tiếp bốn tệp kết quả mà notebook 05, 06, 07, 08 ghi ra:
4          reports/knn_results.json
5          reports/adaboost_results.json
6          reports/deeplearning_house_results.json
7          reports/deeplearning_ecommerce_results.json
8
9      Không có con số nào gõ tay, nên báo cáo không thể lech voi notebook.
10     """
11
12     from __future__ import annotations
13
14     import json
15     from pathlib import Path
16
17     ROOT = Path(__file__).resolve().parents[2]
18     REPORTS = ROOT / "reports"
19
20     TASK_ORDER = ("diabetes", "house_price", "ecommerce_satisfaction", "ecommerce_interest")
21     CATEGORY_OF = {
22         "diabetes": "Tiểu đường",

```

```

23     "house_price": "Giá nhà",
24     "ecommerce_satisfaction": "E-commerce",
25     "ecommerce_interest": "E-commerce",
26 }
27 SHORT_TITLE = {
28     "diabetes": "nguy cơ tiểu đường",
29     "house_price": "giá nhà",
30     "ecommerce_satisfaction": "mức hài lòng",
31     "ecommerce_interest": "nhóm sản phẩm",
32 }
33
34
35 def _load(name: str) -> dict:
36     return json.loads((REPORTS / f"{name}_results.json").read_text(encoding="utf-8"))
37
38
39 def _n(value, digits: int = 4) -> str:
40     return "-" if value is None else f"{value:.{digits}f}"
41
42
43 def _usd(value) -> str:
44     return f"{value:,.0f}"
45
46
47 def _by_task(payload: dict) -> dict[str, dict]:
48     return {block["task"]: block for block in payload["tasks"]}
49
50
51 def _new_model(block: dict) -> tuple[str, dict]:
52     """Ten va chi so test cua mo hinh moi (phan tu dau, khong phai moc so sanh)."""
53     name = block["model_names"][0]
54     return name, block["test_metrics"][name]
55
56
57 def _baseline(block: dict) -> tuple[str, dict]:
58     name = block["model_names"][1]
59     return name, block["test_metrics"][name]
60
61
62 # ----- #
63 # Chuong 9 - ba mo hinh ML cho moi category
64 # ----- #
65
66 def ml_models() -> str:
67     knn = _load("knn")
68     ada = _load("adaboost")
69     k_task = _by_task(knn)
70     a_task = _by_task(ada)
71
72     scope_rows = "".join(
73         f'<tr><td class="a1">{CATEGORY_OF[key]}</td>'
74         f'<td>{SHORT_TITLE[key]}</td>'
75         f'<td>{{"binary": "nhị phân", "regression": "hồi quy", "multiclass": "9 lớp"}[k_task[key]["kind"]]} '
76         f'<td class="num">{k_task[key]["n_train"]}</td>'
77         f'<td class="num">{k_task[key]["n_val"]}</td>'
78         f'<td class="num">{k_task[key]["n_test"]}</td>'
79         f'<td>{_baseline(k_task[key])[0].replace(' (mốc)', ' ')}</td></tr>'
80         for key in TASK_ORDER
81     )
82
83     clf_rows = []
84     for key in ("diabetes", "ecommerce_satisfaction", "ecommerce_interest"):
85         for source, block in (("KNN", k_task[key]), ("AdaBoost", a_task[key])):
86             name, metrics = _new_model(block)
87             clf_rows.append((key, source, name, metrics))
88             base_name, base_metrics = _baseline(k_task[key])
89             clf_rows.append((key, "Mốc", base_name.replace(" (mốc)", ""), base_metrics))

```

```

90     body = ""
91     for key in ("diabetes", "ecommerce_satisfaction", "ecommerce_interest"):
92         group = [row for row in clf_rows if row[0] == key]
93         best_f1 = max(row[3]["f1_macro"] for row in group)
94         for index, (_, source, name, metrics) in enumerate(group):
95             label = (
96                 f'<td class="a1" rowspan="3">{CATEGORY_OF[key]}<br>'
97                 f'<span class="small">{SHORT_TITLE[key]}</span></td>'
98                 if index == 0
99                 else ""
100             )
101             hi = ' class="hi"' if metrics["f1_macro"] == best_f1 else ""
102             body += (
103                 f'<tr{hi}>{label}<td>{source}</td><td>{name}</td>'
104                 f'<td class="num">{n(metrics["accuracy"])}</td>'
105                 f'<td class="num">{n(metrics["recall"])}</td>'
106                 f'<td class="num">{n(metrics["f1_macro"])}</td>'
107                 f'<td class="num">{n(metrics.get("roc_auc"))}</td></tr>'
108             )
109
110     house_k = k_task["house_price"]
111     house_a = a_task["house_price"]
112     knn_house = _new_model(house_k)[1]
113     ada_house = _new_model(house_a)[1]
114     base_house = _baseline(house_k)[1]
115     house_rows = "".join(
116         f'<tr{" class=\\"hi\\" if label == "Môc" else ""}><td>{label}</td><td>{name}</td>'
117         f'<td class="num">{n(m["r2_log"])}</td>'
118         f'<td class="num">{n(m["mae_usd"])}</td>'
119         f'<td class="num">{n(m["rmse_usd"])}</td>'
120         f'<td class="num">{n(m["mape_percent"]:.1f)}</td></tr>'
121         for label, name, m in (
122             ("KNN", _new_model(house_k)[0], knn_house),
123             ("AdaBoost", _new_model(house_a)[0], ada_house),
124             ("Môc", _baseline(house_k)[0].replace(" (môc)", ""), base_house),
125         )
126     )
127
128     dia_knn = _new_model(k_task["diabetes"])[1]
129     dia_ada = _new_model(a_task["diabetes"])[1]
130     dia_base = _baseline(k_task["diabetes"])[1]
131     int_ada = _new_model(a_task["ecommerce_interest"])[1]
132     int_base = _baseline(k_task["ecommerce_interest"])[1]
133
134     return f"""
135 <h2 id="m9">9. Ba mô hình machine learning cho mỗi category</h2>
136
137 <p>Ba ứng dụng của Assignment 02 đã có bảng so sánh riêng, nhưng mọi mô hình trong các bảng
138 đó đều thuộc một trong ba nhóm: tuyến tính, xác suất sinh, hoặc cây và tổ hợp cây. Hai
139 notebook bổ sung lập hai khoảng trống còn lại. <b>KNN</b> mang vào nhóm học dựa trên thực
140 thể, thứ chưa từng xuất hiện trong dự án. <b>AdaBoost</b> mang vào một cách hiện thực hoá
141 boosting khác với gradient boosting đang dùng, để tách bạch xem phần thắng của boosting đến
142 từ ý tưởng chung hay từ chi tiết cài đặt.</p>
143
144 <div class="risk"><span class="lab">Điều kiện đo</span>
145 Dữ liệu trong <code>data</code> đã được bổ sung sau khi notebook 01-03 chạy lần cuối.
146 Vì vậy mô hình môc <b>được huấn luyện lại</b> trên bộ dữ liệu hiện tại thay vì chép lại con
147 số của Assignment 02. Mọi số trong chương này đo trên cùng một lần chia 70/15/15 với
148 seed 42, nên chênh lệch giữa các dòng quy được về thuật toán. Ngược lại, chúng
149 <b>không</b> so trực tiếp được với bảng trong báo cáo Assignment 02.
150 </div>
151
152 <div class="tcap" data-key="t20">Phạm vi đo: ba category, bốn bài toán</div>
153 <table>
154     <thead><tr>
155         <th style="width:14%">Category</th><th style="width:18%">Bài toán</th>
156         <th style="width:11%">Kiểu</th>
157         <th class="num" style="width:11%">Train</th>

```

```

158     <th class="num" style="width:10%">Val</th>
159     <th class="num" style="width:10%">Test</th>
160     <th>Mô hình mốc</th>
161 </tr></thead>
162 <tbody>{scope_rows}</tbody>
163 </table>
164 <p class="tnote">Category e-commerce có hai target độc lập trên cùng một bảng dữ liệu, nên
165 được tính là hai bài toán. Cả bốn bài toán đều nhận đủ ba mô hình.</p>
166
167 <h3 id="m9-1">9.1 Ba bài toán phân loại</h3>
168
169 <div class="tcap" data-key="t21">Kết quả trên tập test của ba bài toán phân loại</div>
170 <table>
171     <thead><tr>
172         <th style="width:13%">Category</th><th style="width:11%">Nhóm</th><th>Mô hình</th>
173         <th class="num" style="width:11%">Accuracy</th>
174         <th class="num" style="width:10%">Recall</th>
175         <th class="num" style="width:11%">F1-macro</th>
176         <th class="num" style="width:11%">ROC-AUC</th>
177     </tr></thead>
178     <tbody>{body}</tbody>
179 </table>
180 <p class="tnote">Đồng in đậm là mô hình đạt F1-macro cao nhất trong từng bài toán. F1-macro
181 được dùng làm tiêu chí thay vì accuracy vì cả ba bài toán đều lệch lớp: đoán toàn lớp đa số
182 đã cho accuracy khá cao mà không học được gì.</p>
183
184 <p>Bài toán tiểu đường cho thấy rõ nhất khác biệt giữa hai hướng bổ sung. AdaBoost đạt
185 F1-macro {_n(dia_ada["f1_macro"])}, gần như chạm mốc Random Forest
186 {_n(dia_base["f1_macro"])}, vì cả hai đều là tổ hợp cây và cùng được cân bằng lớp. KNN chỉ
187 đạt {_n(dia_knn["f1_macro"])}, và toàn bộ khoảng cách nằm ở recall:
188 {_n(dia_knn["recall"])} so với {_n(dia_base["recall"])}. Lý do không nằm ở chất lượng cài
189 đặt mà ở bản chất thuật toán: KNN không có tham số nào để gán trọng số lớp, nên khi 83%
190 hàng xóm thuộc lớp âm thì lá phiếu nghiêng hẳn về lớp âm.</p>
191
192 <p>Bài toán nhóm sản phẩm là ngoại lệ duy nhất: AdaBoost đạt F1-macro
193 {_n(int_ada["f1_macro"])}, vượt mốc {_n(int_base["f1_macro"])}. Không nên đọc con số này
194 thành bằng chứng về chất lượng thuật toán. Cả hai mô hình đều quanh quẩn mức đoán mò trên
195 9 lớp, và khi không mô hình nào thật sự học được quy luật thì thứ hạng bị quyết định bởi
196 cách phân bổ dự đoán ra các lớp chứ không phải bởi năng lực học.</p>
197
198 <h3 id="m9-2">9.2 Bài toán hồi quy</h3>
199
200 <div class="tcap" data-key="t22">Kết quả trên tập test của bài toán giá nhà</div>
201 <table>
202     <thead><tr>
203         <th style="width:12%">Nhóm</th><th>Mô hình</th>
204         <th class="num" style="width:13%">R<sup>2</sup> (log)</th>
205         <th class="num" style="width:14%">MAE (USD)</th>
206         <th class="num" style="width:14%">RMSE (USD)</th>
207         <th class="num" style="width:11%">MAPE (%)</th>
208     </tr></thead>
209     <tbody>{house_rows}</tbody>
210 </table>
211
212 <p>Khoảng cách giữa AdaBoost ({_n(ada_house["r2_log"])} và HistGradientBoosting
213 ({_n(base_house["r2_log"])} trên cùng một biểu diễn 8 chiều là khoảng cách giữa hai cách
214 hiện thực hoá cùng một ý tưởng boosting. AdaBoost.R2 cập nhật trọng số theo sai số tuyệt
215 đối đã chuẩn hoá, tức chỉ biết mẫu nào sai nhiều hơn chứ không biết sai theo hướng nào;
216 gradient boosting học thẳng phần dư có dấu nên mỗi vòng sửa đúng chiều còn thiếu. Bảng quét
217 số vòng trong notebook 06 xác nhận điều đó: chất lượng đạt đỉnh ở vòng
218 {a_task["house_price"]["stage_selected"]} rồi đi xuống khi chạy thêm.</p>
219
220 <p>KNN đạt {_n(knn_house["r2_log"])}, sát AdaBoost hơn mọi bài toán khác. Giá nhà là bài
221 toán có tính địa phương cao: hai căn cùng thành phố, cùng diện tích, cùng số phòng thì giá
222 thật sự gần nhau, và đó đúng là giả thiết mà KNN đặt ra.</p>
223
224 <div class="callout"><span class="lab">Hai phát hiện về phương pháp, không phải về kết quả</span>
225 <p><b>Chuẩn hoá không phải thủ tục hình thức.</b></p> Biểu diễn <code>target_enc</code> của
226 Assignment 02 được thiết kế cho cây quyết định, vốn cắt theo từng chiều nên miễn nhiệm với

```

```

227 thang đo. Đưa KNN vào mới lộ ra rằng biểu diễn đó không dùng ngay được: tám chiều lệch nhau
228 hàng nghìn lần, phải ghép thêm một bước <code>StandardScaler</code> thì khoảng cách Euclid
229 mới có nghĩa.</p>
230 <p><b>Không được đặt <code>class_weight="balanced"</code> lên bộ học yếu của AdaBoost.</b>
231 AdaBoost tính tỉ lệ lỗi theo bộ trọng số của nó, trong khi <code>class_weight</code> khiến
232 cây tối ưu theo một bộ trọng số khác. Hai bộ lệch nhau đẩy tỉ lệ lỗi vòng đầu lên 0.3963 và
233 chạm ngưỡng dừng sau đúng 5 vòng, tức ensemble 300 vòng không bao giờ được dựng. Cách đúng
234 là truyền trọng số cân bằng lớp vào giá trị khởi tạo qua tham số <code>sample_weight</code>
235 của <code>fit</code>.</p>
236 </div>
237 ""
238
239
240 # ----- #
241 # Chuong 10 - deep learning cho gia nha
242 # ----- #
243 def dl_house() -> str:
244     data = _load("deeplearning_house")
245     models = data["models"]
246     best = next(m for m in models if m["key"] == data["best_model"])
247     dl = data["test_metrics_dl"]
248     baseline = data["test_metrics_ml_baseline"]
249     no_relu = next(m for m in models if m["hidden_activation"] == "linear")
250
251     rows = "".join(
252         f'<tr{" class=\"hi\" if m["key"] == data["best_model"] else \""}>'
253         f'<td class="ai">{m["key"]} {m["label"]}</td>'
254         f'<td class="mono">{m["shape"]}</td>'
255         f'<td>{"ReLU" if m["hidden_activation"] == "relu" else "tuyến tính"}</td>'
256         f'<td class="num">{m["n_params"]},</td>'
257         f'<td class="num">{m["loss_val_final"]:.5f}</td>'
258         f'<td class="num">{_n(m["validation"] ["r2_log"])}</td>'
259         f'<td class="num">{_usd(m["validation"] ["mae_usd"])}</td>'
260         f'<td class="num">{m["seconds"]:.0f}</td></tr>'
261         for m in models
262     )
263
264     return f""
265 <h2 id="m10">10. Deep learning cho category giá nhà</h2>
266
267 <p>Notebook 04 dựng mạng nơ-ron cho bài toán phân loại. Chương này đưa cùng bộ công cụ đó
268 sang bài toán hồi quy, để category giá nhà có mô hình deep learning riêng thay vì đi mượn
269 kết luận của category khác. Toàn bộ mạng vẫn viết bằng NumPy: lan truyền xuôi, hàm mất mát,
270 lan truyền ngược và gradient descent đều viết tay.</p>
271
272 <p>Chỉ hai thứ phải đổi khi chuyển từ phân loại sang hồi quy. Lớp cuối bỏ sigmoid, trả về
273 số thực bất kỳ. Hàm mất mát đổi từ binary cross entropy sang sai số bình phương trung bình.
274 Thứ ba thì <b>không</b> phải đổi, và đó mới là điểm đáng nói: hàm lan truyền ngược giữ
275 nguyên từng dòng.</p>
276
277 <div class="formula">∂L/∂z<sup>(L)</sup> = (ŷ - y) / n &nbsp;&nbsp;&nbsp; đúng cho cả
278 sigmoid + cross entropy lẫn đầu ra đồng nhất + sai số bình phương</div>
279
280 <p>Đây là tính chất của các cặp hàm liên kết chuẩn: đạo hàm của hàm kích hoạt triệt tiêu
281 gọn với mẫu số của hàm mất mát tương ứng. Nó cũng là lý do các thư viện deep learning tách
282 rời được phần mạng khỏi phần hàm mất mát.</p>
283
284 <div class="tcap" data-key="t23">Bốn kiến trúc hồi quy, đo trên tập validation</div>
285 <table>
286   <thead><tr>
287     <th style="width:16%">Mô hình</th><th style="width:16%">Kiến trúc</th>
288     <th style="width:11%">Kích hoạt</th>
289     <th class="num" style="width:10%">Tham số</th>
290     <th class="num" style="width:11%">Loss</th>
291     <th class="num" style="width:11%">R<sup>2</sup></th> <th>(log)</th>
292     <th class="num" style="width:13%">MAE (USD)</th>
293     <th class="num" style="width:8%">Giây</th>
294   </tr></thead>
295   <tbody>{rows}</tbody>

```

```

296 </table>
297 <p class="tnote">Cả bốn dùng chung seed khởi tạo {data["init_seed"]}, học suất
298 {data["learning_rate"]}, {data["epochs"]} epoch và lô {data["batch_size"]} dòng, nên chênh
299 lệch giữa chúng quy được về kiến trúc. Biền mục tiêu là log(giá) đã chuẩn hoá bằng trung
300 bình và độ lệch chuẩn <b>của tập train</b>.</p>
301
302 <figure>
303   
304   <figcaption data-key="f10">Mất mát trên tập train và tập validation của bốn kiến trúc hồi
305   quy. Ba kiến trúc có ReLU chồng lên nhau, kiến trúc bỏ ReLU đứng hẳn ở mức cao hơn.</figcaption>
306 </figure>
307
308 <p>Kiến trúc {no_relu["key"]} bỏ ReLU mất {abs(no_relu["validation"]["r2_log"] - best["validation"]
309 ["r2_log"]):.4f}
310 điểm R<sup>2</sup> dù có đúng số tham số như kiến trúc mốc. Ba phép biến đổi tuyến tính xếp
311 chồng vẫn chỉ là một phép tuyến tính, nên mạng suy biến thành hồi quy tuyến tính. Ngược
312 lại, nhân đôi bề rộng chỉ đổi được rất ít: phi tuyến mới là thứ tạo ra giá trị, không phải
313 số tham số.</p>
314 <div class="tcap" data-key="t24">Mạng nơ-ron so với mô hình ML tốt nhất, trên tập test</div>
315 <table>
316   <thead><tr>
317     <th>Mô hình</th>
318     <th class="num" style="width:14%">R<sup>2</sup> (log)</th>
319     <th class="num" style="width:15%">MAE (USD)</th>
320     <th class="num" style="width:15%">RMSE (USD)</th>
321     <th class="num" style="width:12%">MAPE (%)</th>
322   </tr></thead>
323   <tbody>
324     <tr><td>Mạng nơ-ron {best["key"]} ({best["shape"]})</td>
325       <td class="num">{_n(dl["r2_log"])}</td>
326       <td class="num">{_usd(dl["mae_usd"])}</td>
327       <td class="num">{_usd(dl["rmse_usd"])}</td>
328       <td class="num">{dl["mape_percent"]:.1f}</td></tr>
329     <tr class="hi"><td>{data["ml_baseline_name"]}</td>
330       <td class="num">{_n(baseline["r2_log"])}</td>
331       <td class="num">{_usd(baseline["mae_usd"])}</td>
332       <td class="num">{_usd(baseline["rmse_usd"])}</td>
333       <td class="num">{baseline["mape_percent"]:.1f}</td></tr>
334   </tbody>
335 </table>
336
337 <figure>
338   
339   <figcaption data-key="f11">Dự đoán so với giá thật, và phân bố sai số theo phân khúc giá.
340   Sai số loe rộng dần ở hai đầu và lệch xuống có hệ thống ở phân khúc cao.</figcaption>
341 </figure>
342
343 <p>Mạng nơ-ron không vượt được HistGradientBoosting trên tám chiều đặc trưng bảng, chênh
344 {abs(baseline["r2_log"] - dl["r2_log"]):.4f} điểm R<sup>2</sup> và
345 {_usd(dl["mae_usd"] - baseline["mae_usd"])} USD ở MAE. Kết quả này khớp với kết luận phổ
346 biến trong ngành và không có gì bất ngờ: khi đầu vào đã là một bảng số sạch với 8 cột, cây
347 quyết định vẫn là lựa chọn đúng. Khuôn mẫu sai số ở hình trên cũng không phải lỗi của thuật
348 toán mà là giới hạn của dữ liệu: tám đặc trưng hiện có không phân biệt được một căn ba
349 triệu đô với một căn tám trăm nghìn trong cùng thành phố, cùng diện tích.</p>
350 ""
351
352
353 # ----- #
354 # Chuong 11 - deep learning cho e-commerce
355 # ----- #
356 def dl_ecommerce() -> str:
357     data = _load("deeplearning_ecommerce")
358     models = {m["key"]: m for m in data["models"]}
359     best = models[data["best_model"]]
360     dl = data["test_metrics_dl"]
361     baseline = data["test_metrics_ml_baseline"]
362
363     rows = "".join(

```



```

431 <div class="kpi"><b>+0.1431</b><span>TF-IDF &middot; Assignment 02</span></div>
432 <div class="kpi"><b>{best["n_params"]:,}</b><span>Tham số của N2</span></div>
433 </div>
434
435 <p>Mức tăng nhờ có văn bản là {data["gain_text_f1_macro"]:.4f} F1-macro, gần như trùng
436 khớp với +0.1431 mà Assignment 02 đo được bằng TF-IDF trên bộ dữ liệu cũ. Hai cách biểu
437 diễn văn bản hoàn toàn khác nhau, hai bộ dữ liệu khác nhau, nhưng cùng cho một kết luận:
438 bình luận của khách chứa khoảng chừng ấy thông tin về mức hài lòng. Đó là thuộc tính của dữ
439 liệu, không phải của thuật toán.</p>
440
441 <p>Con số thứ hai mới là phần Assignment 02 chưa trả lời được. Chênh lệch
442 {data["gain_learned_embedding_f1_macro"]:.4f} F1-macro giữa N2 và N3 là phần đóng góp của
443 riêng việc <b>học</b> biểu diễn từ, tách bạch khỏi phần đóng góp của việc có văn bản. Hơn
444 một nửa giá trị mà văn bản mang lại chỉ xuất hiện khi biểu diễn của từ được huấn luyện cùng
445 bài toán.</p>
446
447 <p>Ma trận nhúng khởi tạo hoàn toàn ngẫu nhiên và không được cung cấp bất kỳ từ điển cảm
448 xúc nào. Sau {data["epochs"]} epoch, <code>bad</code> và <code>worst</code> là hàng xóm gần
449 nhất của nhau theo cosin, còn <code>good</code> kéo về <code>nice</code>. Cấu trúc ngữ
450 nghĩa đó không được lập trình vào đầu cả: nó xuất hiện vì hai từ có tác động giống nhau lên
451 mục tiêu thì gradient đẩy chúng về cùng một hướng. Xen giữa các từ hợp lý vẫn có những
452 token hiếm chỉ xuất hiện vài lần, hàng nhúng của chúng gần như vẫn là giá trị ngẫu nhiên
453 ban đầu. Đó là giới hạn cố hữu của việc học nhúng trên hai mươi nghìn mẫu.</p>
454
455 <h3 id="m11-2">11.2 So với mô hình machine learning tốt nhất</h3>
456
457 <div class="tcap" data-key="t26">Mạng nơ-ron so với mô hình ML tốt nhất, trên tập test</div>
458 <table>
459 <thead><tr>
460 <th>Mô hình</th>
461 <th class="num" style="width:11%">d vào</th>
462 <th class="num" style="width:12%">Accuracy</th>
463 <th class="num" style="width:11%">Recall</th>
464 <th class="num" style="width:11%">F1</th>
465 <th class="num" style="width:12%">F1-macro</th>
466 <th class="num" style="width:11%">ROC-AUC</th>
467 </tr></thead>
468 <tbody>
469 <tr class="hi"><td>Mạng nơ-ron {best["key"]} ({best["label"]})</td>
470 <td class="num">{best["d_in"]}</td>
471 <td class="num">{_n(dl["accuracy"])}</td>
472 <td class="num">{_n(dl["recall"])}</td>
473 <td class="num">{_n(dl["f1"])}</td>
474 <td class="num">{_n(dl["f1_macro"])}</td>
475 <td class="num">{_n(dl["roc_auc"])}</td></tr>
476 <tr><td>{data["ml_baseline_name"]}</td>
477 <td class="num">11,835</td>
478 <td class="num">{_n(baseline["accuracy"])}</td>
479 <td class="num">{_n(baseline["recall"])}</td>
480 <td class="num">{_n(baseline["f1"])}</td>
481 <td class="num">{_n(baseline["f1_macro"])}</td>
482 <td class="num">{_n(baseline["roc_auc"])}</td></tr>
483 </tbody>
484 </table>
485
486 <p>Đây là lần đầu tiên trong cả dự án một mô hình deep learning viết tay vượt mô hình
487 machine learning tốt nhất. Mốc ML vẫn nhỉnh hơn ở ROC-AUC
488 ({_n(baseline["roc_auc"])} so với {_n(dl["roc_auc"])}), tức nó xếp hạng các ca tốt hơn một
489 chút, nhưng ở ngưỡng quyết định 0.5 thì mạng phân loại chính xác hơn.</p>
490
491 <p>Chênh lệch không lớn, và điều đáng chú ý nằm ở chỗ khác: mạng đạt kết quả đó với
492 {best["d_in"]} chiều đầu vào trong khi hồi quy logistic cần 11,835 chiều TF-IDF. Mạng nén
493 toàn bộ từ vựng vào một ma trận {data["vocab_size"] + 1:,} × {data["d_embed"]} và học cách
494 nén đó từ chính bài toán, thay vì trải mỗi từ thành một chiều riêng.</p>
495
496 <div class="callout"><span class="lab">Đọc chương 10 và 11 cùng nhau</span>
497 <p>Trên tám chiều đặc trưng bằng của bài toán giá nhà, mạng nơ-ron thua
498 HistGradientBoosting một khoảng rộng. Trên dữ liệu có văn bản, chính bộ công cụ ấy lại vượt
499 mốc. Không phải deep learning mạnh hơn hay yếu hơn machine learning cổ điển, mà là nó có

```

```

500 chỗ dùng: khi đầu vào cần được <b>học cách biểu diễn</b>, chứ không phải khi đầu vào đã là
501 một bảng số sạch.</p>
502 </div>
503 """
504
505
506 # ----- #
507 # Chương 12 - bảng tổng hợp
508 # ----- #
509 def summary_table() -> str:
510     knn = _by_task(_load("knn"))
511     ada = _by_task(_load("adaboost"))
512     dia_dl = json.loads((REPORTS / "deeplearning_results.json").read_text(encoding="utf-8"))
513     house_dl = _load("deeplearning_house")
514     ecom_dl = _load("deeplearning_ecommerce")
515
516     dia_best = max(dia_dl["models"], key=lambda m: m["roc_auc_test"])
517     dia_t05 = dia_best["test_by_threshold"]["0.5"]
518
519     def clf_cell(metrics: dict) -> str:
520         return (
521             f'<td class="num">{n(metrics["f1_macro"])}</td>'
522             f'<td class="num">{n(metrics.get("roc_auc"))}</td>'
523         )
524
525     rows = ""
526     for key in ("diabetes", "ecommerce_satisfaction", "ecommerce_interest"):
527         base_name, base_metrics = _baseline(knn[key])
528         knn_name, knn_metrics = _new_model(knn[key])
529         ada_name, ada_metrics = _new_model(ada[key])
530         if key == "diabetes":
531             dl_label = f'{dia_best["key"]} {dia_best["label"]} <span class="small">(*)</span>'
532             dl_cells = (
533                 f'<td class="num">-</td>'
534                 f'<td class="num">{n(dia_best["roc_auc_test"])}</td>'
535             )
536         elif key == "ecommerce_satisfaction":
537             dl_label = f'N2 {m for m in ecom_dl["models"] if m["key"] == "N2"}[0]["label"]'
538             dl_cells = clf_cell(ecom_dl["test_metrics_dl"])
539         else:
540             dl_label = "chưa có"
541             dl_cells = '<td class="num">-</td><td class="num">-</td>'
542
543     rows += (
544         f'<tr><td class="a1" rowspan="4">{CATEGORY_OF[key]}<br>'
545         f'<span class="small">{SHORT_TITLE[key]}</span></td>'
546         f'<td>ML mốc</td><td>{base_name.replace(" (mốc)", "")}</td><td>{clf_cell(base_metrics)}</tr>'
547         f'<tr><td>ML bổ sung</td><td>{knn_name}</td><td>{clf_cell(knn_metrics)}</tr>'
548         f'<tr><td>ML bổ sung</td><td>{ada_name}</td><td>{clf_cell(ada_metrics)}</tr>'
549         f'<tr><td>Deep learning</td><td>{dl_label}</td><td>{dl_cells}</tr>'
550     )
551
552     house_base = _baseline(knn["house_price"])
553     house_knn = _new_model(knn["house_price"])
554     house_ada = _new_model(ada["house_price"])
555     house_best = next(m for m in house_dl["models"] if m["key"] == house_dl["best_model"])
556
557     house_rows = "".join(
558         f'<tr><td>{group}</td><td>{name}</td>'
559         f'<td class="num">{n(m["r2_log"])}</td>'
560         f'<td class="num">{n(m["mae_usd"])}</td>'
561         f'<td class="num">{m["mape_percent"]:.1f}</td></tr>'
562         for group, name, m in (
563             ("ML mốc", house_base[0].replace(" (mốc)", ""), house_base[1]),
564             ("ML bổ sung", house_knn[0], house_knn[1]),
565             ("ML bổ sung", house_ada[0], house_ada[1]),
566             ("Deep learning", f'{house_best["key"]} {house_best["label"]} ({house_best["shape"]})',
567              house_dl["test_metrics_dl"]),

```

```

568         )
569     )
570
571     return f"""
572 <h2 id="m12">12. Tổng hợp: ba mô hình ML và một mô hình DL cho mỗi category</h2>
573
574 <p>Bốn notebook bổ sung đưa mỗi category lên đủ ba mô hình machine learning và một mô hình
575 deep learning. Hai bảng dưới đây gom toàn bộ kết quả trên tập test về một chỗ.</p>
576
577 <div class="tcap" data-key="t27">Ba bài toán phân loại, toàn bộ mô hình, trên tập test</div>
578 <table>
579     <thead><tr>
580         <th style="width:14%">Category</th><th style="width:15%">Nhóm</th><th>Mô hình</th>
581         <th class="num" style="width:13%">F1-macro</th>
582         <th class="num" style="width:13%">ROC-AUC</th>
583     </tr></thead>
584     <tbody>{rows}</tbody>
585 </table>
586 <p class="tnote">(*) Mô hình deep learning của bài toán tiểu đường là mạng của notebook 04.
587 Nó dùng <b>tám</b> trong số 21 đặc trưng và chia tập theo tỉ lệ 80/20 thay vì 70/15/15, nên
588 con số của nó <b>không</b> so trực tiếp được với ba dòng phía trên; đặt cạnh nhau ở đây chỉ
589 để thấy phạm vi đã phủ. Bài toán nhóm sản phẩm chưa có mô hình deep learning riêng.</p>
590
591 <div class="tcap" data-key="t28">Bài toán hồi quy, toàn bộ mô hình, trên tập test</div>
592 <table>
593     <thead><tr>
594         <th style="width:16%">Nhóm</th><th>Mô hình</th>
595         <th class="num" style="width:14%">R<sup>2</sup></th><th><log></th>
596         <th class="num" style="width:16%">MAE (USD)</th>
597         <th class="num" style="width:13%">MAPE (%)</th>
598     </tr></thead>
599     <tbody>{house_rows}</tbody>
600 </table>
601
602 <p>Đọc ngang hai bảng thì thấy một khuôn mẫu nhất quán. Ở ba bài toán có đầu vào là bảng số
603 đã được đặc trưng hoá bằng tay, mô hình mốc của Assignment 02 vẫn đứng đầu và hai mô hình
604 bổ sung xếp dưới. Ở bài toán duy nhất có đầu vào là văn bản thô, mạng nơ-ron học biểu diễn
605 vượt lên. Đây không phải kết luận về việc thuật toán nào mạnh hơn, mà về việc mỗi thuật
606 toán giả định điều gì về dữ liệu, và giả định đó có đúng với dữ liệu đang có hay không.</p>
607
608 <div class="risk"><span class="lab">Điều báo cáo này không kết luận</span>
609 <b>Một, không có mô hình triển khai nào thay đổi.</b> Bốn notebook bổ sung mở rộng bảng so
610 sánh, không thay artifact nào đang phục vụ API.
611 <b>Hai, mọi số trong chương 9 đến 12 đo trên bộ dữ liệu hiện tại</b>, đã được bổ sung sau
612 khi notebook 01-03 chạy lần cuối, nên không so trực tiếp được với bảng của Assignment 02.
613 <b>Ba, siêu tham số chỉ được dò trên một lưới nhỏ</b> (giá trị k của KNN, số vòng của
614 AdaBoost, bốn kiến trúc mạng). Các con số là ước lượng trung thực cho đúng những cấu hình
615 đã thử, không phải cho cấu hình tốt nhất có thể của từng thuật toán.
616 </div>
617 """
618
619
620 def build_extra() -> str:
621     return "\n".join([ml_models(), dl_house(), dl_ecommerce(), summary_table()])

```

reports/build/build_report_dl.py

133 dòng

```

1     """Dung bao cao PDF cho Assignment 03 (deep learning tu dau voi NumPy).
2
3     python reports/build/build_report_dl.py
4
5     Dung lai toan bo ha tang cua build_report.py: tach hinh tu notebook, danh so
6     bang/hinh, sinh muc luc, nhưng anh thanh data URI va render bang Chrome + Paged.js.
7     Khac o ba cho: noi dung sinh tu content03.py theo so lieu that trong
8     reports/deeplearning_results.json, phu luc chi gom notebook 04, va ten tep dau ra.
9     """
10

```

```

11 from __future__ import annotations
12
13 import sys
14 from pathlib import Path
15
16 sys.path.insert(0, str(Path(__file__).resolve().parent))
17
18 from build_report import ( # noqa: E402
19     BUILD,
20     ROOT,
21     build_toc,
22     code_block,
23     extract_figures,
24     inline_images,
25     notebook_html,
26     number_captions,
27     render_pdf,
28 )
29 from content03 import build_content # noqa: E402
30
31 CONTENT_HTML = BUILD / "content03.html"
32 OUT_HTML = BUILD / "report03.rendered.html"
33 OUT_PDF = ROOT / "reports" / "Assignment03_BaoCao.pdf"
34 NOTEBOOK = ROOT / "notebooks" / "04_deep_learning.ipynb"
35 EXTRA_NOTEBOOKS = (
36     ("pl-b", "B", "Notebook 05: K-Nearest Neighbors", "notebooks/05_knn.ipynb"),
37     ("pl-c", "C", "Notebook 06: AdaBoost", "notebooks/06_adaboost.ipynb"),
38     ("pl-d", "D", "Notebook 07: Mạng nơ-ron hồi quy cho giá nhà",
39      "notebooks/07_deep_learning_house.ipynb"),
40     ("pl-e", "E", "Notebook 08: Mạng nơ-ron có lớp nhúng cho e-commerce",
41      "notebooks/08_deep_learning_ecommerce.ipynb"),
42 )
43 SOURCE_FILES = (
44     "ml/datasets.py",
45     "ml/benchmark.py",
46     "ml/metrics.py",
47     "ml/split.py",
48     "reports/build/content03.py",
49     "reports/build/content03_extra.py",
50     "reports/build/build_report_dl.py",
51 )
52 TITLE = "Assignment 03: Deep Learning from Scratch with NumPy"
53
54
55 def appendix() -> str:
56     parts = [
57         '<h2 id="pl-a">Phụ lục A. Notebook 04: Deep Learning từ đầu</h2>',
58         '<p class="small">Toàn bộ nội dung notebook, giữ nguyên thứ tự cell. Phần điển '
59         'giải là markdown gốc trong notebook, khối <code>In [n]</code> là mã nguồn, '
60         'khối <code>Out</code> là kết quả đã chạy thật. Mọi con số trong báo cáo đều '
61         'sinh từ <code>reports/deeplearning_results.json</code> mà notebook này ghi ra, '
62         'nên báo cáo không thể lệch với notebook.</p>',
63         notebook_html(NOTEBOOK),
64     ]
65
66     for anchor, letter, title, rel in EXTRA_NOTEBOOKS:
67         parts.append(f'<h2 id="{anchor}">Phụ lục {letter}. {title}</h2>')
68         parts.append(
69             '<p class="small">Toàn bộ nội dung notebook, giữ nguyên thứ tự cell. Mọi con số '
70             'trong chương tương ứng của báo cáo đều sinh từ tệp kết quả mà notebook này ghi '
71             'ra, nên báo cáo không thể lệch với notebook.</p>'
72         )
73         parts.append(notebook_html(ROOT / rel))
74
75     parts.append('<h2 id="pl-f">Phụ lục F. Mã nguồn dùng chung và kịch bản dựng báo cáo</h2>')
76     parts.append(
77         '<p class="small">Bốn tệp đầu là gói <code>ml</code>: nạp dữ liệu, làm sạch, chia '
78         '"70/15/15 và đo chỉ số, dùng chung cho cả bốn notebook bổ sung nên chúng chắc chắn '
79         '"đo trên cùng một tập train, val, test. Ba tệp cuối sinh ra chính tài liệu này: "

```

```

80         "<code>content03.py</code> và <code>content03_extra.py</code> đọc các tệp kết quả và "
81         "dùng phần thân, <code>build_report_dl.py</code> ghép phụ lục, đánh số bảng hình, "
82         "sinh mục lục và gọi Chrome headless để in PDF.</p>"
83     )
84     for rel in SOURCE_FILES:
85         path = ROOT / rel
86         n_lines = path.read_text(encoding="utf-8").count("\n") + 1
87         parts.append(
88             f'<div class="file-head">{rel}<span>{n_lines}</span> dòng</div>'
89             + code_block(path.read_text(encoding="utf-8"), "python")
90         )
91     return f'<section class="appendix">{"".join(parts)}</section>'
92
93
94 def main() -> int:
95     n_figs = extract_figures()
96     print(f" [figs] {n_figs} biểu đồ tách từ notebook", file=sys.stderr)
97
98     content = build_content()
99     CONTENT_HTML.write_text(content, encoding="utf-8")
100    print(f" [content] {CONTENT_HTML.name} - {len(content) / 1024:.0f} KB", file=sys.stderr)
101
102    body = number_captions(content + appendix())
103    toc = build_toc(body)
104
105    fonts = (BUILD / "fonts.css").read_text(encoding="utf-8")
106    css = (BUILD / "report.css").read_text(encoding="utf-8")
107    paged = (BUILD / "paged.polyfill.js").read_text(encoding="utf-8")
108
109    doc = f'<!doctype html>
110    <html lang="vi"><head><meta charset="utf-8">
111    <title>{TITLE}</title>
112    <style>{fonts}</style>
113    <style>{css}</style>
114    </head><body>
115    {body.replace("<!--TOC-->", toc)}
116    <script>window.PagedConfig = {{ auto: true, after: () => {{ window.__pagedDone = true; }} }};</script>
117    <script>{paged}</script>
118    </body></html>'
119
120    doc = inline_images(doc, BUILD)
121    OUT_HTML.write_text(doc, encoding="utf-8")
122    print(f" [html] {OUT_HTML.name} - {len(doc) / 1_048_576:.1f} MB", file=sys.stderr)
123
124    render_pdf(OUT_HTML, OUT_PDF)
125    raw = OUT_PDF.read_bytes()
126    pages = raw.count(b"/Type /Page\n") or raw.count(b"/Type/Page")
127    print(f" [pdf] {OUT_PDF} - {len(raw) / 1_048_576:.1f} MB, ~{pages} trang", file=sys.stderr)
128    return 0
129
130
131 if __name__ == "__main__":
132     raise SystemExit(main())

```